

微信搜索 **代码情缘** 关注瓜哥公众号，瓜哥带你编程上高速，免费领取海量资料。

CSDN 搜索 **极客江南** 关注瓜哥博客

哈喽，大家好，我是瓜哥，致力于为大家分享互联网各领域干货。

这篇文章可以说是一本书了，排版，码字耗费了瓜哥很长的时间，觉得有价值记得分享转发支持。

扫码关注获取资料



回复 C语言代码 获取配套代码

回复 书籍 获取瓜哥精华电子书

回复 微信 添加瓜哥微信交流

什么是Go语言

- Go语言(Golang)是Google公司2009年推出的一门"高级编程言语", 目的是为了解决:
 - "现有主流编程语言"明显落后于硬件发展速度的问题
 - 不能合理利用多核CPU的优势提升软件系统性能的问题
 - 软件复杂度越来越高, 维护成本也越来越高的问题
 - 企业开发中不得不在快速开发和性能之间艰难抉择的问题

科普小知识:

1.静态语言:

1.1一般都需要通过编译器 (compiler) 将源代码翻译成机器码, 之后才能执行。程序被编译之后无论是程序中的数据类型还是程序的结构都不可以被改变

1.2静态语言的性能和安全性都非常好, 例如C和C++、Go, 但是C和C++的缺点是开发速度慢, 维护成本高

2.动态语言

2.1一般不需要通过编译器将源代码翻译成机器码, 在运行程序的时候才逐行翻译。程序在运行的过程中可以动态修改程序中的数据类型和程序的结构

2.2动态语言开发速度快, 维护成本低, 例如Ruby和Python, 但是Ruby和Python的性能和安全性又略低

- Go语言专门针对多核CPU进行了优化, 能够充分使用硬件多核CPU的优势, 使得通过Go语言编写的软件系统性能能够得到很大提升
- Go语言编写的程序, 既可以媲美C或C++代码的运行速度, 也可以媲美Ruby或Python开发的效率
- 所以Go语言很好的解决了"现有主流编程语言"存在的问题, 被誉为"现代化的编程语言"

Go语言优势

- 简单易学
 - Go语言的作者都有C的基因，Go自然而然也有了C的基因，但是Go的语法比C还简单，并且几乎支持大多数你在其他语言见过的特性：封装、继承、多态、反射等
- 丰富的标准库
 - Go目前已经内置了大量的库，特别是网络库非常强大
 - 前面说了作者是C的作者，所以Go里面也可以直接包含c代码，利用现有的丰富的C库
- 跨平台编译和部署
 - Go代码可直接编译成机器码，不依赖其他库，部署就是扔一个文件上去就完事了. 并且Go代码还可以做到跨平台编译(例如: window系统编译linux的应用)
- 内置强大的工具
 - Go语言里面内置了很多工具链，最好的应该是gofmt工具，自动化格式化代码，能够让团队review变得如此的简单，代码格式一模一样，想不一样都很困难
- 性能优势: Go 极其地快。其性能与 C 或 C++相似。在我们的使用中，Go 一般比 Python 要快 30 倍左右
 - 语言层面支持并发，这个就是Go最大的特色，天生的支持并发，可以充分的利用多核，很容易的使用并发
 - 内置runtime，支持垃圾回收
-

Go语言的吉祥物是地鼠

地鼠的特点是速度快、成群结队、头脑简单

而Go语言的特点正好也是编程速度快、并发性好、简单易学



I'm Golang(够浪)



信链App

Go语言发展史

2007年，谷歌工程师Rob Pike，Ken Thompson和Robert Griesemer开始设计一门全新的语言，这是Go语言的最初原型。

2009年11月10日，Go语言以开放源代码的方式向全球发布。

2011年3月16日，Go语言的第一个稳定(stable)版本r56发布。

2012年3月28日，Go语言的第一个正式版本Go1发布。

2013年4月04日，Go语言的第一个Go 1.1beta1测试版发布。

2013年4月08日，Go语言的第二个Go 1.1beta2测试版发布。

2013年5月02日, Go语言Go 1.1RC1版发布。
2013年5月07日, Go语言Go 1.1RC2版发布。
2013年5月09日, Go语言Go 1.1RC3版发布。
2013年5月13日, Go语言Go 1.1正式版发布。
2013年9月20日, Go语言Go 1.2RC1版发布。
2013年12月1日, Go语言Go 1.2正式版发布。
2014年6月18日, Go语言Go 1.3版发布。
2014年12月10日, Go语言Go 1.4版发布。
2015年8月19日, Go语言Go 1.5版发布, 本次更新中移除了”最后残余的C代码”。
2016年2月17日, Go语言Go 1.6版发布。
2016年8月15日, Go语言Go 1.7版发布。
2017年2月17日, Go语言Go 1.8版发布。
2017年8月24日, Go语言Go 1.9版发布。
2018年2月16日, Go语言Go 1.10版发布。

Go作者

- Go语言是UNIX作者、C语言作者、谷歌V8引擎作者携手打造的, 由谷歌公司2009年推出的一门高级编程语言。

GO语言核心开发团队



罗布·派克 (Rob Pike)
□ Unix小组成员, 参与Plan9和Inferno操作系统
□ Limbo语言和UTF-8编码的主要设计者
□ 《Unix编程环境》《编程实践》作者之一



伊安·泰勒 (Ian Lance Taylor)
□ GCC社区的活跃人物, gold连接器和GCC过程间优化LTO的主要设计者



肯·汤普逊 (Ken Thompson)
□ 设计了B语言和C语言
□ Unix和Plan9操作系统创立者之一
□ 1983年图灵奖得主



布拉德·菲茨帕特里克 (Brad Fitzpatrick)
□ SNS交友网站 LogoLiveJournal创始人
□ Memcached和MogileFS开源项目作者



罗伯特·格里泽默 (Robert Griesemer)
□ 曾参与制作了Java的HotSpot编译器以及Chrome浏览器的JavaScript引擎V8



拉斯·考克斯 (Russ Cox)
□ 参与Plan9操作系统开发
□ Google Code Search项目负责人
□ 将Plan9操作系统的运行环境几乎完整的移植到Linux, FreeBSD和MacOSX上

跟着谷歌走吃喝啥都有

Go语言现状

- 多次获得TIOBE年度最佳语言

Programming Language Hall of Fame

The hall of fame listing all "Programming Language of the Year" award winners is shown below. The award is given to the programming language that has the highest rise in ratings in a year.

年度最佳语言

Year	Winner
2017	🏆 C
2016	🏆 Go
2015	🏆 Java
2014	🏆 JavaScript
2013	🏆 Transact-SQL
2012	🏆 Objective-C
2011	🏆 Objective-C
2010	🏆 Python
2009	🏆 Go
2008	🏆 C
2007	🏆 Python
2006	🏆 Ruby
2005	🏆 Java
2004	🏆 PHP
2003	🏆 C++

- 2018年Go语言一度超过Java, [进入编程语言排行榜前三名](#).

July Highlights

Sunday, July 15, 2018

For most months, this post examines technologies across the spectrum of software development, from languages to frameworks to products. And, that data is all still here for you to examine as well. But, I thought it would be interesting to take a look at the state of Programming Languages specifically as their trends can often be obscured in the overall analysis.

Top 10 Programming Languages

1. Python
2. JavaScript
3. Go
4. Ruby
5. Java
6. C++
7. PHP
8. Scala
9. Swift
10. Kotlin

- 从公司角度:
 - [许多大厂](#)都已经拥抱 Go 语言, 包括阿里巴巴、京东、今日头条、小米、滴滴、七牛云、360等明星公司, 也包括知乎、轻松筹、快手、探探、美图、猎豹移动等等。同时, 创业公司也很喜欢 Go 语言, 主要因为其入门快、程序库多、运行迅速, 很适合快速构建互联网软件产品。
- 从业务维度:
 - Go 程序可以在装有 Windows、Linux、FreeBSD 等操作系统的服务器上运行, 并用于提供基础软件支撑、API 服务、Web 服务、网页服务等等。
 - 在云计算、微服务、大数据、区块链、物联网等领域, Go 语言早已蓬勃发展. 除了语法简单, 性能优越以外, K8S底层架构在云计算的领导地位(K8S就是Go开发的), 也让这些各大公司不得不拥抱Go语言。

- 区块链的崛起更进一步带动了Go工程师的需求,市面上大部分区块链明星项目都是用Go开发的,足以说明Go在分布式系统中的地位,这也就是为什么今年开始,大批金融公司开始招聘Go工程师的重要原因。
- 从薪资角度来看
 - 应届生普遍在 4~8K, 1年左右普遍在 10K左右, 2年~3年普遍在 20K左右

热门职位	最新职位
实习生 后端开发 Gol... 4k-8k 北京 海淀区 清河 应届生 本科	小米 互联网 已上市 10000人以上 卿吉明 招聘者 发布于07月24日
golang工程师 (实习) 6k-12k 北京 朝阳区 望京 应届生 本科	云账户 移动互联网 A轮 100-499人 cat 招聘经理 发布于09月11日
golang后端实习生 5k-8k 北京 海淀区 上地 应届生 本科	滴滴出行 移动互联网 D轮及以上 10000人以上 施剑波 招聘者 发布于08月22日
测试开发实习生 4k-8k 北京 东城区 王府井 应届生 本科	今日头条 移动互联网 D轮及以上 10000人以上 李婧 招聘者 发布于08月23日
Golang 4k-8k 北京 朝阳区 望京 应届生 本科	映客直播 移动互联网 已上市 500-999人 王振涛 创新业务线... 发布于07月19日
热门职位	最新职位
Golang开发工程师 25k-40k 北京 海淀区 上地 1-3年 本科	滴滴出行 移动互联网 D轮及以上 10000人以上 杨硕 资深专家开发... 发布于08月20日
Golang 25k-40k 北京 海淀区 双榆树 1-3年 本科	今日头条 移动互联网 D轮及以上 10000人以上 陈杰 工程师 发布于05月30日
Golang 20k-40k 北京 大兴区 1-3年 本科	京东集团 电子商务 已上市 10000人以上 屈先生 招聘者 发布于08月15日
Golang 开发工程师 (... 30k-50k 北京 1-3年 本科	知乎 移动互联网 D轮及以上 1000-9999人 段永亮 招聘者 发布于08月10日
go开发工程师 15k-25k 北京 海淀区 知春路 1-3年 大专	掌游天下 移动互联网 已上市 100-499人 蔡旭昊 招聘者 发布于09月07日
高级Golang研发工程师 20k-40k 北京 朝阳区 高碑店 1-3年 本科	猎豹移动 互联网 已上市 1000-9999人 张荣亮 研发总监 发布于07月19日
Go 软件工程师 15k-30k 北京 东城区 北新桥 1-3年 本科	轻松筹 互联网 C轮 500-999人 郝林 大数据负责人 发布于04月15日

Go语言应用场景

- 网络编程, 这一块目前应用最广, 包括Web应用、API应用、下载应用、内存数据库等
- 云平台开发, 目前国外很多云平台在采用Go开
- 服务器编程, 以前你如果使用Java或者C++做的那些事情, 都可以用Go来做
- 分布式系统, 数据库代理器等
- 它可以做从底层到前端的任何工作

如何学习Go语言

- Go语言被称之为现代化的C语言, 所以无论是从语法特性, 还是作者本身, Go语言都与C语言有着莫大的关系, 所以学习本套课程之前如果你有C语言的基础, 那么将会事半功倍
- 对于初学者而言, 学习编程的捷径只有一条, 那就是**多动手**

竹子用了4年的时间, 仅仅长了3cm, 从第五年开始, 以每天30cm的速度疯狂地生长, 仅仅用了六周的时间就长到了15米。其实, 在前面的四年, 竹子将根在土壤里延伸了数百平米。做人做事亦是如此, 不要担心你此时此刻的付出得不到回报, 因为这些付出都是为了扎根。

源文件对比

- C语言源文件

文件扩展名	源类型
.h	头文件, 存放代码声明
.c	C语言源文件, 存放代码实现

- Go语言源文件

文件扩展名	源类型
.go	Go语言源文件, 存放代码实现

代码管理对比

- C语言中通过文件来管理代码
 - 想使用某一个函数时,只需要include导入对应的.h文件即可
- Go语言中通过包来管理代码
 - Go语言没有.h文件的概念, 在Go中想使用某一个函数时, 只需要import导入对应的包即可
- C语言中函数、变量公私有管理
 - 通过extern和static实现是否公开函数和变量
- Go语言中函数、变量公私有管理
 - 通过函数名称首字母大小写实现是否公开函数
 - 通过变量名称首字母大小写实现是否公开变量

关键字对比

- C语言中一共有32个关键字

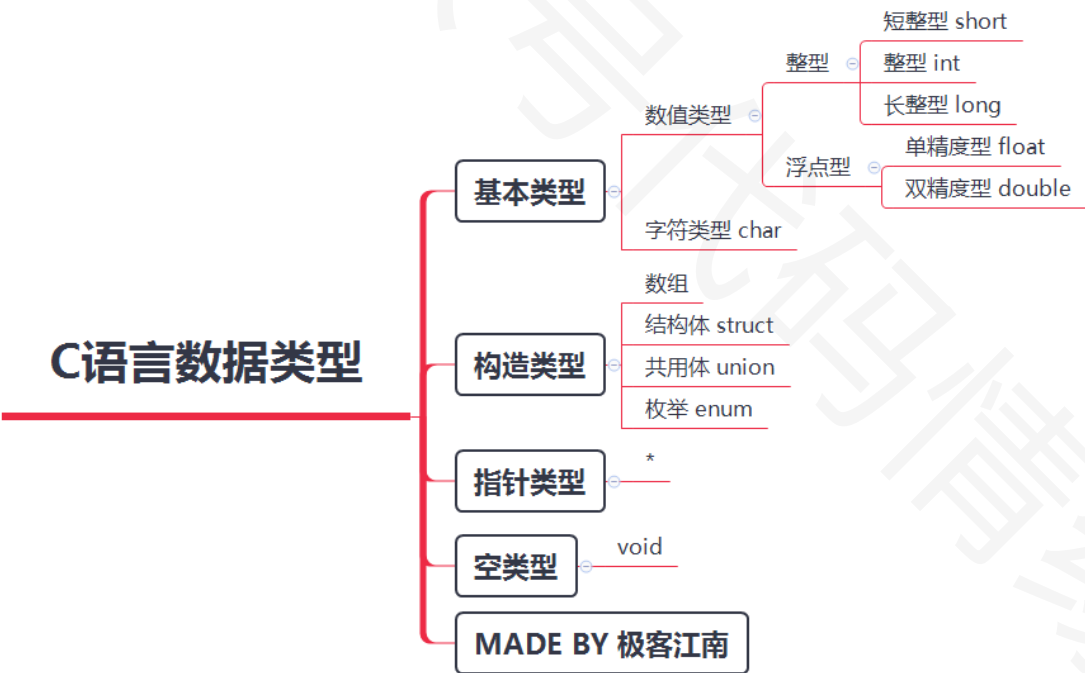
1	2	3	4	5	6	7	8
<i>if</i>	<i>else</i>	<i>switch</i>	<i>case</i>	<i>default</i>	<i>break</i>	<i>return</i>	<i>goto</i>
do	while	<i>for</i>	<i>continue</i>	typedef	<i>struct</i>	enum	union
char	short	int	long	float	double	void	sizeof
signed	unsigned	<i>const</i>	auto	register	static	extern	volatile

- Go语言中一共有25个关键字

1	2	3	4	5	6	7	8
<i>if</i>	<i>else</i>	<i>switch</i>	<i>case</i>	<i>default</i>	<i>break</i>	<i>return</i>	<i>goto</i>
fallthrough	<i>for</i>	<i>continue</i>	type	<i>struct</i>	var	<i>const</i>	map
func	interface	range	import	package	defer	go	select
chan							

数据类型对比

- C语言数据类型



• Go语言数据类型



• C语言各数据类型占用内存空间

类型	32位编译器	64位编译器
char	1	1
int	4	4
float	4	4
double	8	8
short	2	2
long	4	8
long long	8	8
void*	4	8

• Go语言各数据类型占用内存空间

类型	32位编译器	64位编译器	本质
int8/uint8	1	1	signed char/unsigned char
int16/uint16	2	2	signed short/unsigned short
int32/uint32	4	4	signed int/unsigned int
int64/uint64	8	8	signed long long int/unsigned long long int
byte	1	1	uint8/unsigned char
rune	4	4	int32/signed int
int	4	8	根据机器位数决定长度
uintptr	4	8	根据机器位数决定长度 uint32/uint64
float32	4	4	float
float64	8	8	double
true	1	1	char类型的整型
false	1	1	char类型的整型

- 和C语言一样,Go语言也提供了Sizeof计算变量的内存空间

- 1.导入import "unsafe"包
- 2.通过unsafe.Sizeof()计算变量内存空间

- Go语言基本数据类型内部实现

- [golang官方网站下载go1.4版本源代码](#)
 - 越老版本的代码越纯粹,越适合新手学习
 - 随着代码的更新迭代会逐步变得非常复杂, 所以此处建议下载1.4版本
- 解压后打开路径: go\src\runtime\runtime.h



- 得到如下实现代码

```
// 第8行到35行
typedef signed char      int8;
typedef unsigned char    uint8;
typedef signed short     int16;
typedef unsigned short   uint16;
typedef signed int       int32;
typedef unsigned int     uint32;
typedef signed long long int64;
typedef unsigned long long uint64;
```

```

typedef float          float32;
typedef double         float64;

#ifdef _64BIT
typedef uint64         uintptr;
typedef int64          intptr;
typedef int64          intgo; // Go's int
typedef uint64         uintgo; // Go's uint
#else
typedef uint32         uintptr;
typedef int32          intptr;
typedef int32          intgo; // Go's int
typedef uint32         uintgo; // Go's uint
#endif

#ifdef _64BITREG
typedef uint64         uintreg;
#else
typedef uint32         uintreg;
#endif

// 第153行到157行
enum
{
    true    = 1,
    false   = 0,
};

```

install B 时刻:

Go本质就是用C语言编写的一门高级编程语言

所以江哥前面教你C语言就是为了今天能让你看懂Go的实现代码,做到知其然知其所以然

常量变量对比

- C语言定义常量和变量格式

```

数据类型 变量名称 = 值;
const 数据类型 常量名称 = 值;

```

- Go语言定义常量和变量格式
 - 除了以下标准格式外,Go语言还提供了好几种简单的语法糖

```

var 变量名称 数据类型 = 值;
const 变量名称 数据类型 = 值;

```

注释对比

- 和C语言一样,Go语言也支持单行注释和多行注释, 并且所有注释的特性都和C语言一样
 - 单行注释 `// 被注释内容`
 - 多行注释 `/* 被注释内容*/`
- 在Go语言中,官方更加推荐使用单行注释,而非多行注释(详情可以直接查看Go官方源码)

运算符对比

- 算数运算符和C语言几乎一样
 - Go语言中++、--运算符不支持前置
 - 错误写法: ++i; --i;
 - Go语言中++、--是语句,不是表达式,所以必须独占一行
 - 错误写法: a = i++; return i++;

运算符	描述	实例
+	相加	A + B
-	相减	A - B
*	相乘	A * B
/	相除	B / A
%	求余	B % A
++	自增	A++
--	自减	A--

- 关系算符和C语言一样

运算符	描述	实例
==	检查两个值是否相等, 如果相等返回 True 否则返回 False。	A == B
!=	检查两个值是否不相等, 如果不相等返回 True 否则返回 False。	A != B
>	检查左边值是否大于右边值, 如果是返回 True 否则返回 False。	A > B
<	检查左边值是否小于右边值, 如果是返回 True 否则返回 False。	A < B
>=	检查左边值是否大于等于右边值, 如果是返回 True 否则返回 False。	A >= B
<=	检查左边值是否小于等于右边值, 如果是返回 True 否则返回 False。	A <= B

- 逻辑运算符和C语言一样

运算符	描述	实例
&&	如果两边的操作数都是 True, 则条件 True, 否则为 False。	A && B
	如果两边的操作数有一个 True, 则条件 True, 否则为 False。	A B
!	如果条件为 True, 则逻辑 NOT 条件 False, 否则为 True。	!A

- 位运算符和C语言几乎一样
 - 新增一个&^运算符

运算符	描述	实例
&	参与运算的两数各对应的二进位相与, 对应位只要都是1结果就为1	A & B
	参与运算的两数各对应的二进位相或,对应位只要其中一个是1结果就为1	A B
^	参与运算的两数各对应的二进位相异或,对应位只要不同结果就是1	A ^ B
<<	左移运算符,左移n位就是乘以2的n次方	A << 2
>>	右移运算符,右移n位就是除以2的n次方	B >> 2
&^	逻辑清零运算符, B对应位是1,A对应位清零,B对应位是0, A对应位保留原样	A &^ B

```

int main(){
    /*
        0110      a
        &^1011     b 如果b位位1,那么结果为0, 否则结果为a位对应的值
        -----
        0100
    */
    a1 := 6
    b1 := 11
    res1 := a1 &^ b1
    fmt.Println("res1 = ", res1) // 4

    /*
        1011      a
        &^1101     b 如果b位位1,那么结果为0, 否则结果为a位对应的值
        -----
        0010
    */
    a2 := 11
    b2 := 13
    res2 := a2 &^ b2
    fmt.Println("res2 = ", res2) // 2
}

```

- 赋值运算符和C语言几乎一样
 - 新增一个&^=运算符

运算符	描述	实例
=	将右边赋值给左边	C = A + B 将 A + B 表达式结果赋值给 C
+=	相加后再赋值	C += A 等于 C = C + A
-=	相减后再赋值	C -= A 等于 C = C - A
*=	相乘后再赋值	C *= A 等于 C = C * A
/=	相除后再赋值	C /= A 等于 C = C / A
%=	求余后再赋值	C %= A 等于 C = C % A
<<=	左移赋值	C <<= 2 等于 C = C << 2
>>=	右移赋值	C >>= 2 等于 C = C >> 2
&=	位逻辑与赋值	C &= 2 等于 C = C & 2
^=	位逻辑或赋值	C ^= 2 等于 C = C ^ 2
=	位逻辑异或赋值	C = 2 等于 C = C 2
&^=	位逻辑清零赋值	C &^= 2 等于 C = C &^ 2

流程控制语句对比

- C语言流程控制中的if、switch、for在Go语言都可以使用
- C语言中的四大跳转语句return、break、continue、goto在Go语言都可以使用
- Go语言除了实现C语言中if、switch、for、return、break、continue、goto的基本功能以外,还对if、switch、for、break、continue进行了增强
 - 例如: if 条件表达式前面可以添加初始化表达式
 - 例如: break、continue可以指定标签
 - 例如: switch语句可以当做if/elseif来使用
 -
- 值得注意的是Go语言中没有while循环和dowhile循环, 因为它们能做的Go语言中的for循环都可以做

函数和方法对比

- C语言定义函数格式

```
返回值类型 函数名称(形参列表) {
    函数体相关语句;
    return 返回值;
}
```

- Go语言定义函数格式

```
func 函数名称(形参列表)(返回值列表) {  
    函数体相关语句;  
    return 返回值;  
}
```

- C语言中没有方法的概念, 但是Go语言中有方法
 - 对于初学者而言, 可以简单的把方法理解为一种特殊的函数

```
func (接收者 接受者类型)函数名称(形参列表)(返回值列表) {  
    函数体相关语句;  
    return 返回值;  
}
```

编程思想对比

- C语言是一门面向过程的编程语言
 - 面向过程: 按部就班, 亲力亲为, 关注的是我应该怎么做?
 - 做饭例子: 面向过程做饭
 - 1.上街买菜
 - 2.摘菜
 - 3.洗菜
 - 4.切菜
 - 5.开火炒菜
 - 6.淘米煮饭
 - 7.吃饭
- Go语言是一门面向对象的编程语言
 - 面向对象: 化繁为简, 能不自己干自己就不干, 关注的是我应该让谁来做?
 - 做饭例子: 面向对象做饭
 - 1.找个会做饭女朋友 or 男朋友
 - 2.老婆我饿了 or 老公我饿了
 - 3.躺着...等她/他把饭做好
 - 4.吃饭
- 不要把面向过程和面向对象想象得那么神奇, 它们只是思考问题的方式不同而已

其它新增特性

- 接口
- 并发
- 反射
- 异常处理
-

Go语言SDK安装和配置

- 什么是SDK
 - 软件开发工具包 (外语首字母缩写: **SDK**、外语全称: **SoftwareDevelopmentKit**) 一般都是一些软件工程师为特定的软件包、软件框架、硬件平台、操作系统等建立应用软件时的开发工具的集合

- 如果不安装SDK, 你可以编写Go语言代码, 但是你不能编译执行编写好的Go语言代码
- 如何安装?
- 1. 下载SDK安装包。地址: <https://golang.google.cn/dl/>
 - 由于新版本一般不太稳定, 所以我们选择下载上一个版本

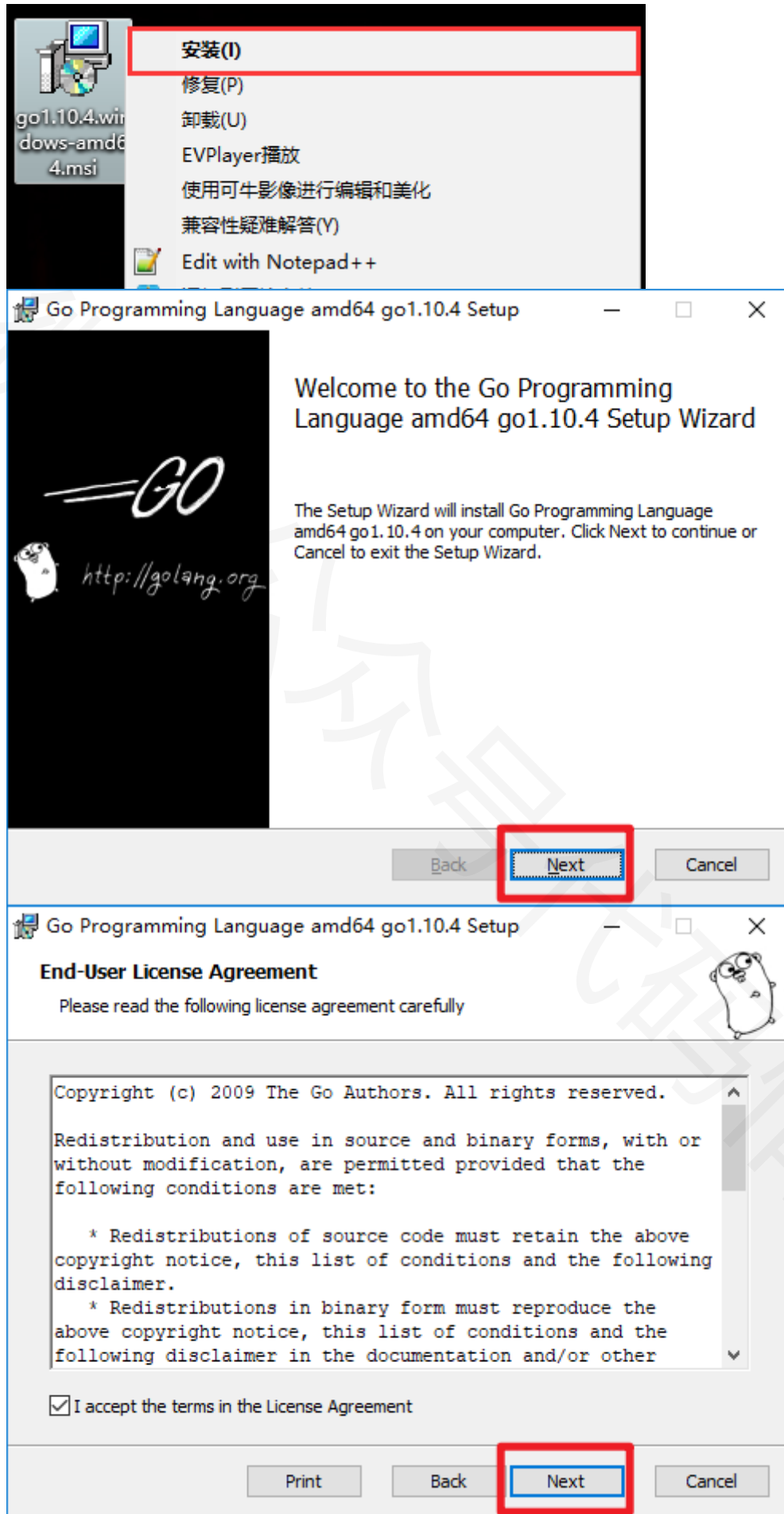
Stable versions

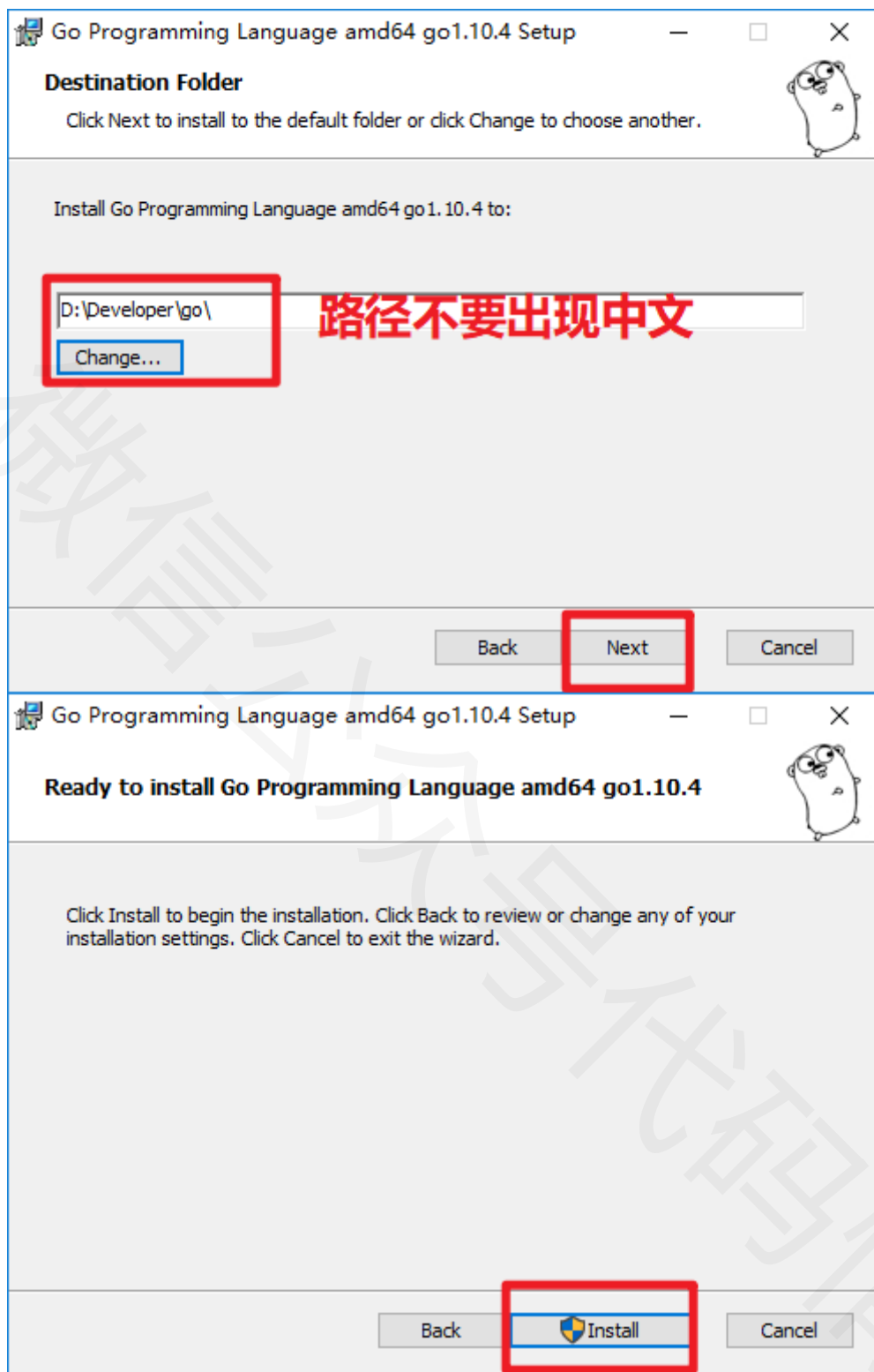
go1.11 ▸

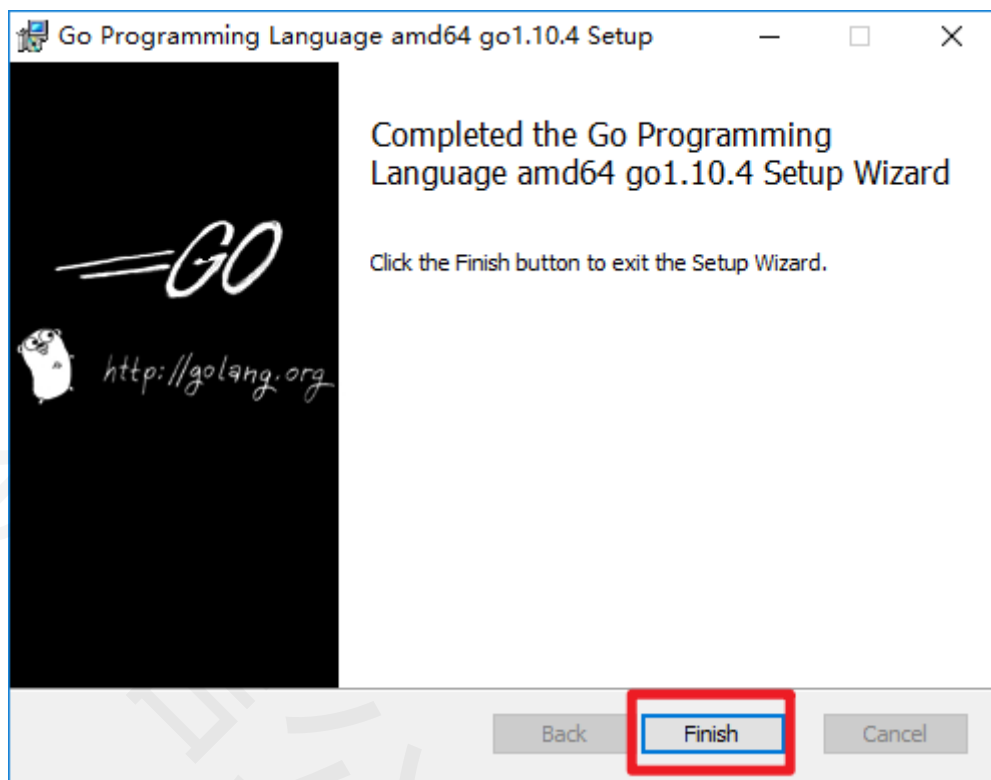
go1.10.4 ▾

File name	Kind	OS	Arch	Size	
go1.10.4.src.tar.gz	Source			17MB	源代码
go1.10.4.darwin-amd64.tar.gz	Archive	macOS	x86-64	86MB	Mac手动安装
go1.10.4.darwin-amd64.pkg	Installer	macOS	x86-64	86MB	Mac图形化安装包
go1.10.4.linux-386.tar.gz	Archive	Linux	x86	77MB	Linux32位安装包
go1.10.4.linux-amd64.tar.gz	Archive	Linux	x86-64	88MB	Linux64位安装包
go1.10.4.linux-armv6l.tar.gz	Archive	Linux	ARMv6	72MB	
go1.10.4.windows-386.zip	Archive	Windows	x86	82MB	Win32位手动安装包
go1.10.4.windows-386.msi	Installer	Windows	x86	67MB	Win32位图形化安装包
go1.10.4.windows-amd64.zip	Archive	Windows	x86-64	94MB	Win64位手动安装包
go1.10.4.windows-amd64.msi	Installer	Windows	x86-64	78MB	Win64位图形化安装包
Other Ports					
go1.10.4.freebsd-386.tar.gz	Archive	FreeBSD	x86	73MB	
go1.10.4.freebsd-amd64.tar.gz	Archive	FreeBSD	x86-64	84MB	
go1.10.4.linux-arm64.tar.gz	Archive	Linux	ARMv8	72MB	
go1.10.4.linux-ppc64le.tar.gz	Archive	Linux	ppc64le	71MB	
go1.10.4.linux-s390x.tar.gz	Archive	Linux	s390x	70MB	

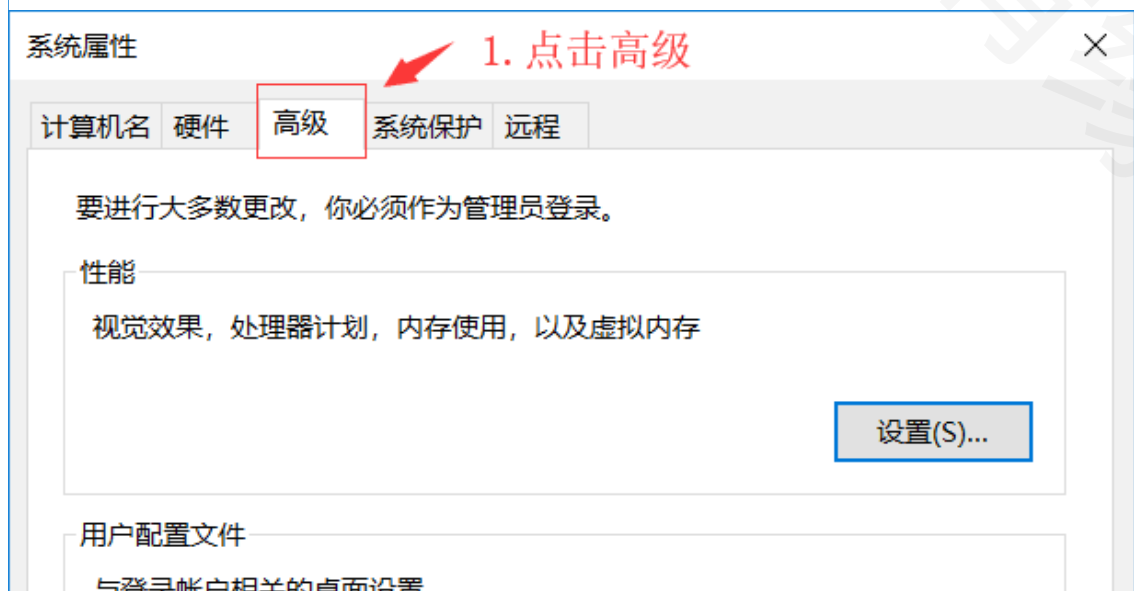
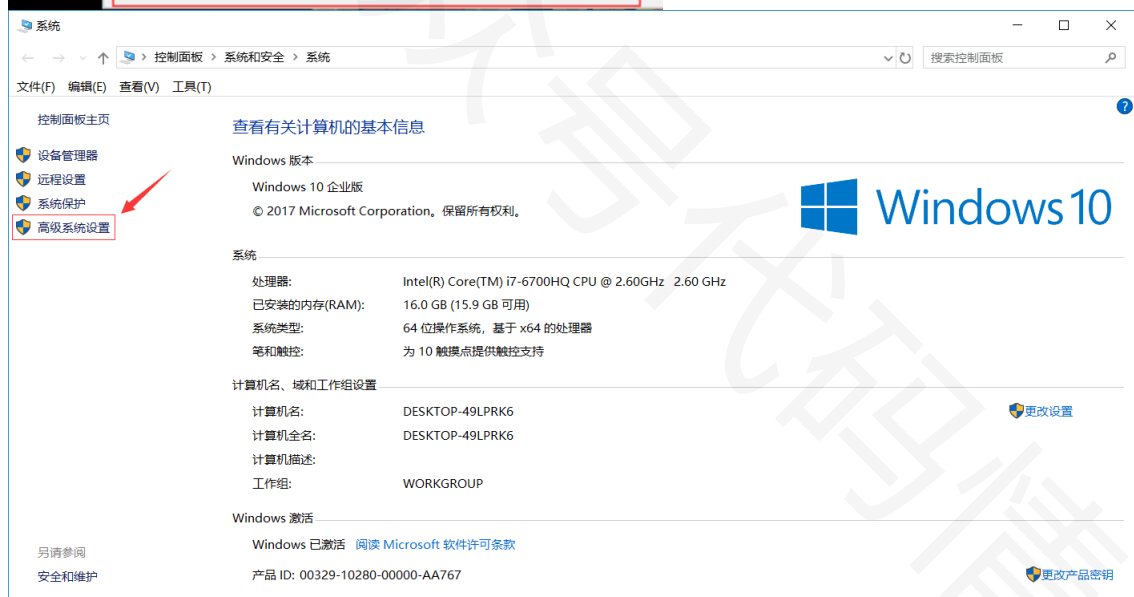
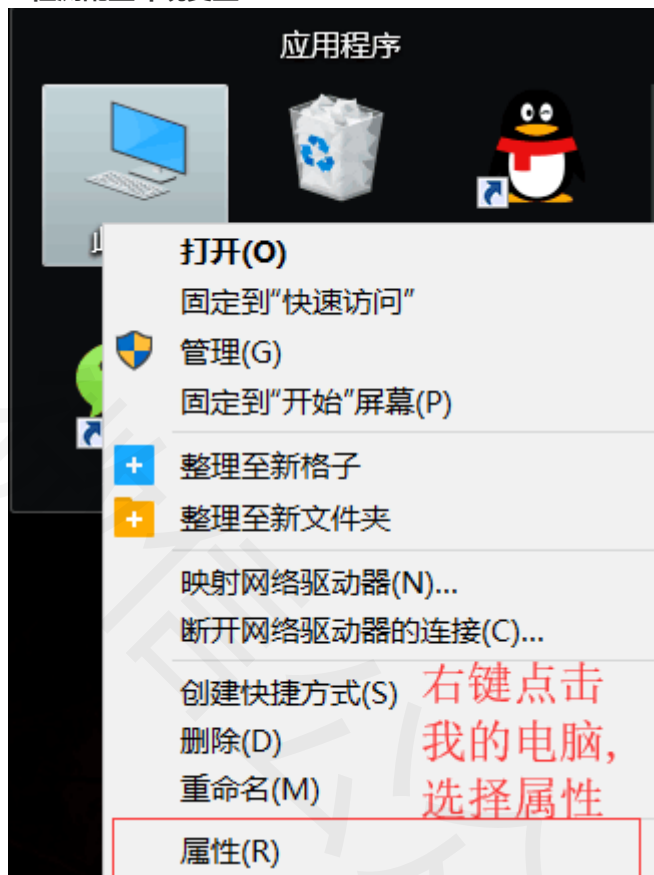
- 2.运行图形化安装包







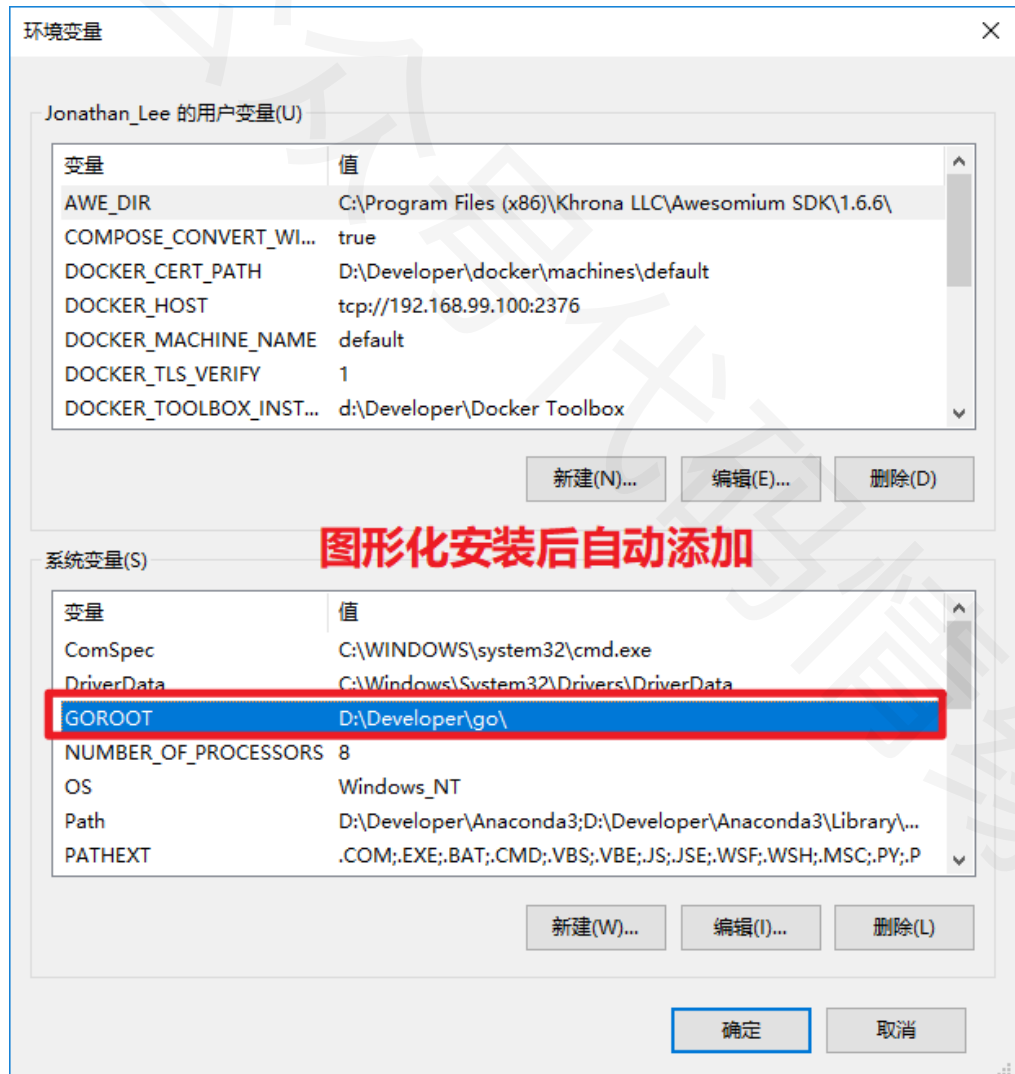
- 3.检测配置环境变量





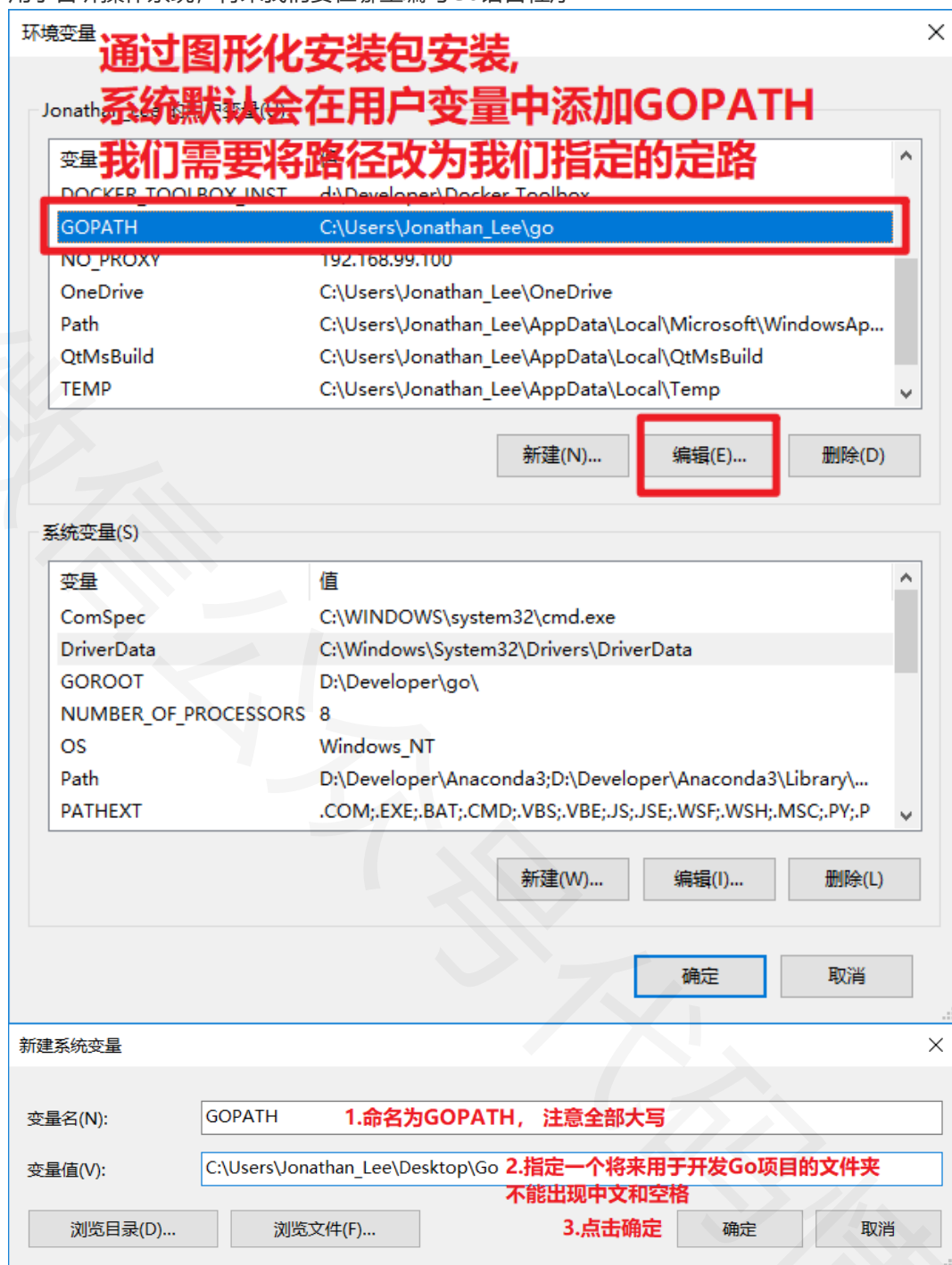
3.1. 添加GOROOT环境变量

- 用于告诉操作系统，我们把Go语言SDK安装到哪了



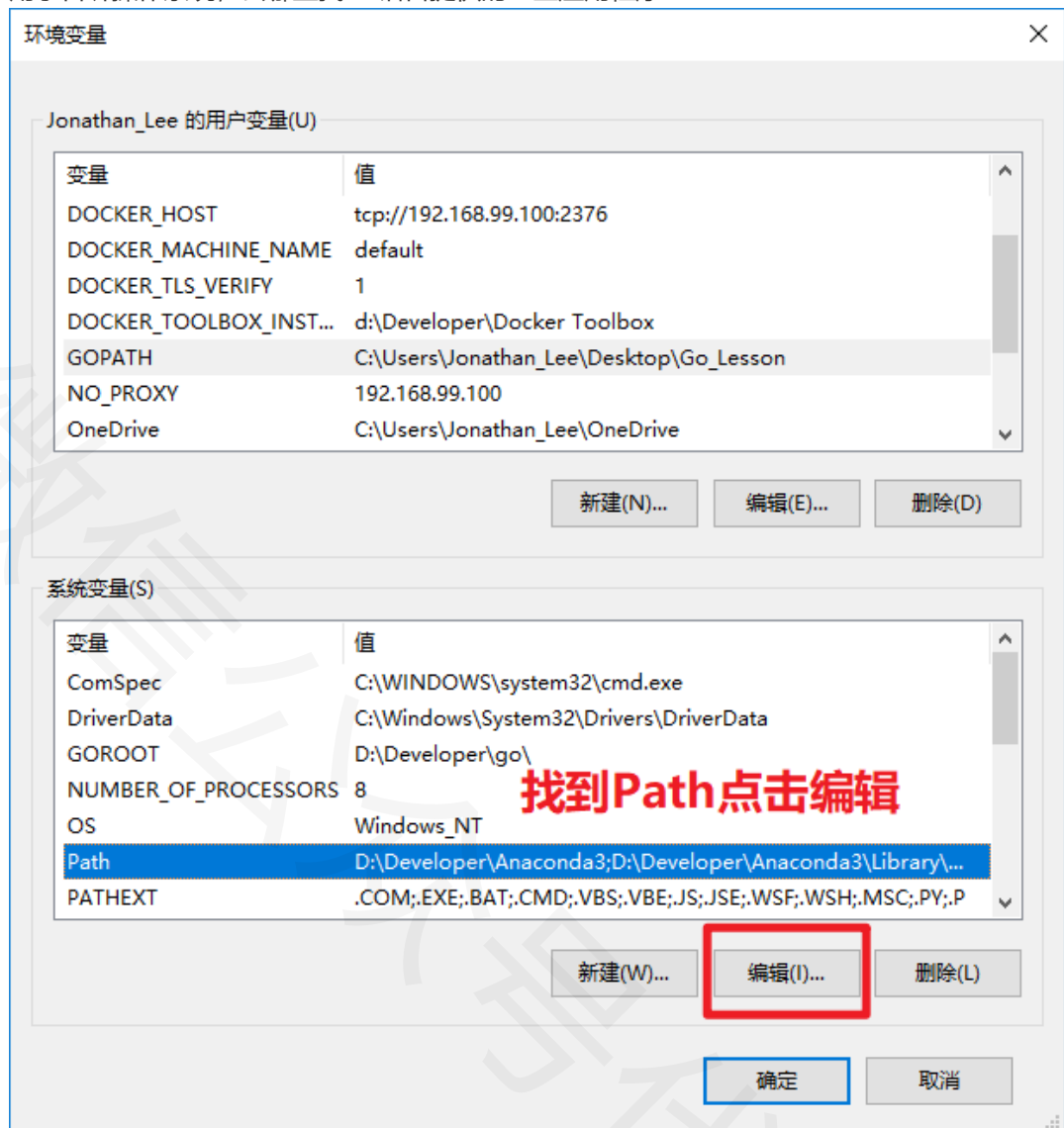
3.2. 配置GOPATH环境变量

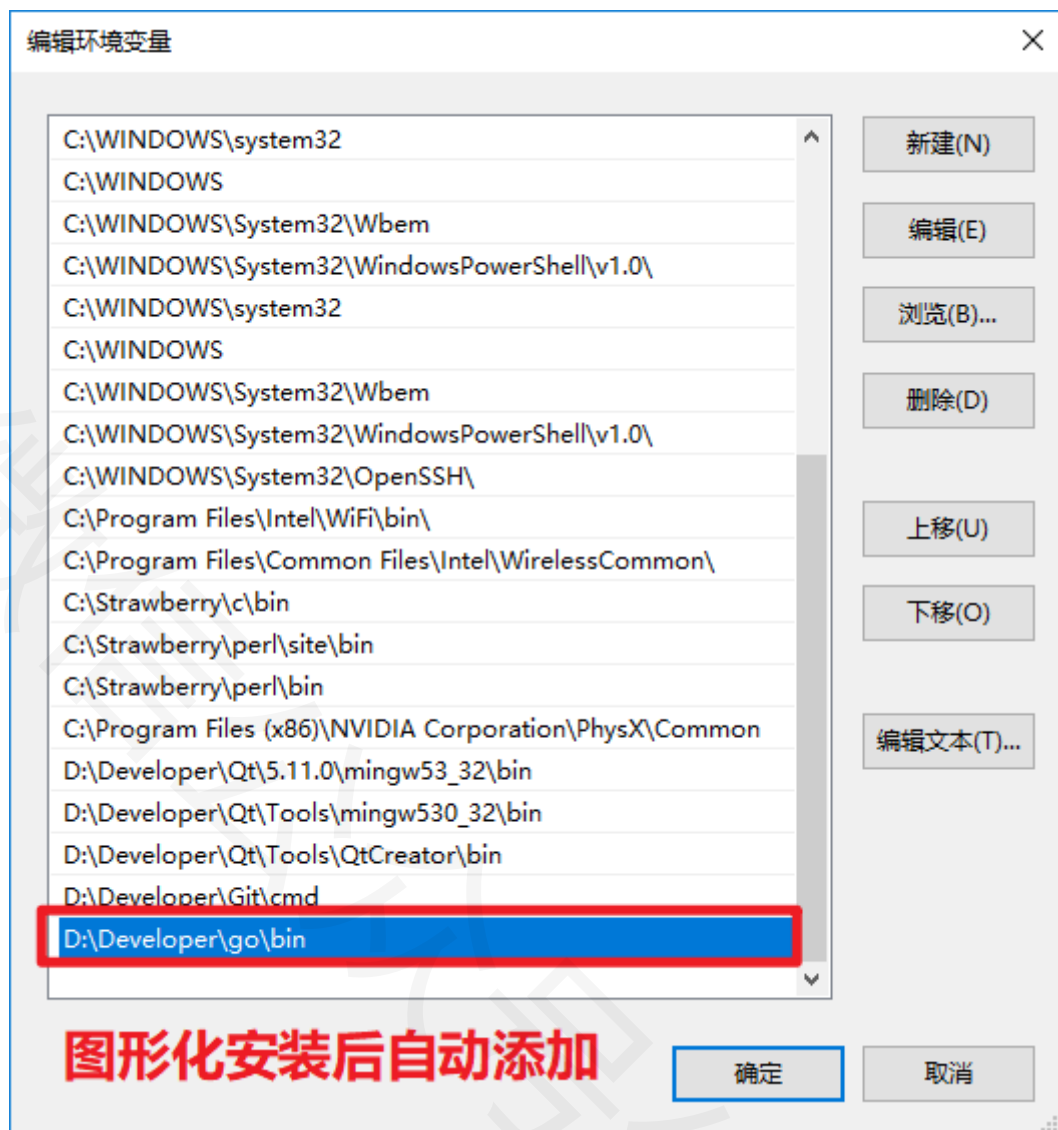
- o 用于告诉操作系统，将来我们要在哪里编写Go语言程序



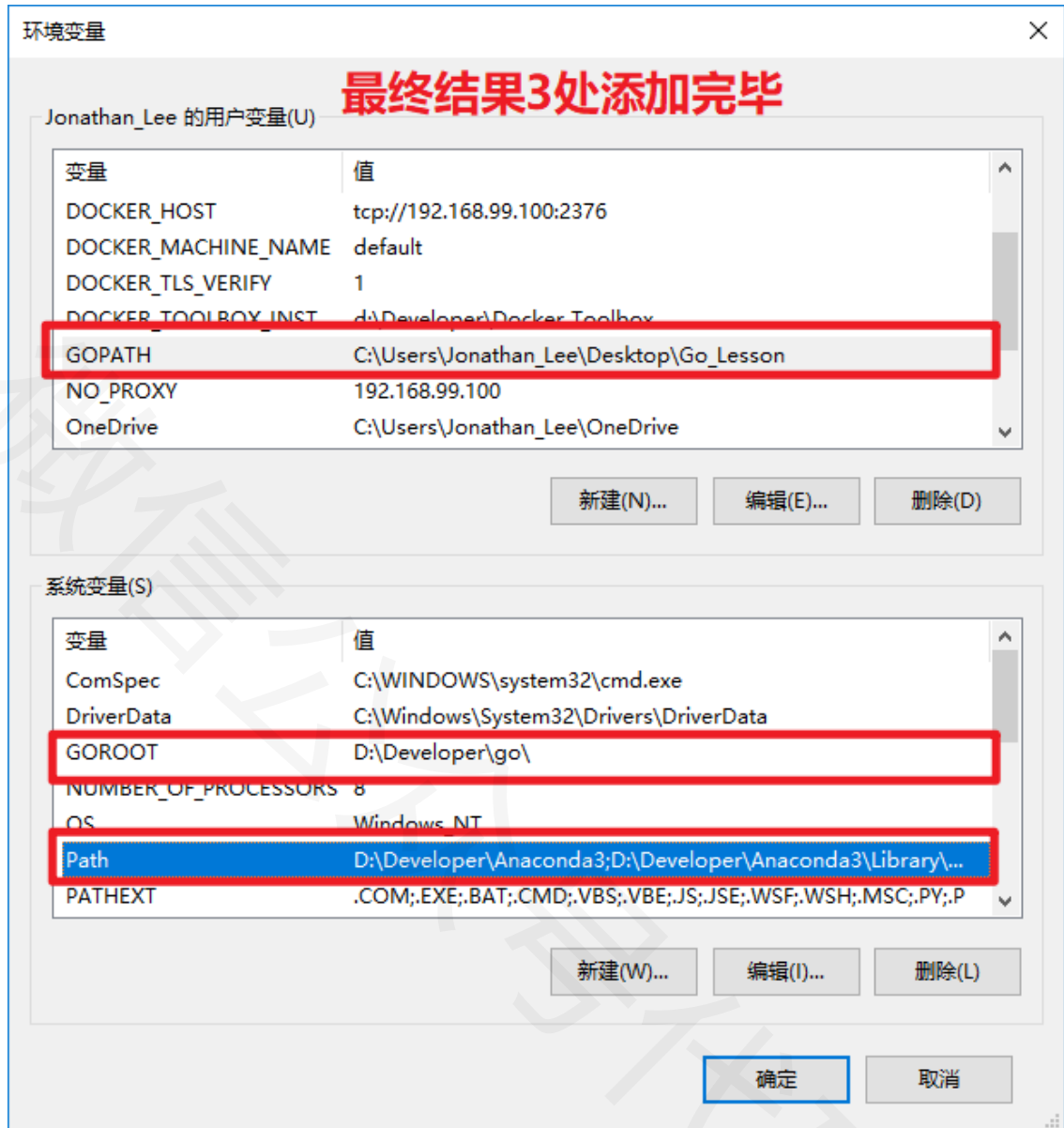
3.3.配置GoBin环境变量

- o 用于告诉操作系统，去哪查找Go语言提供的一些应用程序





- 最终结果



- 4. 检查是否安装配置成功

o 4.1 打开CMD



- o 4.2输入 go version

```
命令提示符
Microsoft Windows [版本 10.0.17134.228]
(c) 2018 Microsoft Corporation。保留所有权利。

C:\Users\Jonathan_Lee>go version
go version go1.10.4 windows/amd64

C:\Users\Jonathan_Lee>
```

- o 4.3输入 go env

```
命令提示符
Microsoft Windows [版本 10.0.17134.285]
(c) 2018 Microsoft Corporation。保留所有权利。

C:\Users\Jonathan_Lee>go env
set GOARCH=amd64
set GOBIN=
set GOCACHE=C:\Users\Jonathan_Lee\AppData\Local\go-build
set GOEXE=.exe
set GOHOSTARCH=amd64
set GOHOSTOS=windows
set GOOS=windows
set GOPATH=C:\Users\Jonathan_Lee\Desktop\Go_Lesson
set GORACE=
set GOROOT=D:\Developer\go
set GOTMPDIR=
set GOTOOOLDIR=D:\Developer\go\pkg\tool\windows_amd64
set GCCGO=gccgo
set CC=gcc
set CXX=g++
set CGO_ENABLED=1
set CGO_CFLAGS=-g -O2
set CGO_CPPFLAGS=
set CGO_CXXFLAGS=-g -O2
set CGO_FFLAGS=-g -O2
set CGO_LDFLAGS=-g -O2
set PKG_CONFIG=pkg-config
set GOGCCFLAGS=-m64 -mthreads -fmessage-length=0 -fdebug-prefix-m
18=/tmp/go-build -gno-record-gcc-switches

C:\Users\Jonathan_Lee>
```

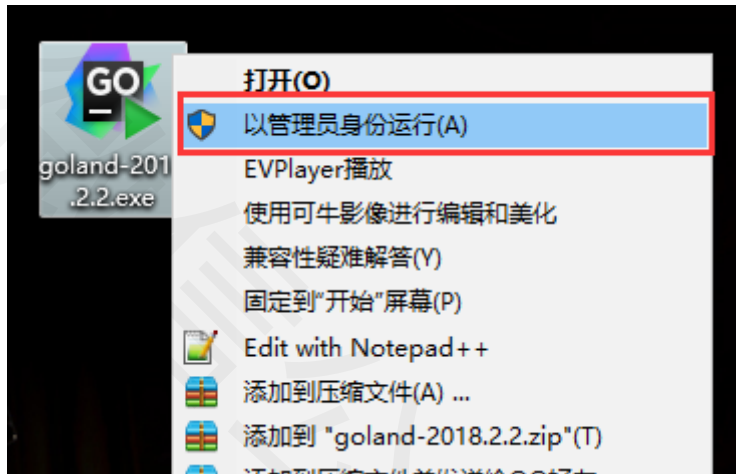
安装Go语言开发工具

- 记事本(开发效率极低)
- Vim(初学者入门门槛高)
- VSCode(不喜欢)
- Sublime Text(不喜欢)
- GoLand(喜欢,当收费)

- LiteIDE (开源免费, 跨平台运行, 轻量级)
- 生男生女都一样, 最关键是你中意哪个就用哪个

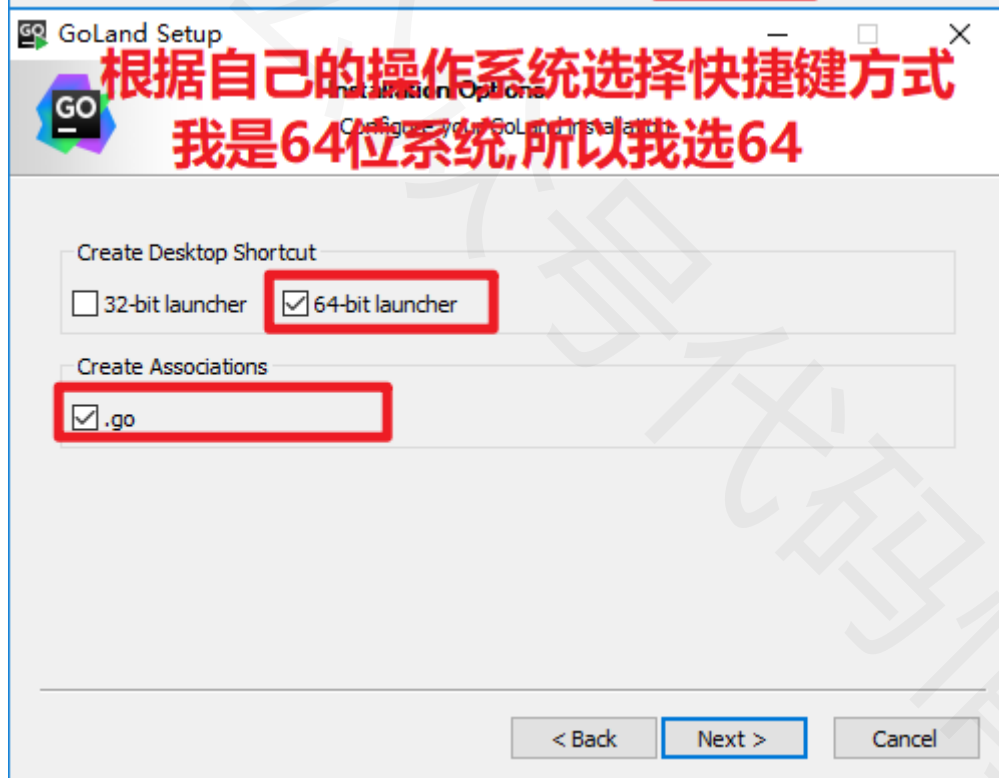
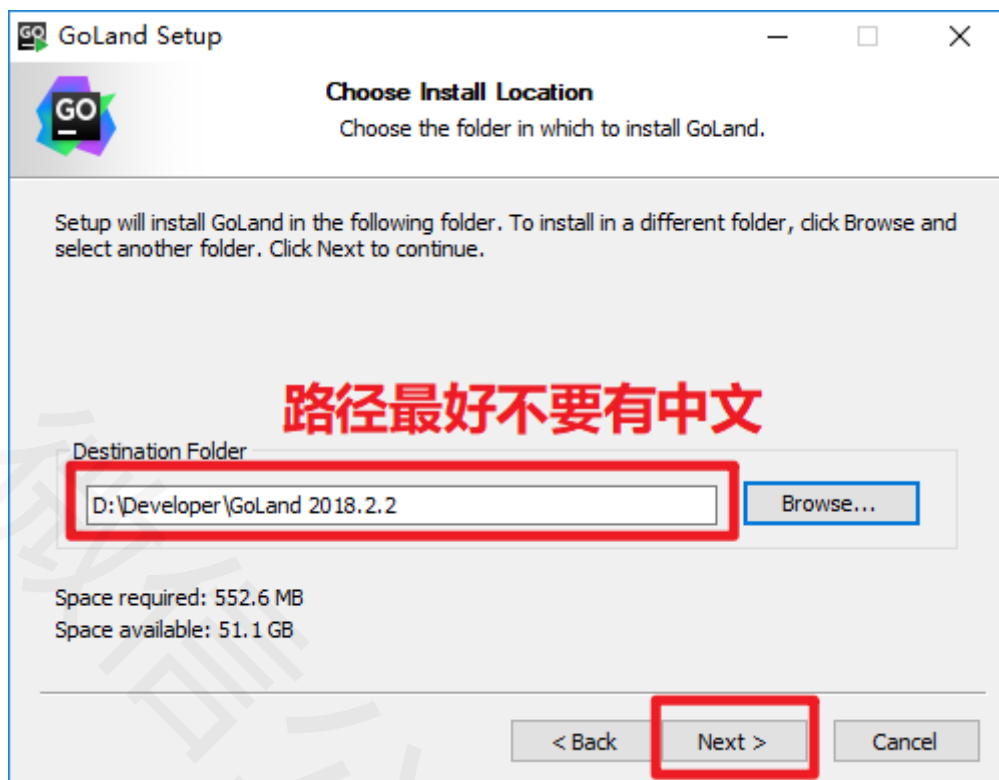
GoLand安装

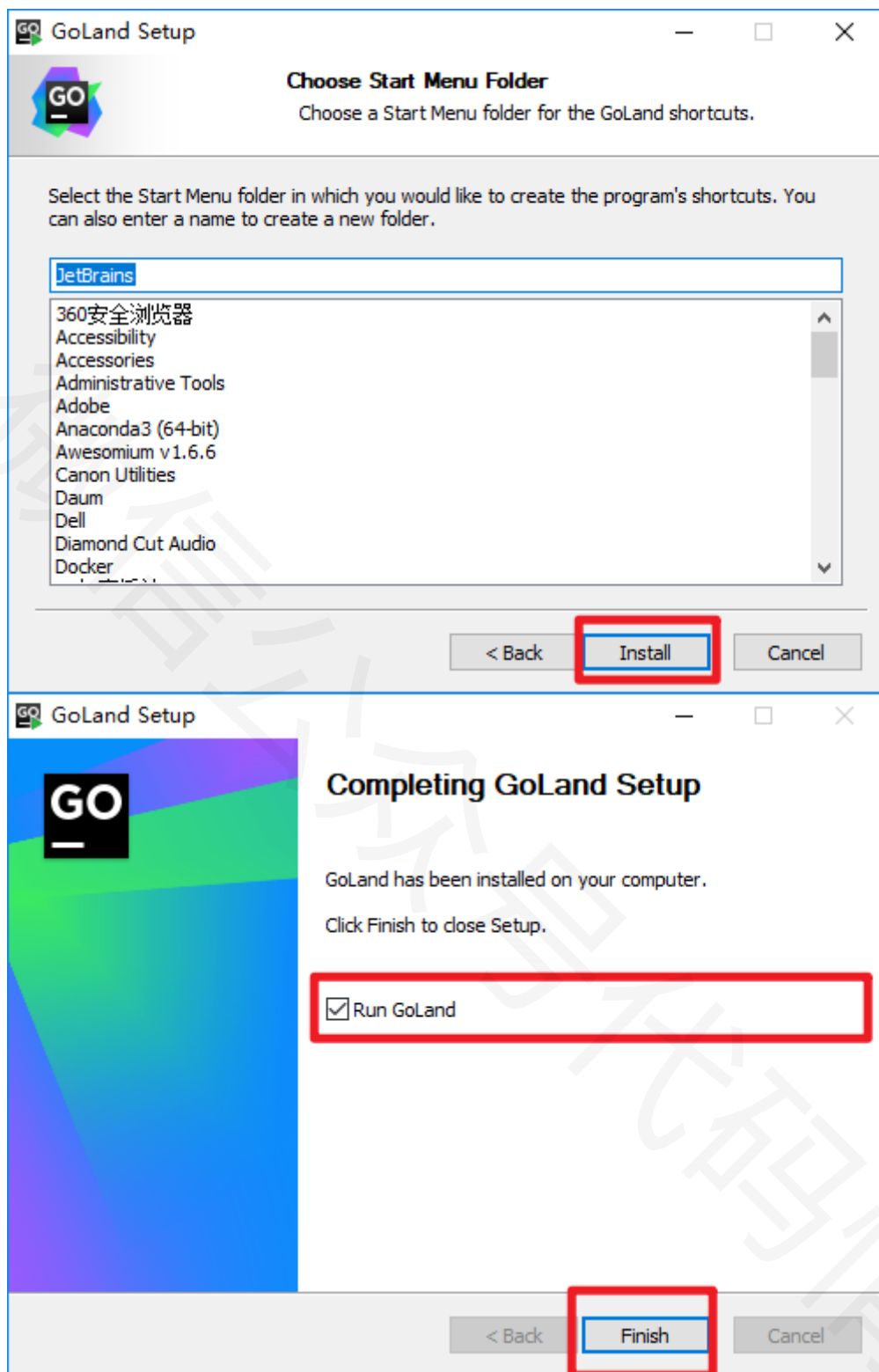
- 下载安装包: [点我下载GoLand](#)
- 提取码: lm7v
- 运行安装文件



- 疯狂下一步







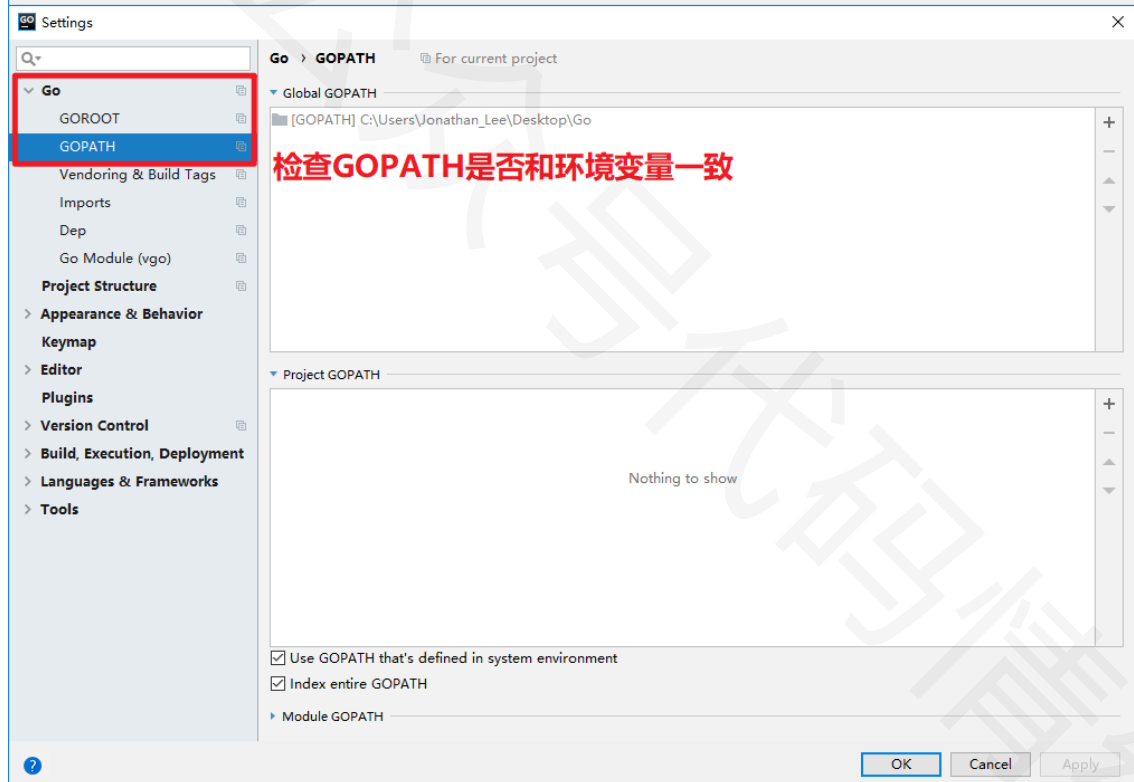
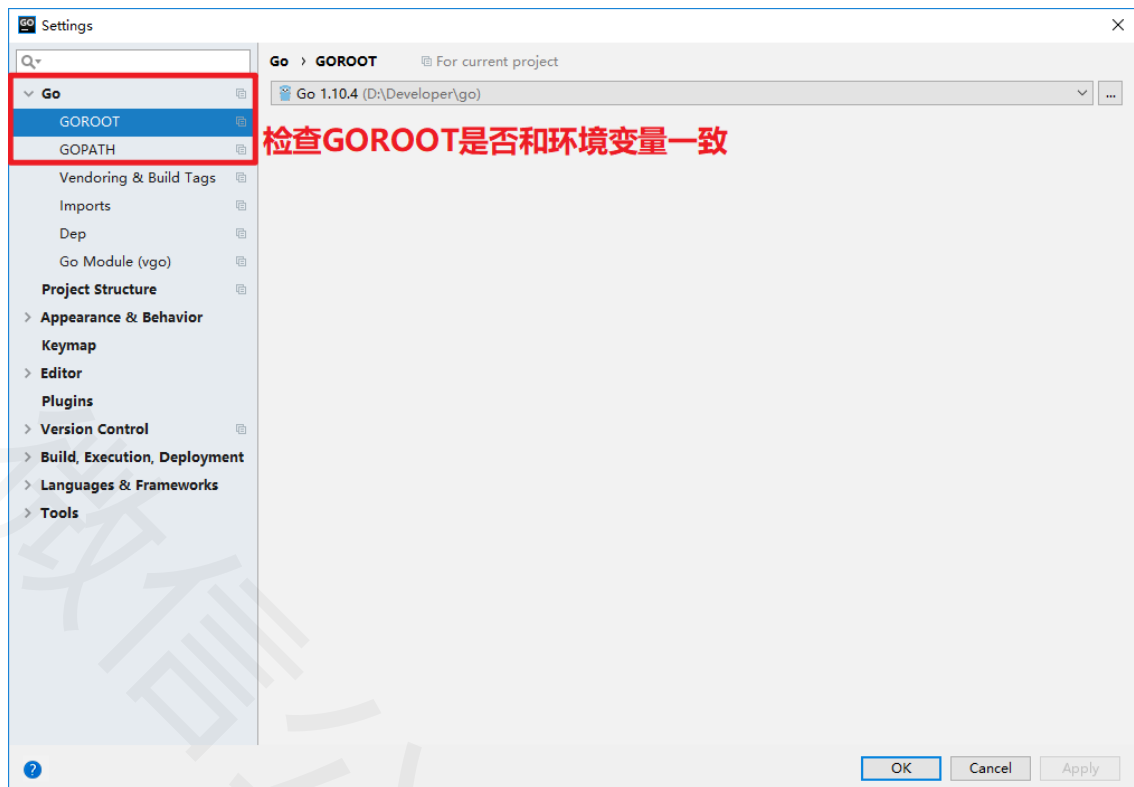
- 激活程序: 自行淘宝 JetBrains 激活 (仅供学生党参考, 在职人员请支持正版)

专业版 来这里 IJ PC WS PS GO (定制) 秒发 (通用) 15 款自定义账号 8 元 在线升级 多台电脑 不限版本 送 编程学习视频 / 风格主题 / 汉化 支持2017-2018最新版 ¥5.00 包邮 1314人收货 JetBrains IntelliJ Idea PyCharm webstorm 账号 激活 注册 / 码 三毛电脑 北京	皇冠店 彼岸老店回归 专业版 IJ PC WS PS GO (一号) 秒发 (全套) 通用十五款自定义账号 8 元 通用十五款全家桶激活码 5 元 在线升级 多台电脑 不限版本 送 编程学习视频 / 风格主题 / 汉化 支持2017-18最新版 ¥5.00 包邮 495人收货 JetBrains IntelliJ Idea PyCharm WebStorm 全家桶账号 激活 码 彼岸software 河南 商丘	送Python3+前端开发教程 彼岸software 正规 WS 优惠 稳定 品质 方式: Account (独享账号) 通用15款自定义全家桶账号 2017.2.4 在线升级 WebStorm 多台终端 ¥5.00 包邮 228人收货 JetBrains WebStorm 2018 激活码 全家桶 账号 版本不限 送汉化 彼岸software 河南 商丘	WebStorm 2016.2 ¥5.00 包邮 165人收货 JetBrains WebStorm 2018.2 激活码 送汉化 Win/Mac/Linux shenozit 北京
---	--	---	--

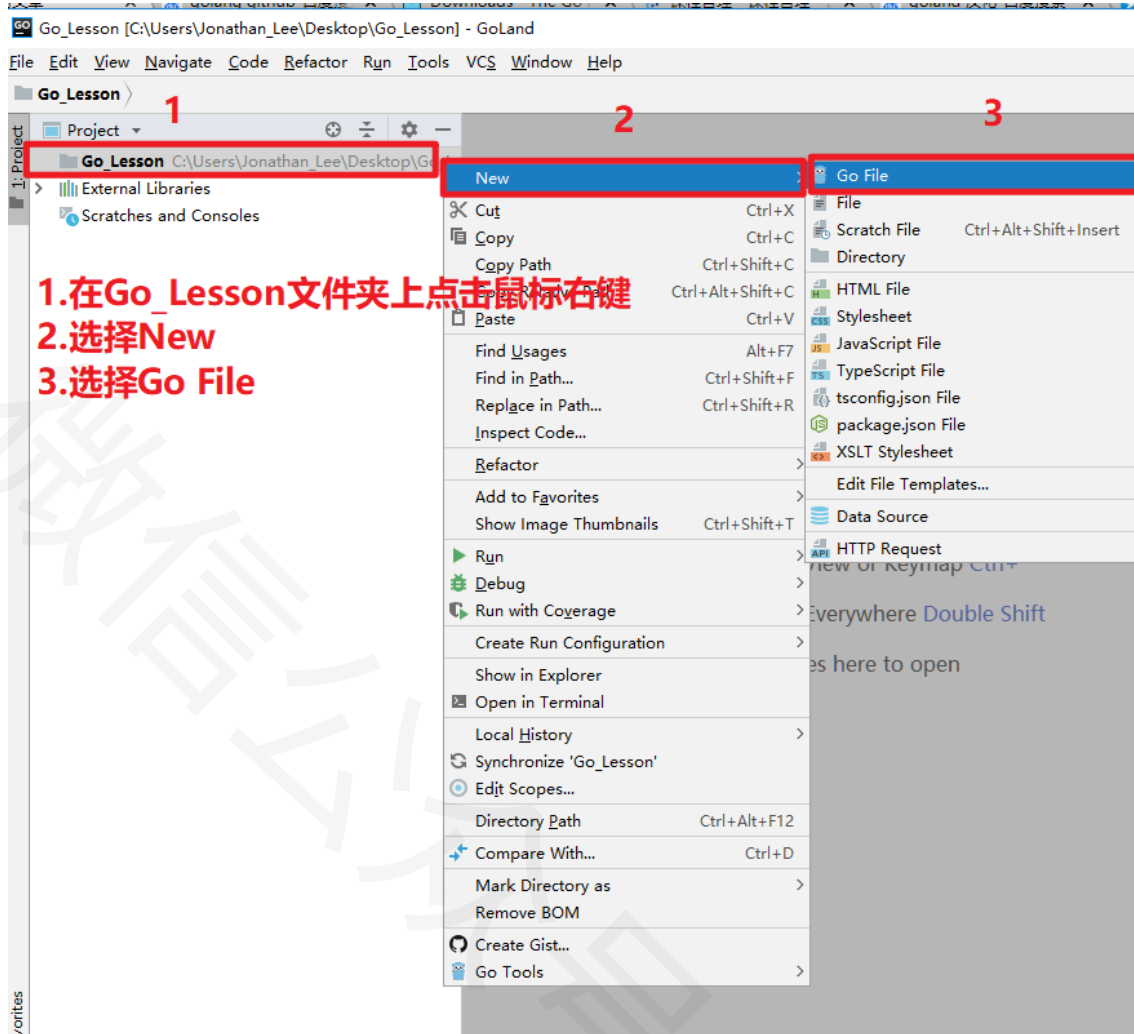
- 看不习惯英文的可以自行百度 GoLand 汉化包

- 打开项目文件夹

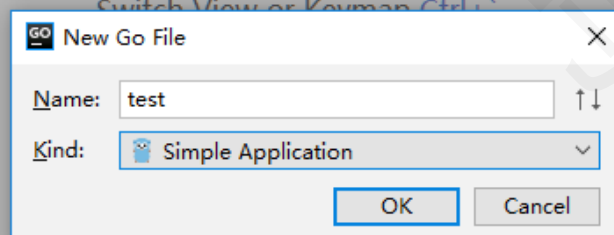




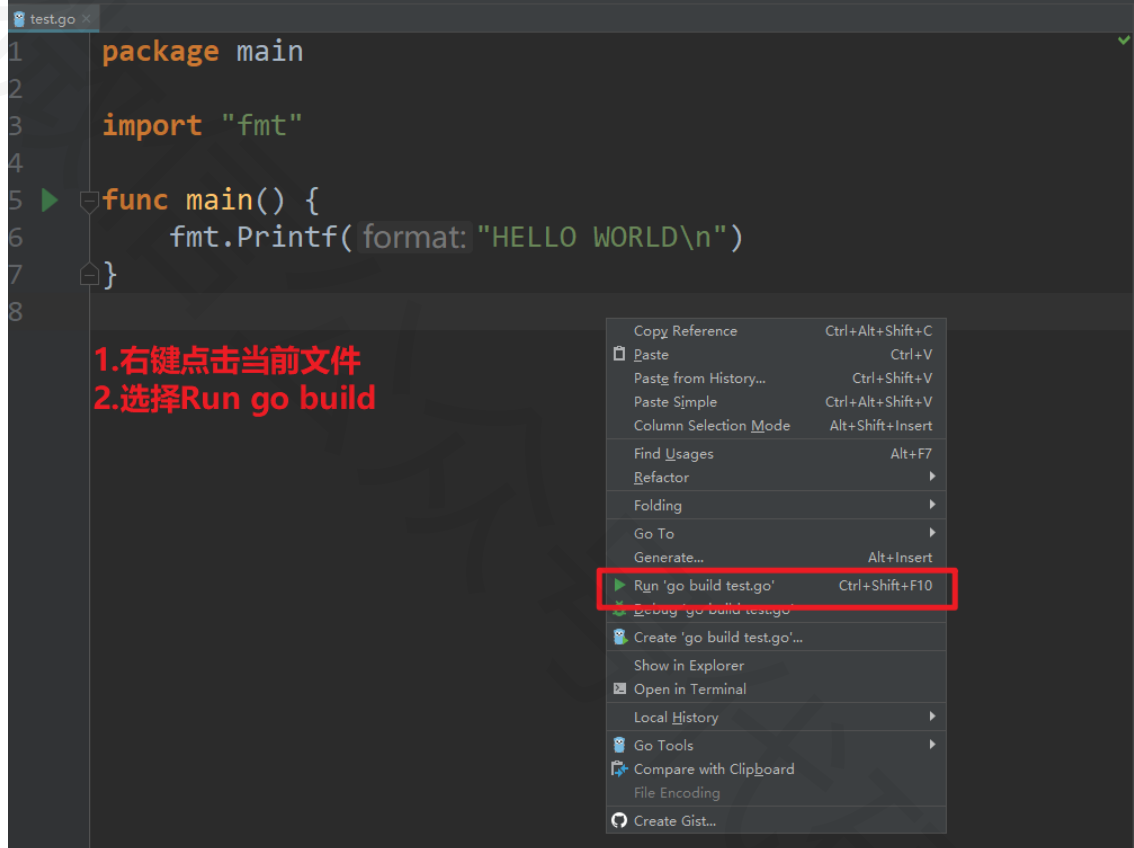
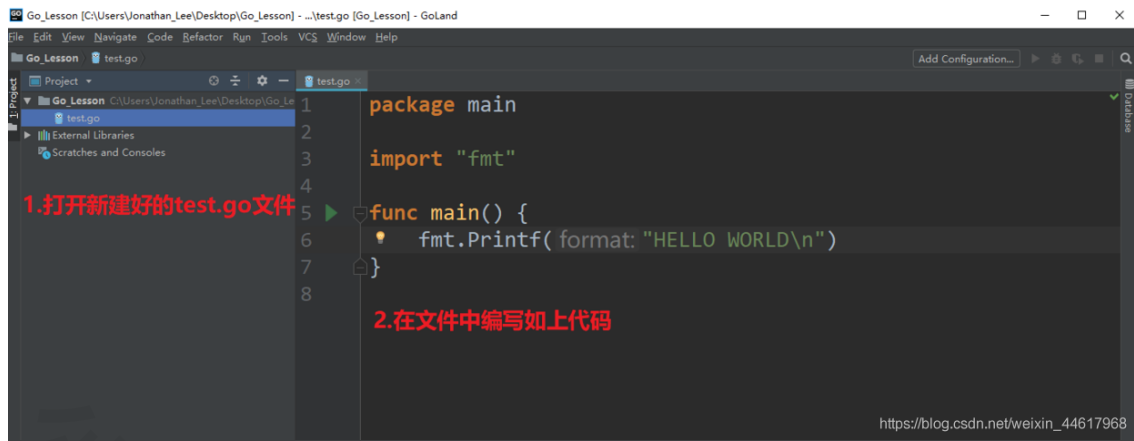
- 测试开发工具是否安装正确

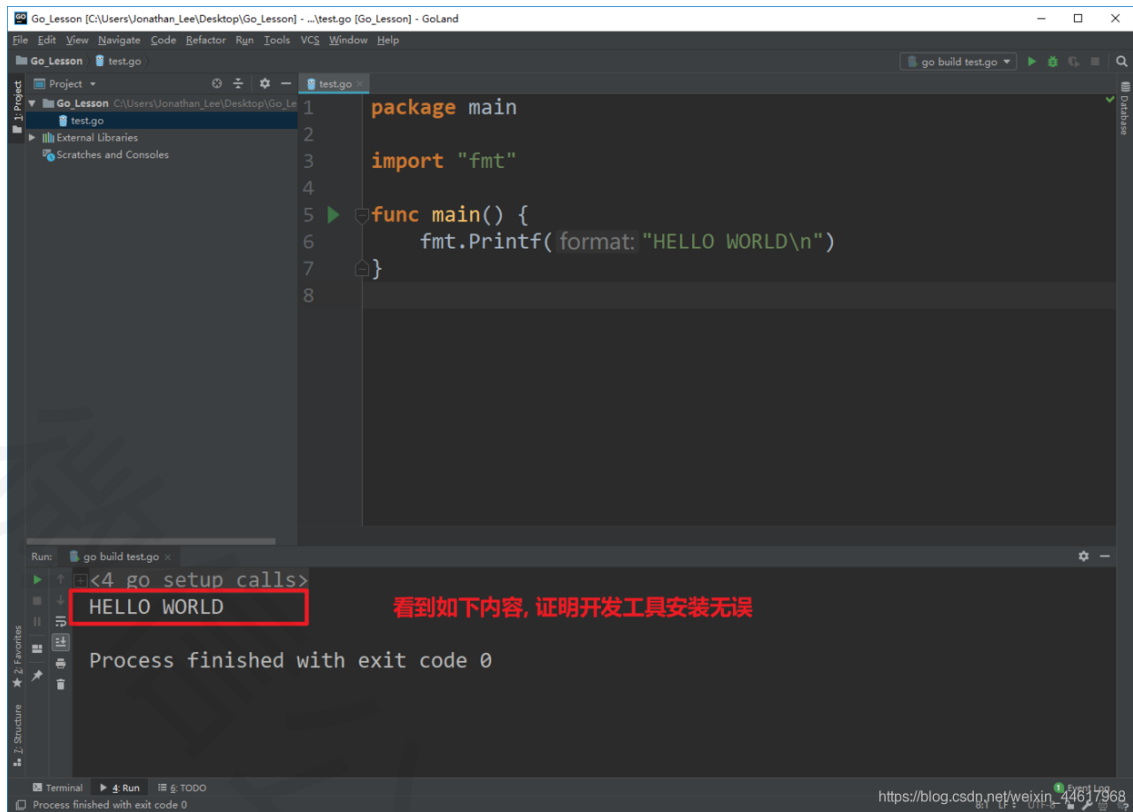


- 1.在Go_Lesson文件夹上点击鼠标右键
- 2.选择New
- 3.选择Go File

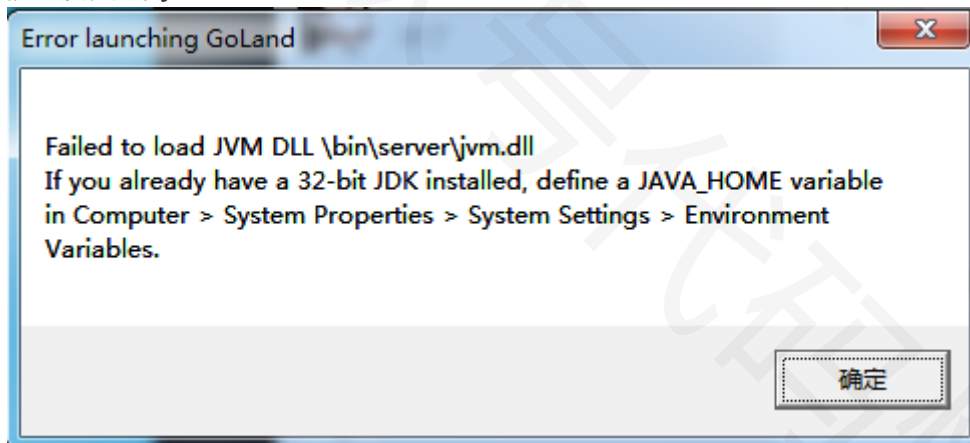


- 1.填写文件名称
- 2.选择Simple Application





- 其它问题:
 - 提示没有安装JVM



- 下载Java SDK [点击下载](#)

Java SE Development Kit 8u181		
You must accept the Oracle Binary Code License Agreement for Java SE to download this software.		
☒ Accept License Agreement ☐ Decline License Agreement		
Product / File Description	File Size	Download
Linux ARM 32 Hard Float ABI	72.95 MB	jdk-8u181-linux-arm32-vfp-hflt.tar.gz
Linux ARM 64 Hard Float ABI	69.89 MB	jdk-8u181-linux-arm64-vfp-hflt.tar.gz
Linux x86	165.06 MB	jdk-8u181-linux-i586.rpm
Linux x86	179.87 MB	jdk-8u181-linux-i586.tar.gz
Linux x64	162.15 MB	jdk-8u181-linux-x64.rpm
Linux x64	177.05 MB	jdk-8u181-linux-x64.tar.gz
Mac OS X x64	242.83 MB	jdk-8u181-macosx-x64.dmg
Solaris SPARC 64-bit (SVR4 package)	133.17 MB	jdk-8u181-solaris-sparcv9.tar.Z
Solaris SPARC 64-bit	94.34 MB	jdk-8u181-solaris-sparcv9.tar.gz
Solaris x64 (SVR4 package)	133.83 MB	jdk-8u181-solaris-x64.tar.Z
Solaris x64	92.11 MB	jdk-8u181-solaris-x64.tar.gz
Windows x86	194.41 MB	jdk-8u181-windows-i586.exe
Windows x64	202.73 MB	jdk-8u181-windows-x64.exe

- 安装即可

Go语言程序组成

- 和C语言程序一样，Go语言程序也是由众多函数组成的
- 和C语言程序一样，程序运行时系统会**自动调用**名称叫做**main的函数**
- 和C语言程序一样，如果一个程序**没有主函数**，则这个程序**不具备运行能力**
- 和C语言程序一样，一个Go语言程序**有且只能有一个主函数**

Go语言程序主函数定义格式

- C语言main函数格式

```
int main(int argc, const char * argv[]) {  
    return 0;  
}
```

- Go语言main函数格式
 - func 告诉系统这是一个函数
 - main主函数固定名称
 - 函数左括号必须和函数名在同一行
 - main函数必须在main包中

```
// 告诉系统当前编写的代码属于哪个包  
package main  
// 定义了一个名称叫做main的函数  
func main() {  
}
```

Go语言HelloWorld

```
package main // 告诉系统当前代码属于main这个包
import "fmt" // 导入打印函数对应的fmt包
func main() {
    // 通过包名.函数名称的方式, 利用fmt包中的打印函数输出语句
    fmt.Println("Hello world!!!")
}
```

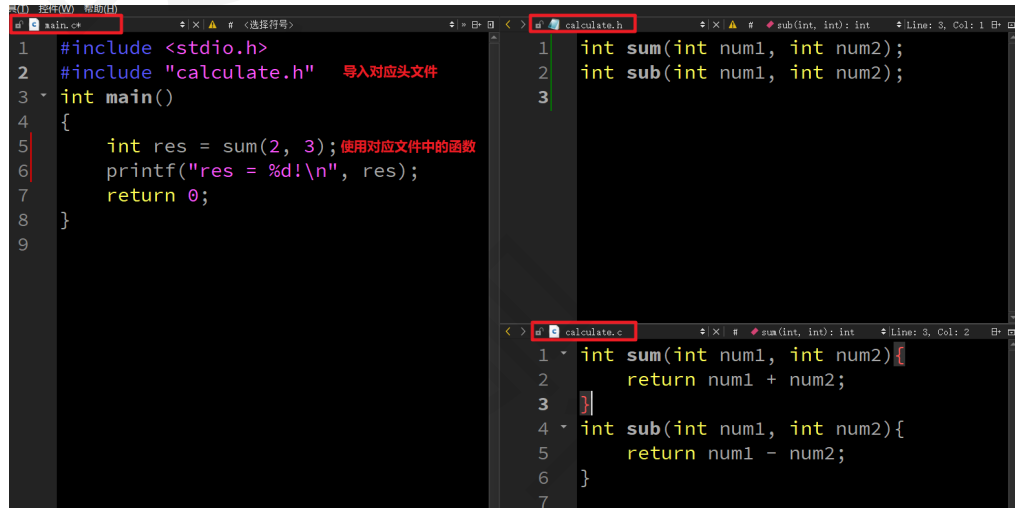
Go语言HelloWorld和C语言HelloWorld异同

• 1. 文件类型不同

- C语言代码保存在.c为后缀的文件中
- Go语言代码保存在.go为后缀的文件中

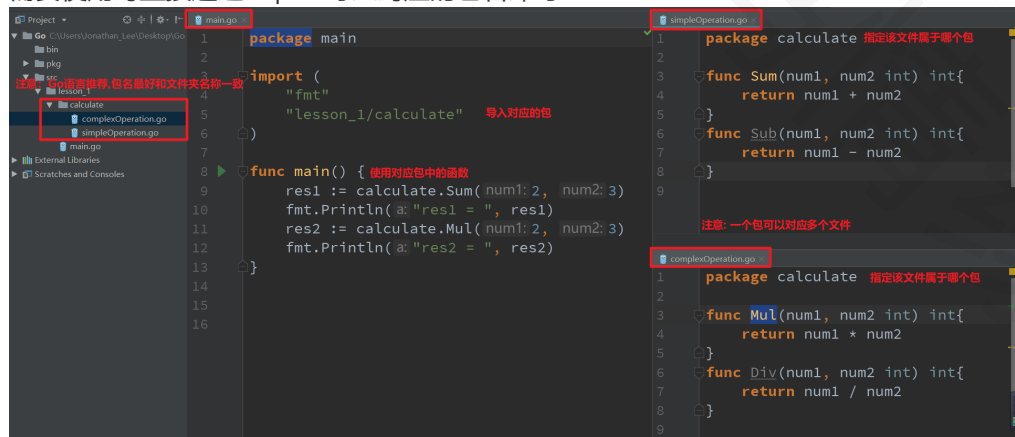
• 2. 代码管理方式不同

- C语言程序用**文件**的方式管理代码
 - C语言会把不同类型的代码放到不同的.c文件中, 然后再编写对应的.h文件
 - 需要使用时直接通过#include导入对应文件的.h文件即可



- Go语言程序用**包**的形式管理代码

- 我们会把不同类型的代码放到不同的.go文件中, 然后通过package给该文件指定一个包名
- 需要使用时直接通过import导入对应的包名即可



• 3. main函数书写文件不同

- C语言中main函数可以写在任意文件中, 只要保证一个程序只有一个main函数即可
- Go语言中main函数只能写在包名为main的文件夹中, 同样需要保存一个程序只有一个main函数

• 4. 函数编写的格式不同

- C语言中函数的格式为

- 注意:C语言函数的左括号可以和函数名称在同一行,也可以不在同一行

```
返回值类型 函数名称(形参列表) {  
    函数体相关语句;  
    return 返回值;  
}
```

- Go语言函数定义格式

注意:Go语言函数的左括号必须和函数名称在同一行,否则会报错

```
func 函数名称(形参列表)(返回值列表) {  
    函数体相关语句;  
    return 返回值;  
}
```

- 5.函数调用的格式不同

- C语言通过#include导入.h文件后,直接通过函数名称调用函数
- Go语言通过import导入对应的包后,需要通过包名.函数名称的方式调用

```
#include <stdio.h>  
#include "calculate.h"  
int main()  
{  
    int res = sum(2, 3); // 直接利用函数名称调用函数  
    printf("res = %d!\n", res);  
    return 0;  
}
```

```
package main  
import (  
    "fmt"  
    "lesson_1/calculate"  
)  
func main() {  
    res := calculate.Sum(2, 3) // 使用包名.函数名称调用函数  
    fmt.Println("res1 = ", res)  
}
```

- 6.语句的结束方式不同

- C语言中每条语句都必须以分号结尾
- Go语言中每条语句后面不用添加分号(编译器会自动添加)

```
#include <stdio.h>  
#include "calculate.h"  
int main()  
{  
    int res = sum(2, 3); // 不写分号会报错  
    printf("res = %d!\n", res); // 不写分号会报错  
    return 0; // 不写分号会报错  
}
```

```
package main
import (
    "fmt"
    "lesson_1/calculate"
)
func main() {
    res := calculate.Sum(2, 3) // 不用写分号
    fmt.Println("res1 = ", res) // 不用写分号
}
```

Go语言注释

- 和C语言一样,Go语言也支持单行注释和多行注释, 并且所有注释的特性都和C语言一样
 - 单行注释 `// 被注释内容`
 - 多行注释 `/* 被注释内容*/`
- 在Go语言中,官方更加推荐使用单行注释,而非多行注释(详情可以直接查看Go官方源码)

Go语言编码风格

- 1.go程序编写在.go为后缀的文件中
- 2.包名一般使用文件所在文件夹的名称
- 2.包名应该简洁、清晰且全小写
- 3.main函数只能编写在main包中
- 4.每一条语句后面可以不用编写分号(推荐)
- 5.如果没有编写分号,一行只能编写一条语句
- 6.函数的左括号必须和函数名在同一行
- 7.导入包但没有使用包编译会报错
- 8.定义局部变量但没有使用变量编译也会报错
- 9.定义函数但没有使用函数不会报错
- 10.给方法、变量添加说明,尽量使用单行注释

关键字

- Go语言中的关键字和C语言中的关键字的含义样, 是指被Go语言赋予特殊含义的单词
- Go语言中关键字的特征和C语言也一样
 - 全部都是小写
 - 在开发工具中会显示特殊颜色
- Go语言中关键字的注意点和C语言也一样
 - 因为关键字在C语言中有特殊的含义, 所以不能用作变量名、函数名等

C语言关键字和Go语言关键字对比

- C语言中一共有32个关键字

1	2	3	4	5	6	7	8
if	else	switch	case	default	break	return	goto
do	while	for	continue	typedef	struct	enum	union
char	short	int	long	float	double	void	sizeof
signed	unsigned	const	auto	register	static	extern	volatile

- Go语言中一共有25个关键字

1	2	3	4	5	6	7	8
if	else	switch	case	default	break	return	goto
fallthrough	for	continue	type	struct	var	const	map
func	interface	range	import	package	defer	go	select
chan							

- Go语言中除了关键字以外,还有30多个 预定义标识符

内建常量			
true	false	iota	nil

内建类型			
int	int8	int16	int32
int64	uint	uint8	uint16
uint32	uint64	uintptr	float32
float64	complex64	complex128	bool
byte	rune	string	error

内建函数			
make	len	cap	new
append	copy	delete	real
imag	panic	recover	complex

标识符

- Go语言中的标识符和C语言中的标识符的含义样, 是指程序员在程序中自己起的名字(变量名称、函数名称等)

- 和C语言一样Go语言标识符也有一套 命名规则, Go语言标识符的命名规则几乎和C语言一模一样

- 只能由字母(a~z、A~Z)、数字、下划线组成
 - 不能包含除下划线以外的其它特殊字符串
 - 不能以数字开头
 - 不能是Go语言中的关键字
 - 标识符严格区分大小写, test和Test是两个不同的标识符
- 和C语言标识符命名规则不同的是
 - Go语言中_单独作为标识符出现时, 代表 空标识符, 它对应的值会被忽略

```
package main

import "fmt"

func main() {
    // 将常量10保存到名称叫做num的变量中
    var num int = 10
    fmt.Println("num = ", num)

    // 忽略常量20, 不会分配存储空间, 也不会保存常量20
    //var _ int = 20
    //fmt.Println("_ = ", _) // cannot use _ as value

    // Go语言中如果定义了变量没有使用, 那么编译会报错(sub declared and not used)
    // 所以如果我们只使用了sum, 没有使用sub会报错
    // 为了解决这个问题, 我们可以使用_忽略sub的值
    //var sum, sub int = calculate(20, 10)
    var sum, _ int = calculate(20, 10)
    fmt.Println("sum = ", sum)
}

func calculate(a, b int)(int, int) {
    var sum int = a + b
    var sub int = a - b
    return sum, sub
}

```go
+ Go语言默认的编码方式就是UTF-8, 所以Go语言支持中文, 所以可以用中文作为标识符(非常非常非常不推荐)
```go
package main

import "fmt"

func main() {
    // 不会报错, 可以正常运行
    var 年龄 int = 33
    fmt.Println("年龄 = ", 年龄) // 33

    // 不会报错, 可以正常运行
    var 结果 int = 计算器(10, 20)
    fmt.Println("结果 = ", 结果) // 30
}

func 计算器(第一个变量, 第二个变量 int)int {
    return 第一个变量 + 第二个变量
}
```

- 和C语言一样,标识符除了有命名规则以外,还有标识符命名规范
 - 规则必须遵守, 规范不一定要遵守, 但是建议遵守
 - Go语言的命名规范和C语言一样, 都是采用驼峰命名, 避免采用_命名
 - 驼峰命名: sendMessage / sayHello
 - _命名: send_message / say_hello

Go语言数据类型

- Go语言本质是用C语言编写的一套高级开发语言, 所以Go语言中的数据类型大部分都是由C语言演变而来的
- C语言数据类型



• Go语言数据类型



• C语言各数据类型占用内存空间

类型	32位编译器	64位编译器
char	1	1
int	4	4
float	4	4
double	8	8
short	2	2
long	4	8
long long	8	8
void*	4	8

• Go语言各数据类型占用内存空间

类型	32位编译器	64位编译器	本质
int8/uint8	1	1	signed char/unsigned char
int16/uint16	2	2	signed short/unsigned short
int32/uint32	4	4	signed int/unsigned int
int64/uint64	8	8	signed long long int/unsigned long long int
byte	1	1	uint8/unsigned char
rune	4	4	int32/signed int
int	4	8	根据机器位数决定长度
uintptr	4	8	根据机器位数决定长度 uint32/uint64
float32	4	4	float
float64	8	8	double
true	1	1	char类型的整型
false	1	1	char类型的整型

- 和C语言一样,Go语言也提供了Sizeof计算变量的内存空间
 - 1.导入import "unsafe"包
 - 2.通过unsafe.Sizeof()计算变量内存空间

```
package main

import (
    "fmt"
    "unsafe"
)

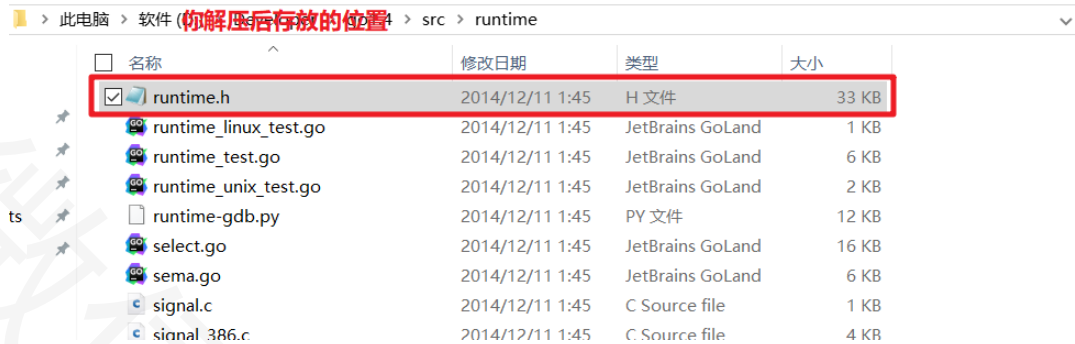
func main() {
    fmt.Println("int size = ", unsafe.Sizeof(int(0)))
    fmt.Println("int8 size = ", unsafe.Sizeof(int8(0)))
    fmt.Println("int16 size = ", unsafe.Sizeof(int16(0)))
    fmt.Println("int32 size = ", unsafe.Sizeof(int32(0)))
    fmt.Println("int64 size = ", unsafe.Sizeof(int64(0)))
    fmt.Println("uint size = ", unsafe.Sizeof(uint(0)))
    fmt.Println("uint8 size = ", unsafe.Sizeof(uint8(0)))
    fmt.Println("uint16 size = ", unsafe.Sizeof(uint16(0)))
    fmt.Println("uint32 size = ", unsafe.Sizeof(uint32(0)))
    fmt.Println("uint64 size = ", unsafe.Sizeof(uint64(0)))
    fmt.Println("uintptr size = ", unsafe.Sizeof(uintptr(0)))
    fmt.Println("byte size = ", unsafe.Sizeof(byte(0)))
    fmt.Println("rune size = ", unsafe.Sizeof(rune(0)))
    fmt.Println("float32 size = ", unsafe.Sizeof(float32(0)))
    fmt.Println("float64 size = ", unsafe.Sizeof(float64(0)))
    fmt.Println("true size = ", unsafe.Sizeof(true))
    fmt.Println("false size = ", unsafe.Sizeof(false))
}
```

- Go语言基本数据类型内部实现

- [golang官方网站下载go1.4版本源代码](#)

- 越老版本的代码越纯粹,越适合新手学习
 - 随着代码的更新迭代会逐步变得非常复杂, 所以此处建议下载1.4版本

- 解压后打开路径: `go\src\runtime\runtime.h`



- 得到如下实现代码

```
// 第8行到35行
typedef signed char      int8;
typedef unsigned char    uint8;
typedef signed short     int16;
typedef unsigned short   uint16;
typedef signed int       int32;
typedef unsigned int     uint32;
typedef signed long long int64;
typedef unsigned long long uint64;
typedef float            float32;
typedef double           float64;

#ifdef _64BIT
typedef uint64      uintptr;
typedef int64       intptr;
typedef int64       intgo; // Go's int
typedef uint64      uintgo; // Go's uint
#else
typedef uint32      uintptr;
typedef int32       intptr;
typedef int32       intgo; // Go's int
typedef uint32      uintgo; // Go's uint
#endif

#ifdef _64BITREG
typedef uint64      uintreg;
#else
typedef uint32      uintreg;
#endif

// 第153行到157行
enum
{
    true    = 1,
    false   = 0,
};
```

install B 时刻:

Go本质就是用C语言编写的一门高级编程语言

所以江哥前面教你C语言就是为了今天能让你看懂Go的实现代码,做到知其然知其所以然

注意点: 企业开发中一般使用int, 因为int会根据你当前的操作系统自动转换为int32和int64

Go语言变量

- Go语言中变量的概念和C语言中也一样, 所以我们直接来看下如何定义和使用变量即可
- C语言中定义变量的格式

数据类型 变量名称;

数据类型 变量名称1, 变量名称2;

```
#include <stdio.h>

int main(int argc, const char * argv[])
{
    int num1; // 先定义
    num1 = 10; // 后初始化
    printf("num1 = %d\n", num1);

    int num2 = 20; // 定义的同时初始化
    printf("num2 = %d\n", num2);

    // 注意: 同时定义多个变量,不支持定义时初始化, 只能先定义后初始化
    int num3, num4; //同时定义多个变量
    num3 = 30;
    num4 = 40;
    printf("num3 = %d\n", num3);
    printf("num4 = %d\n", num4);

    return 0;
}
```

- Go语言中定义变量有三种格式

```
// 标准格式
var 变量名称 数据类型 = 值;
// 自动推到类型格式
var 变量名称 = 值;
// 简短格式(golang官方推荐格式)
变量名称 := 值;
```

```
package main
import "fmt"
func main() {
    var num1 int // 先定义
    num1 = 10 // 后赋值
    fmt.Println("num1 = ", num1)

    var num2 int = 20 // 定义的同时赋值
    fmt.Println("num2 = ", num2)
```

```

var num3 = 30 // 定义的同时赋值，并省略数据类型
fmt.Println("num3 = ", num3)

num4 := 40 // 定义的同时赋值，并省略关键字和数据类型
/*
num4 := 40 等价于
var num4 int
num4 = 40
*/
fmt.Println("num4 = ", num4)
}

```

- 和C语言一样,除了可以定义单个变量以外,还支持一次性定义多个变量
 - 方式一,连续定义

```

package main
import "fmt"
func main() {
    var num1, num2 int // 先定义
    num1 = 10 // 后赋值
    num2 = 20
    fmt.Println("num1 = ", num1)
    fmt.Println("num2 = ", num2)

    var num3, num4 int = 30, 40 // 定义的同时赋值
    fmt.Println("num3 = ", num3)
    fmt.Println("num4 = ", num4)

    var num5, num6 = 50, 60 // 定义的同时赋值，并省略数据类型
    fmt.Println("num5 = ", num5)
    fmt.Println("num6 = ", num6)

    num7, num8 := 70, 80 // 定义的同时赋值，并省略关键字和数据类型
    fmt.Println("num7 = ", num7)
    fmt.Println("num8 = ", num8)
}

```

- 方式二,变量组

```

package main
import "fmt"
func main() {
    var( // 先定义
        num1 int
        num2 float32
    )
    num1 = 10 // 后赋值
    num2 = 3.14
    fmt.Println("num1 = ", num1)
    fmt.Println("num2 = ", num2)

    var( // 定义的同时赋值
        num3 int = 30
        num4 float32 = 6.66
    )
    fmt.Println("num3 = ", num3)
}

```

```

fmt.Println("num4 = ", num4)

var( // 定义的同时赋值, 并省略数据类型
    num5 = 50
    num6 = 7.77
)
fmt.Println("num5 = ", num5)
fmt.Println("num6 = ", num6)

var( // 一行定义多个
    num7, num8 = 70, 80
    num9, num10 = 9.99, 100
)
fmt.Println("num7 = ", num7)
fmt.Println("num8 = ", num8)
fmt.Println("num9 = ", num9)
fmt.Println("num10 = ", num10)
}

```

Go语言变量定义注意点

- 简短模式的含义是定义的同时初始化

```

package main
import "fmt"
func main() {
    num := 10
    num := 20 // 编译报错, 重复定义
    fmt.Println("num = ", num)
}

```

- 一定不要把:=当做赋值运算符来使用

```

package main
import "fmt"
var num = 10 // 定义一个全局变量
func main() {
    num := 20 // 定义一个局部变量
    fmt.Println("num = ", num)
    test()
}
func test() {
    fmt.Println("num = ", num) // 还是输出10
}

```

- :=只能用于定义局部变量,不能用于定义全局变量

```

package main
import "fmt"
num := 10 // 编译报错
func main() {
    fmt.Println("num = ", num)
}

```

- 使用:=定义变量时,不能指定var关键字和数据类型

```
package main
import "fmt"
func main() {
    //var num int := 10 // 编译报错
    //var num := 10 // 编译报错
    num int := 10 // 编译报错
    fmt.Println("num = ", num)
    fmt.Println("num = ", num)
}
```

- 变量组中不能够使用:=

```
package main
import "fmt"
func main() {
    var(
        num := 10 // 编译报错
    )
    fmt.Println("num = ", num)
}
```

- 通过:=同时定义多个变量,必须给所有变量初始化

```
package main
import "fmt"
func main() {
    //num1, num2 := 666, 888 // 正确
    num1, num2 := 666 // 报错
    fmt.Printf("%d, %d\n", num1, num2)
}
```

- 通过:=同时定义多个变量,只要任意一个变量没有定义过,都会做退化赋值操作

```
package main
import "fmt"
func main() {
    // 定义一个变量num1
    num1 := 10
    // 同时定义两个变量num1和num2, 由于num2从来没有定义过,
    // 所以对于num1来说:=退化为赋值运算符, 而对于num2来说:=仍然是定义+赋值
    num1, num2 := 20, 30
    fmt.Println("num1 = ", num1)
    fmt.Println("num2 = ", num2)
}
```

```

package main
import "fmt"
func main() {
    num1 := 10
    num2 := 20
    // 报错, 因为num1, 和num2都被定义过
    // 至少要有任意一个变量没有被定义过, 才会退化赋值
    num1, num2 := 30, 40
    fmt.Println("num1 = ", num1)
    fmt.Println("num2 = ", num2)
}

```

- 定义的局部变量或者导入的包没有被使用, 那么编译器会报错, 无法编译运行, 但是定义的全局变量没有被使用, 编译器不会报错, 可以编译运行

局部变量和全局变量

- 和C语言一样, 按照变量的作用域, 我们可以把变量划分为局部变量和全局变量
- Go语言中局部变量的概念以及全局变量的概念和C语言一模一样
- 局部变量:
 - 定义在函数内部的变量以及函数的形参称为局部变量
 - 作用域: 从定义哪一行开始直到与其所在的代码块结束
 - 生命周期: 从程序运行到定义哪一行开始分配存储空间到程序离开该变量所在的作用域
- 全局变量:
 - 定义在函数外面的变量称为全局变量
 - 作用域范围: 从定义哪行开始直到文件结尾
 - 生命周期: 程序一启动就会分配存储空间, 直到程序结束
- 和C语言不同的是, C语言中可以定义相同名称的全局变量, 而Go语言中无论全局变量还是局部变量, 只要作用域相同都不能出现同名的变量

```

package main
import "fmt"
//var num1 int
//var num1 int // 报错, 重复定义
var num3 int
func main() {
    //var num2
    //var num2 // 报错, 重复定义

    var num3 int // 不报错, 因为作用域不同
    fmt.Println("num3 = ", num3)
}

```

- C语言中全局变量没有赋值, 那么默认初始值为0, 局部变量没有赋值, 那么默认初始值是随机值
- Go语言中无论是全局变量还是局部变量, 只要定义了一个变量都有默认的0值
 - int/int8/int16/int32/int64/uint/uint8/uint16/uint32/uint64/byte/rune/uintptr的默认值是0
 - float32/float64的默认值是0.0
 - bool的默认值是false
 - string的默认值是""
 - pointer/function/interface/slice/channel/map/error的默认值是nil

- 其它复合类型array/struct默认值是内部数据类型的默认值

```
package main
import "fmt"
func main() {
    var intV int // 整型变量
    var floatV float32 // 实型变量
    var boolV bool // 布尔型变量
    var stringV string // 字符串变量
    var pointerV *int // 指针变量
    var funcV func(int, int)int // function变量
    var interfaceV interface{} // 接口变量
    var sliceV []int // 切片变量
    var channelV chan int // channel变量
    var mapV map[string]string // map变量
    var errorV error // error变量

    fmt.Println("int = ", intV) // 0
    fmt.Println("float = ", floatV) // 0
    fmt.Println("bool = ", boolV) // false
    fmt.Println("string = ", stringV) // ""
    fmt.Println("pointer = ", pointerV) // nil
    fmt.Println("func = ", funcV) // nil
    fmt.Println("interface = ", interfaceV) // nil
    fmt.Println("slice = ", sliceV) // []
    fmt.Println("slice = ", sliceV == nil) // true
    fmt.Println("channel = ", channelV) // nil
    fmt.Println("map = ", mapV) // map[]
    fmt.Println("map = ", mapV == nil) // true
    fmt.Println("error = ", errorV) // nil

    var arrayV [3]int // 数组变量
    type Person struct{
        name string
        age int
    }
    var structV Person // 结构体变量
    fmt.Println("array = ", arrayV) // [0, 0, 0]
    fmt.Println("struct = ", structV) // {"" 0}
}
```

数据类型转换

- C语言中数据可以隐式转换或显示转换, 但是Go语言中数据只能显示转换
- C语言隐式转换

```
#include <stdio.h>
int main(){
    // 隐式转换: 自动将实型10.6转换为整型后保存
    int a = 10.6;
    // 自动类型提升: 运算时会自动将小类型转换为大类型后运算
    double b = 1.0 / 2; // 等价于1.0 / 2.0
}
```

- C语言显示转换(强制转换)

```
#include <stdio.h>
int main(){
    // 显示转换:强制将实型10.6转换为整型后保存
    int a = (int)10.5;
}
```

- Go语言数值类型之间转换
 - 格式: 数据类型(需要转换的数据)
 - 注意点: 和C语言一样数据可以从大类型转换为小类型, 也可以从小类型转换为大类型. 但是大类型转换为小类型可能会丢失精度

```
package main
import "fmt"
func main() {
    var num0 int = 10
    var num1 int8 = 20
    var num2 int16
    //num2 = num0 // 编译报错, 不同长度的int之间也需要显示转换
    //num2 = num1 // 编译报错, 不同长度的int之间也需要显示转换
    num2 = int16(num0)
    num2 = int16(num1)
    fmt.Println(num2)

    var num3 float32 = 3.14
    var num4 float64
    //num4 = num3 // 编译报错, 不同长度的float之间也需要显示转换
    num4 = float64(num3)
    fmt.Println(num4)

    var num5 byte = 11
    var num6 uint8 // 这里不是隐式转换, 不报错的原因是byte的本质就是uint8
    num6 = num5
    fmt.Println(num6)

    var num7 rune = 11
    var num8 int32
    num8 = num7 // 这里不是隐式转换, 不报错的原因是byte的本质就是int32
    fmt.Println(num8)
}
```

数值类型和字符串类型之间转换

- Go语言中不能通过 数据类型(变量)的格式将数值类型转换为字符串, 也不能通过 数据类型(变量)的格式将字符串转换为数值类型

```
package main
import "fmt"
func main() {
    var num1 int32 = 65
    // 可以将整型强制转换, 但是会按照ASCII码表来转换
    // 但是不推荐这样使用
    var str1 string = string(num1)
    fmt.Println(str1)
}
```

```

var num2 float32 = 3.14
// 不能将其它基本类型强制转换为字符串类型
var str2 string = string(num2)
fmt.Println(str2)

var str3 string = "97"
// 不能强制转换, cannot convert str2 (type string) to type int
var num3 int = int(str3)
fmt.Println(num3)
}

```

- 数值类型转字符串类型 strconv.FormatXxx()

```

package main
import "fmt"
func main() {
    var num1 int32 = 10
    // 第一个参数: 需要被转换的整型, 必须是int64类型
    // 第二个参数: 转换为几进制, 必须在2到36之间
    // 将32位十进制整型变量10转换为字符串, 并继续保留10进制格式
    str1 := strconv.FormatInt(int64(num1), 10)
    fmt.Println(str1) // 10
    // 将32位十进制整型变量10转换为字符串, 并转换为2进制格式
    str2 := strconv.FormatInt(int64(num1), 2)
    fmt.Println(str2) // 1010

    var num5 float64 = 3.1234567890123456789
    // 第一个参数: 需要转换的实型, 必须是float64类型
    // 第二个参数: 转换为什么格式, f小数格式, e指数格式
    // 第三个参数: 转换之后保留多少位小数, 传入-1按照指定类型有效位保留
    // 第四个参数: 被转换数据的实际位数, float32就传32, float64就传64
    // 将float64位实型, 按照小数格式并保留默认有效位转换为字符串
    str3 := strconv.FormatFloat(num5, 'f', -1, 64)
    fmt.Println(str3) // 3.1234567
    str4 := strconv.FormatFloat(num5, 'f', -1, 64)
    fmt.Println(str4) // 3.1234567890123457
    // 将float64位实型, 按照小数格式并保留2位有效位转换为字符串
    str5 := strconv.FormatFloat(num5, 'f', 2, 64)
    fmt.Println(str5) // 3.12
    // 将float64位实型, 按照指数格式并保留2位有效位转换为字符串
    str6 := strconv.FormatFloat(num5, 'e', 2, 64)
    fmt.Println(str6) // 3.12

    var num6 bool = true
    str7 := strconv.FormatBool(num6)
    fmt.Println(str7) // true
}

```

- 字符串类型转数值类型 strconv.ParseXxx()

```

package main
import "fmt"
func main() {
    var str1 string = "125"
    // 第一个参数: 需要转换的数据
    // 第二个参数: 转换为几进制

```

```

// 第三个参数：转换为多少位整型
// 注意点：ParseInt函数会返回两个值，一个是转换后的结果，一个是错误
// 如果被转换的数据转换之后没有超出指定的范围或者不能被转换时，
// 那么错误为nil，否则错误不为nil
// 将字符串"125"转换为10进制的int8
num1, err := strconv.ParseInt(str1, 10, 8)
if err != nil {
    fmt.Println(err)
}
fmt.Println(num1)

var str2 string = "150"
// 将字符串"150"转换为10进制的int8
// 由于int8的取值范围是-128~127，所以转换之后超出了指定的范围，error不为nil
num2, err := strconv.ParseInt(str2, 10, 8)
if err != nil {
    fmt.Println(err)
}
fmt.Println(num2)

var str3 string = "3.1234567890123456789"
// 第一个参数：需要转换的数据
// 第二个参数：转换为多少位小数，32 or 64
// ParseFloat同样有两个返回值，如果能够正常转换则错误为nil，否则不为nil
num3, err := strconv.ParseFloat(str3, 32)
if err != nil {
    // 例如：把字符串"3.14abc"转换为小数就会报错，因为"3.14abc"不是一个小数
    fmt.Println(err)
}
fmt.Println(num3)

var str4 string = "true"
// 第一个参数：需要转换的数据
// ParseBool同样有两个返回值，如果能够正常转换则错误为nil，否则不为nil
num4, _ := strconv.ParseBool(str4)
fmt.Println(num4)
}

```

- 字符串类型转换为数值类型时,如果不能转换除了返回error以外,还会返回对应类型的默认值

```

package main
import "fmt"
func main() {
    var str1 string = "abc"
    num1, _ := strconv.ParseInt(str1, 10, 32)
    fmt.Println(num1) // 0

    num2, _ := strconv.ParseFloat(str1, 32)
    fmt.Println(num2) // 0

    num3, _ := strconv.ParseBool(str1)
    fmt.Println(num3) // false
}

```

- 看完上面的代码有没有种想打人的感觉? 如果有那么请继续往下看
- 字符串类型和整型快速转换

```

package main
import "fmt"
func main() {
    var num1 int32 = 110
    // 快速将整型转换为字符串类型
    // 注意: Itoa方法只能接受int类型
    var str1 string = strconv.Itoa(int(num1))
    fmt.Println(str1)

    var str2 string = "666"
    // 快速将字符串类型转换为整型
    // 注意: Atoi方法返回两个值, 一个值是int, 一个值是error
    // 如果字符串能被转换为int, 那么error为nil, 否则不为nil
    num2, err := strconv.Atoi(str2)
    if err != nil {
        fmt.Println(err)
    }
    fmt.Println(num2)
}

```

- 数值类型转字符串类型其它方式

```

package main
import "fmt"
func main() {
    var num1 int32 = 110
    // Sprintf函数和Printf函数很像, 只不过不是输出而将格式化的字符串返回给我们
    var str1 string = fmt.Sprintf("%d", num1)
    fmt.Println(str1)

    var num2 float32 = 3.14
    var str2 string = fmt.Sprintf("%f", num2)
    fmt.Println(str2)

    var num3 bool = true
    var str3 string = fmt.Sprintf("%t", num3)
    fmt.Println(str3)
}

```

Go语言常量

- 和C语言一样Go语言中的常量也分为 整型常量、实型常量、字符常量、字符串常量、自定义常量
- 自定义常量
 - C语言自定义常量: `const 数据类型 常量名称 = 值;`

```

#include <stdio.h>
int main(int argc, const char * argv[])
{
    const float PI = 998;
    PI = 110; // 报错
    printf("PI = %d\n", PI );
    return 0;
}

```

- Go语言自定义常量: `const 常量名称 数据类型 = 值` or `const 常量名称 = 值`

```
package main
import "fmt"
func main() {
    //const PI float32 = 3.14
    //PI = 110 // 报错
    //fmt.Println("PI = ", PI )

    const PI = 3.14
    PI = 110 // 报错
    fmt.Println("PI = ", PI )
}
```

- 除此之外Go语言还支持 一次性定义多个常量

```
package main
import "fmt"
func main() {
    // 多重赋值方式
    const num1, num2 int = 100, 200
    fmt.Println("num1 = ", num1)
    fmt.Println("num2 = ", num2)

    // 常量组方式
    const (
        num3 = 100
        num4 = 200
    )
    fmt.Println("num3 = ", num3)
    fmt.Println("num4 = ", num4)

    // 常量组+多重赋值
    const (
        num5, num6 = 100, 200
        num7 = 300
    )
    fmt.Println("num5 = ", num5)
    fmt.Println("num6 = ", num6)
    fmt.Println("num7 = ", num7)
}
```

- Go语言自定义常量注意点

- 定义的局部变量或者导入的包没有被使用, 那么编译器会报错, 无法编译运行
- 但是定义的常量没有被使用, 编译器不会报错, 可以编译运行

```
package main
import "fmt"
func main() {
    // 可以编译运行
    const PI float32 = 3.14
}
```

- 在常量组中, 如果上一行常量有初始值, 但是下一行没有初始值, 那么下一行的值就是上一行的值

```
package main
import "fmt"
func main() {
const (
    num1 = 998
    num2 // 和上一行的值一样
    num3 = 666
    num4 // 和上一行的值一样
    num5 // 和上一行的值一样
)
fmt.Println("num1 = ", num1) // 998
fmt.Println("num2 = ", num2) // 998
fmt.Println("num3 = ", num3) // 666
fmt.Println("num4 = ", num4) // 666
fmt.Println("num5 = ", num5) // 666

const (
    num1, num2 = 100, 200
    num3, num4 // 和上一行的值一样, 注意变量个数必须也和上一行一样
)
fmt.Println("num1 = ", num1)
fmt.Println("num2 = ", num2)
fmt.Println("num3 = ", num3)
fmt.Println("num4 = ", num4)
}
```

- 枚举常量
 - C语言中枚举类型的本质就是整型常量
 - Go语言中没有C语言中明确意义上的enum定义, 但是可以借助iota标识符来实现枚举类型
- C语言枚举格式:

```
enum 枚举名 {
    枚举元素1,
    枚举元素2,
    ... ..
};
```

- C语言枚举中, 如果没有指定初始值, 那么从0开始递增

```
#include <stdio.h>
int main(int argc, const char * argv[])
{
    enum Gender{
        male,
        female,
        yao,
    };

    // enum Gender g = male;
    // printf("%d\n", g); // 0
    // enum Gender g = female;
    // printf("%d\n", g); // 1
}
```

```
enum Gender g = yao;
printf("%d\n", g); // 2
return 0;
}
```

- C语言枚举中, 如果指定了初始值, 那么从指定的数开始递增

```
#include <stdio.h>
int main(int argc, const char * argv[])
{
    enum Gender{
        male = 5,
        female,
        yao,
    };
    // enum Gender g = male;
    // printf("%d\n", g); // 5
    // enum Gender g = female;
    // printf("%d\n", g); // 6
    enum Gender g = yao;
    printf("%d\n", g); // 7
    return 0;
}
```

- Go语言实现枚举格式

```
const(
    枚举元素1 = iota
    枚举元素2 = iota
    ... ..
)
```

- 利用iota标识符标识符实现从0开始递增的枚举

```
package main
import "fmt"
func main() {
    const (
        male = iota
        female = iota
        yao = iota
    )
    fmt.Println("male = ", male) // 0
    fmt.Println("male = ", female) // 1
    fmt.Println("male = ", yao) // 2
}
```

- iota注意点:
 - 在同一个常量组中, iota从0开始递增, 每一行递增1
 - 在同一个常量组中, 只要上一行出现了iota, 那么后续行就会自动递增

```

package main
import "fmt"
func main() {
    const (
        male = iota // 这里出现了iota
        female // 这里会自动递增
        yao
    )
    fmt.Println("male = ", male) // 0
    fmt.Println("male = ", female) // 1
    fmt.Println("male = ", yao) // 2
}

```

- 在同一个常量组中,如果iota被中断, 那么必须显示恢复

```

package main
import "fmt"
func main() {
    const (
        male = iota
        female = 666 // 这里被中断, 如果没有显示恢复, 那么下面没有赋值的常量都和上一行
        yao
    )
    fmt.Println("male = ", male) // 0
    fmt.Println("male = ", female) // 666
    fmt.Println("male = ", yao) // 666
}

```

```

package main
import "fmt"
func main() {
    const (
        male = iota
        female = 666 // 这里被中断
        yao = iota // 这里显示恢复, 会从当前常量组第一次出现iota的地方开始, 每一行递增
    )
    fmt.Println("male = ", male) // 0
    fmt.Println("male = ", female) // 666
    fmt.Println("male = ", yao) // 2
}

```

- iota也支持常量组+多重赋值, 在同一行的iota值相同

```
package main
import "fmt"
func main() {
    const (
        a, b = iota, iota
        c, d = iota, iota
    )
    fmt.Println("a = ", a) // 0
    fmt.Println("b = ", b) // 0
    fmt.Println("c = ", c) // 1
    fmt.Println("d = ", d) // 1
}
```

- iota自增默认数据类型为int类型, 也可以显示指定类型

```
package main
import "fmt"
func main() {
    const (
        male float32 = iota // 显示指定类型, 后续自增都会按照指定类型自增
        female
        yao
    )
    fmt.Printf("%f\n", male) // 0.0
    fmt.Printf("%f\n", female) // 1.0
    fmt.Printf("%f\n", yao) // 2.0
    fmt.Println("male = ", reflect.TypeOf(female)) // float32
}
```

- Go语言fmt包实现了类似C语言printf和scanf的格式化I/O, 格式化动作源自C语言但更简单
##输出函数
- func Printf(format string, a ...interface{}) (n int, err error)
 - 和C语言用法几乎一模一样, 只不过新增了一些格式化符号

```
package main
import "fmt"
func main() {
    name := "微信搜索: 代码情缘"
    age := 33
    fmt.Printf("name = %s, age = %d\n", name, age) // name = 1nj, age = 33
}
```

- 值得注意的是, 输出十进制只能通过%d, 不能像C语言一样通过%i
- 除了和C语言一样, 可以通过%o、%x输出八进制和十六进制外, 还可以直接通过%b输出二进制

```
package main
import "fmt"
func main() {
    num := 15
    fmt.Printf("十进制 = %d\n", num)
    fmt.Printf("八进制 = %o\n", num)
    fmt.Printf("十六进制 = %x\n", num)
    fmt.Printf("二进制 = %b\n", num)
}
```

- 除此之外,Go语言还增加了%T控制符,用于输出值的类型

```
package main
import "fmt"
func main() {
    type Person struct {
        name string
        age int
    }
    num1 := 10
    num2 := 3.14
    per := Person{"lnj", 33}
    fmt.Printf("num1 = %T\n", num1) // int
    fmt.Printf("num2 = %T\n", num2) // float64
    fmt.Printf("per = %T\n", per) // main.Person
}
```

- 除此之外,Go语言还增加了%v控制符,用于打印所有类型数据
 - Go语言中输出某一个值,很少使用%d%f等,一般都使用%v即可
 - 输出复合类型时会自动生成对应格式后再输出

```
package main
import "fmt"
func main() {
    type Person struct {
        name string
        age int
    }
    num1 := 10
    num2 := 3.14
    per := Person{"lnj", 33}
    // 此时相当于把%v当做%d
    fmt.Printf("num1 = %v\n", num1) // 10
    // 此时相当于把%v当做%f
    fmt.Printf("num2 = %v\n", num2) // 3.14
}
```

- Go语言Printf函数其它特性,如宽度、标志、精度、长度、转移符号等,和C语言一样.

- func Println(a ...interface{}) (n int, err error)
 - 采用默认格式将其参数格式化并写入标准输出,
 - 输出之后 会 在结尾处添加换行
 - 传入多个参数时,会自动在相邻参数之间添加空格
 - 传入符合类型数据时,会自动生成对应格式后再输出

```
package main
import "fmt"
func main() {
    num1 := 10
    num2 := 3.14
    fmt.Println(num1, num2) // 10 3.14
    fmt.Println("num1 =", num1, "num2 =", num2) // num1 = 10 num2 = 3.14
}
```

```

type Person struct {
    name string
    age int
}
per := Person{"lnj", 33}
fmt.Println(per) // {lnj 33}
}

```

- func Print(a ...interface{}) (n int, err error)
 - 和Println几乎一样
 - 输出之后 不会 在结尾处添加换行
 - 传入多个参数时, 只有两个相邻的参数 都不是 字符串, 才会在相邻参数之间添加空格
 - 传入符合类型数据时, 会自动生成对应格式后再输出

```

package main
import "fmt"
func main() {
    num1 := 10
    num2 := 3.14
    fmt.Print(num1, num2) // 10 3.14
    fmt.Print("num1 =", num1, "num2 =", num2) // num1 =10 num2 =3.14

    type Person struct {
        name string
        age int
    }
    per := Person{"lnj", 33}
    fmt.Print(per) // {lnj 33}
}

```

- 以下三个函数和Printf/Println/Print函数一样, 只不过上面三个函数是输出到标准输出, 而下面三个函数可以通过w指定输出到什么地方
- func Fprintf(w io.Writer, format string, a ...interface{}) (n int, err error)
- func Fprintln(w io.Writer, a ...interface{}) (n int, err error)
- func Fprint(w io.Writer, a ...interface{}) (n int, err error)

```

package main
import (
    "fmt"
    "net/http"
    "os"
)
func main() {
    // os.Stdout 写入到标准输出
    name := "lnj"
    age := 33
    // 第一个参数: 指定输出到什么地方
    // 第二个参数: 指定格式控制字符串
    // 第三个参数: 指定要输出的数据
    fmt.Fprintf(os.Stdout, "name = %s, age = %d\n", name, age)

    // http.ResponseWriter 写入到网络响应
}

```

```

http.HandleFunc("/", func(writer http.ResponseWriter, request *http.Request)
{
    fmt.Fprintf(writer, "name = %s, age = %d\n", name, age)
})
http.ListenAndServe(":8888", nil)
}

```

- 以下三个函数和Printf/Println/Print函数一样, 只不过上面三个函数是输出到标准输出, 而下面三个函数不会输出, 而是将字符串返回给我们
- func Sprintf(format string, a ...interface{}) string
- func Sprint(a ...interface{}) string
- func Sprintln(a ...interface{}) string

```

package main
import (
    "fmt"
    "net/http"
    "os"
)
func main() {
    name := "lnj"
    age := 33
    // 按照指定的格式生成字符串
    str := fmt.Sprintf("name = %s, age = %d\n", name, age)
    // 输出生成的字符串
    fmt.Println(str)
}

```

输入函数

- func Scanf(format string, a ...interface{}) (n int, err error)
 - 和C语言用法几乎一模一样, 但是只能输入一行数据

```

package main
import "fmt"
func main() {
    var num1 int
    var num2 int
    fmt.Scanf("%d%d", &num1, &num2)
    fmt.Println(num1, num2)
}

```

- func Scan(a ...interface{}) (n int, err error)
 - 和C语言scanf函数几乎一样, 只不过不用指定格式化字符串

```

package main
import "fmt"
func main() {
    var num1 int
    var num2 int
    fmt.Scan(&num1, &num2)
    fmt.Println(num1, num2)

    var num3 float32
    var num4 float32
    fmt.Scan(&num3, &num4)
    fmt.Println(num3, num4)
}

```

- func Scanln(a ...interface{}) (n int, err error)
 - 和C语言用法几乎一模一样, 只不过不用指定格式化字符串, 并且只能输入一行数据

```

package main
import "fmt"
func main() {
    var num1 int
    var num2 int
    fmt.Scanln(&num1, &num2)
    fmt.Println(num1, num2)
}

```

- 以下三个函数和Scan/Scanln/Scanf函数一样, 只不过上面三个函数是从标准输入读取数据, 而下面三个函数可以通过r指定从哪读取数据
- func Fscanf(r io.Reader, format string, a ...interface{}) (n int, err error)
- func Fscanln(r io.Reader, a ...interface{}) (n int, err error)
- func Fscan(r io.Reader, a ...interface{}) (n int, err error)

```

package main
import "fmt"
func main() {
    var num1 int
    var num2 int
    // 第一个参数: 指定从哪读取数据
    // 第二个参数: 指定格式控制字符串
    // 第三个参数: 指定要输出的数据
    fmt.Fscanf(os.Stdin, "%d%d", &num1, &num2)
    fmt.Println(num1, num2)

    s := strings.NewReader("\nj 33")
    var name string
    var age int
    // 从指定字符串中扫描出想要的数据
    // 注意:
    fmt.Fscanf(s, "%s%d", &name, &age)
    fmt.Println("name =", name, "age =", age)
}

```

- 以下三个函数和Scan/Scanln/Scanf函数一样, 只不过上面三个函数是从标准输入读取数据, 而下面三个函数是从字符串中读取数据
- func Sscan(str string, a ...interface{}) (n int, err error)
- func Sscanf(str string, format string, a ...interface{}) (n int, err error)
- func Sscanln(str string, a ...interface{}) (n int, err error)

```
package main
import "fmt"
func main() {
    str := "Inj 33"
    var name string
    var age int
    //fmt.Sscanf(str, "%s %d",&name, &age)
    //fmt.Sscanln(str,&name, &age)
    fmt.Sscan(str,&name, &age)
    fmt.Println("name =",name, "age =",age)
}
```

go命令行操作指令

- 标准go语言项目文件目录格式
 - 项目文件夹就是GOPATH指向的文件夹
 - src文件夹是专门用于存放源码文件的
 - main文件夹是专门用于存储package main包相关源码文件的
 - 其它文件夹是专门用于存储除package main包以外源码文件的
 - bin文件夹是专门用于存储编译之后的可执行程序的
 - pag文件夹是专门用于存储编译之后的.a文件的

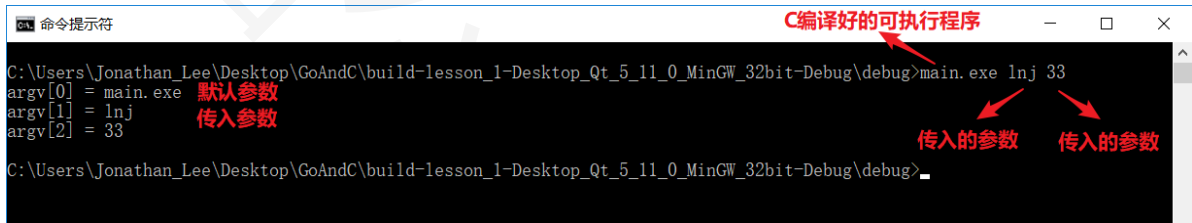
```
|---项目文件夹
|-----|-----src文件夹
|-----|-----main文件夹
|-----|-----其它文件夹
|-----|-----bin文件夹
|-----|-----pkg文件夹
```

- `go version` 查看当前安装的go版本
- `go env` 查看当前go的环境变量
- `go fmt` 格式化代码
 - 会将指定文件中凌乱的代码按照go语言规范格式化
- `go run 命令文件` 编译并运行go程序
 - package main包中包含main函数的文件, 我们称之为命令文件
 - 其它包中的文件, 我们称之为源码文件
- `go build` 编译检查
 - 对于非命令文件只会执行编译检查, 不会产生任何文件
 - 对于命令文件除了编译检查外,还会在当前目录下生成一个可执行文件
 - 对应只想编译某个文件, 可以在命令后面指定文件名称 `go build 文件名称`
- `go install` 安装程序
 - 对于非命令文件会执行编译检查, 并生成.a结尾的包, 放到 \$GOPATH/pkg目录中
 - 对于命令文件会执行编译检查, 并生成可执行程序, 放到\$GOPATH/bin目录中

通过os包获取命令行参数

- C语言中的命令行参数
 - argc: argv中保存数据的个数
 - argv: 默认情况下系统只会传入一个值, 这个值就是main函数执行文件的路径
 - 我们可以通过配置开发工具, 或者命令行运行时以 空格+参数 形式传递其它参数
 - 注意点: 无论外界传入的是什么类型, 我们拿到的都是 字符串类型

```
#include <stdio.h>
int main(int argc, const char * argv[])
{
    for(int i = 0; i < argc; i++){
        printf("argv[%d] = %s\n", i, argv[i]);
    }
    return 0;
}
```



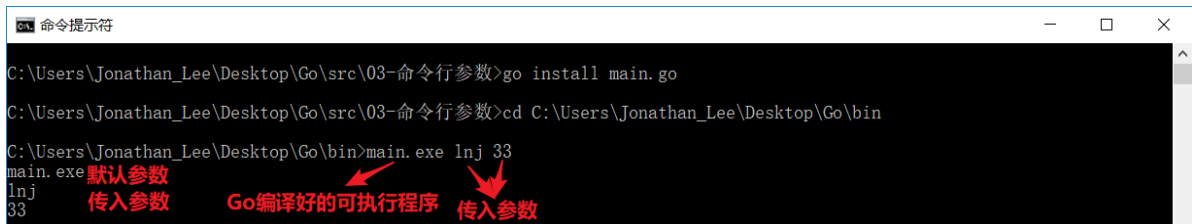
```
C:\Users\Jonathan_Lee\Desktop\GoAndC\build-lesson_1-Desktop_Qt_5_11_0_MinGW_32bit-Debug\debug>main.exe lnj 33
argv[0] = main.exe
argv[1] = lnj
argv[2] = 33
C:\Users\Jonathan_Lee\Desktop\GoAndC\build-lesson_1-Desktop_Qt_5_11_0_MinGW_32bit-Debug\debug>
```

- Go语言中的命令行参数
 - Go语言中main函数没有形参, 所以不能直接通过main函数获取命令行参数
 - 想要获取命令行参数必须导入os包, 通过os包的Args获取
 - 注意点: 无论外界传入的是什么类型, 我们拿到的都是 字符串类型

```
package main

import (
    "fmt"
    "os" // 用于获取命令行参数的包
)

func main() {
    // 1. 获取传入参数个数
    num := len(os.Args)
    // 2. 打印所有获取到的参数
    for i := 0; i < num; i++ {
        fmt.Println(os.Args[i])
    }
}
```



```
C:\Users\Jonathan_Lee\Desktop\Go\src\03-命令行参数>go install main.go
C:\Users\Jonathan_Lee\Desktop\Go\src\03-命令行参数>cd C:\Users\Jonathan_Lee\Desktop\Go\bin
C:\Users\Jonathan_Lee\Desktop\Go\bin>main.exe lnj 33
main.exe
lnj
33
```

通过flag包获取命令行参数

- Go语言中除了可以通过os包获取命令行参数以外,还可以通过flag包获取命令行参数

```
package main
import (
    "flag"
    "fmt"
)
func main() {
    /*
        flag.Xxxx(name, value, usage)
        第一个参数: 命令行参数名称
        第二个参数: 命令行参数对应的默认值
        第三个参数: 命令行参数对应的说明
    */
    // 1. 设置命令行参数
    name := flag.String("name", "lnj", "请输入人的姓名")
    age := flag.Int("age", 33, "请输入人的年龄")
    // 2. 将命令行参数解析到注册的参数
    flag.Parse()
    // 3. 使用命令行参数
    // 注意flag对应方法返回的都是指针类型, 所以使用时必须通过指针访问
    fmt.Println("name = ", *name)
    fmt.Println("age = ", *age)
}
```



```
C:\Users\Jonathan_Lee\Desktop\Go\bin>main.exe 直接运行不传递参数, 会使用默认值
name = lnj 打印默认值
age = 33

C:\Users\Jonathan_Lee\Desktop\Go\bin>main.exe -name=zs -age=18 运行运行时可以通过-参数名称=值方式传递参数
name = zs 打印传递的参数
age = 18

C:\Users\Jonathan_Lee\Desktop\Go\bin>
```

- flag获取命令行参数第二种写法

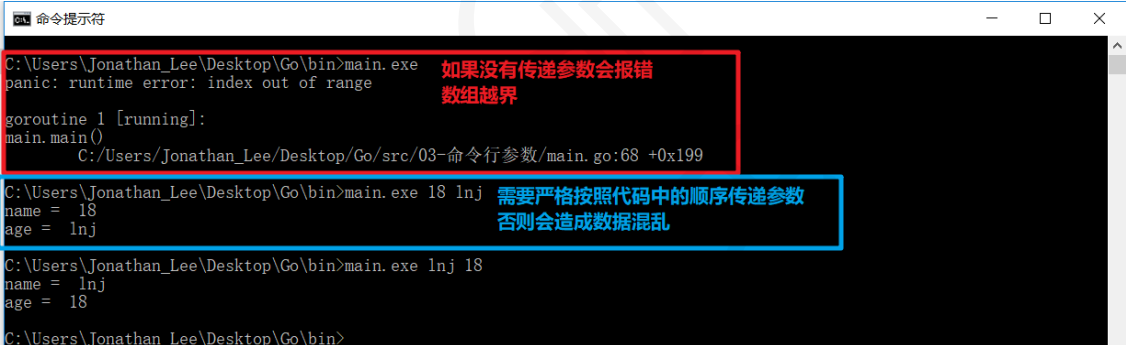
```
package main
import (
    "flag"
    "fmt"
)
func main() {
    /*
        flag.Xxxx(*type, name, value, usage)
        第一个参数: 保存命令行参数变量地址
        第二个参数: 命令行参数名称
        第三个参数: 命令行参数对应的默认值
        第四个参数: 命令行参数对应的说明
    */
    // 1. 定义变量, 保存命令行参数的值
    var name string
    var age int
    // 2. 设置命令行参数
    flag.StringVar(&name, "name", "lnj", "请输入人的姓名")
    flag.IntVar(&age, "age", 33, "请输入人的姓名")
    // 3. 注册解析命令行参数
    flag.Parse()
    // 4. 使用命令行参数
```

```
fmt.Println("name = ", name)
fmt.Println("age = ", age)
}
```

os包和flag包获取命令行参数对比

- 通过os包获取命令行参数
 - 如果用户没有传递参数 会 报错
 - 需要 严格按照代码中的顺序传递参数, 否则 会 造成数据混乱
 - 不能 指定参数的名称
 - 获取到的数据都是 字符串 类型

```
package main
import (
    "os"
    "fmt"
)
int main(){
    name := os.Args[1]
    age := os.Args[2]
    fmt.Println("name = ", name)
    fmt.Println("age = ", age)
}
```

- 

- 通过flag包获取命令行参数
 - 如果用户没有传递参数 不会 报错
 - 不需要 严格按照代码中的顺序传递参数, 不会 造成数据混乱
 - 可以 指定参数的名称
 - 获取到的数据是我们自己指定的类型

```
package main
import (
    "flag"
    "fmt"
)
int main(){
    name := flag.String("name", "lnj", "请输入人的姓名")
    age := flag.Int("age", 33, "请输入人的年龄")
    // 2.注册解析命令行参数
    flag.Parse()
    // 3.使用命令行参数
    // 注意flag对应方法返回的都是指针类型, 所以使用时必须通过指针访问
    fmt.Println("name = ", *name)
```

```
fmt.Println("age = ", *age)
}
```

```
命令提示符
C:\Users\Jonathan_Lee\Desktop\Go\bin>main.exe
name = lnj
age = 33

C:\Users\Jonathan_Lee\Desktop\Go\bin>main.exe -age=18 -name=zs
name = zs
age = 18

C:\Users\Jonathan_Lee\Desktop\Go\bin>main.exe -name=zs -age=18
name = zs
age = 18

C:\Users\Jonathan_Lee\Desktop\Go\bin>
```

如果没有传递参数不会报错
会使用默认值

不需要严格按照代码中的顺序传递参数
因为系统会根据参数名称自动匹配和解析

算数运算符

- 算数运算符和C语言几乎一样

运算符	描述	实例
+	相加	A + B
-	相减	A - B
*	相乘	A * B
/	相除	B / A
%	求余	B % A
++	自增	A++
--	自减	A--

- 注意点:
 - 只有相同类型的数据才能进行运算

```
package main
import "fmt"
int main(){
    var num1 int32 = 10
    //var num2 int64 = num1 // 类型不同不能进行赋值运算
    var num2 int64 = int64(num1) // 类型不同不能进行赋值运算
    fmt.Println(num2)

    var num3 int32 = 10
    var num4 int64 = 20
    //var res int64 = num3 + num4 // 类型不同不能进行算数运算
    var res1 int64 = int64(num3) + num4 // 类型不同不能进行算数运算
    fmt.Println(res1)

    var num5 int32 = 10
    var num6 int64 = 20
    //var res2 bool = (num5 == num6) // 类型不同不能进行关系运算
    var res2 bool = (num5 == int32(num6)) // 类型不同不能进行关系运算
    fmt.Println(res2)

    // ... .. 其它以此类推
```

```
}
```

- Go语言中++、--运算符不支持前置
 - 错误写法: ++i; --i;
- Go语言中++、--是语句,不是表达式,所以必须独占一行
 - 错误写法: a = i++; return i++;

```
package main
import "fmt"
func main() {
    num1 := 0
    num1++
    fmt.Println(num1)
    //++num1 // 编译报错, Go语言中++只能后置,不能前置
    //fmt.Println(num1)
    //var num2 int = num1++ // 编译报错, num1++是语句不是表达式, 所以必须独占一行
    //fmt.Println(num2)
}
```

- Go语言中字符串支持利用+号进行拼接

```
package main
import "fmt"
func main() {
    str := "abc" + "def"
    //fmt.Println(str)
}
```

关系算符

- 关系算符和C语言一样

运算符	描述	实例
==	检查两个值是否相等, 如果相等返回 True 否则返回 False。	A == B
!=	检查两个值是否不相等, 如果不相等返回 True 否则返回 False。	A != B
>	检查左边值是否大于右边值, 如果是返回 True 否则返回 False。	A > B
<	检查左边值是否小于右边值, 如果是返回 True 否则返回 False。	A < B
>=	检查左边值是否大于等于右边值, 如果是返回 True 否则返回 False。	A >= B
<=	检查左边值是否小于等于右边值, 如果是返回 True 否则返回 False。	A <= B

- 注意点:
 - 和C语言不通的是, Go语言中关系运算符只能返回true和false

逻辑运算符

- 逻辑运算符和C语言一样

运算符	描述	实例
&&	如果两边的操作数都是 True，则条件 True，否则为 False。	A && B
	如果两边的操作数有一个 True，则条件 True，否则为 False。	A B
!	如果条件为 True，则逻辑 NOT 条件 False，否则为 True。	!A

- 注意点:
 - 和C语言不通的是, Go语言中关系运算符只能返回true和false
 - 逻辑非只能用于true和false

位运算符

- 位运算符和C语言几乎一样

运算符	描述	实例
&	参与运算的两数各对应的二进位相与, 对应位只要都是1结果就为1	A & B
	参与运算的两数各对应的二进位相或, 对应位只要其中一个是1结果就为1	A B
^	参与运算的两数各对应的二进位相异或, 对应位只要不同结果就是1	A ^ B
<<	左移运算符, 左移n位就是乘以2的n次方	A << 2
>>	右移运算符, 右移n位就是除以2的n次方	B >> 2
&^	逻辑清零运算符, B对应位是1, A对应位清零, B对应位是0, A对应位保留原样	A &^ B

- 新增一个&^运算符

```
int main(){
    /*
        0110      a
        &^1011    b 如果b位位1, 那么结果为0, 否则结果为a位对应的值
        -----
        0100
    */
    a1 := 6
    b1 := 11
    res1 := a1 &^ b1
    fmt.Println("res1 = ", res1) // 4

    /*
        1011      a
        &^1101    b 如果b位位1, 那么结果为0, 否则结果为a位对应的值
        -----
        0010
    */
    a2 := 11
    b2 := 13
    res2 := a2 &^ b2
    fmt.Println("res2 = ", res2) // 2
}
```

赋值运算符

- 赋值运算符和C语言几乎一样
 - 新增一个`&^=`运算符

运算符	描述	实例
<code>=</code>	将右边赋值给左边	<code>C = A + B</code> 将 <code>A + B</code> 表达式结果赋值给 <code>C</code>
<code>+=</code>	相加后再赋值	<code>C += A</code> 等于 <code>C = C + A</code>
<code>-=</code>	相减后再赋值	<code>C -= A</code> 等于 <code>C = C - A</code>
<code>*=</code>	相乘后再赋值	<code>C *= A</code> 等于 <code>C = C * A</code>
<code>/=</code>	相除后再赋值	<code>C /= A</code> 等于 <code>C = C / A</code>
<code>%=</code>	求余后再赋值	<code>C %= A</code> 等于 <code>C = C % A</code>
<code><<=</code>	左移赋值	<code>C <<= 2</code> 等于 <code>C = C << 2</code>
<code>>>=</code>	右移赋值	<code>C >>= 2</code> 等于 <code>C = C >> 2</code>
<code>&=</code>	位逻辑与赋值	<code>C &= 2</code> 等于 <code>C = C & 2</code>
<code>^=</code>	位逻辑或赋值	<code>C ^= 2</code> 等于 <code>C = C ^ 2</code>
<code> =</code>	位逻辑异或赋值	<code>C = 2</code> 等于 <code>C = C 2</code>
<code>&^=</code>	位逻辑清零赋值	<code>C &^= 2</code> 等于 <code>C = C &^ 2</code>

其它运算符

运算符	描述	实例
<code>&</code>	返回变量存储地址	<code>&a</code> ; 将给出变量的实际地址
<code>*</code>	访问指针指向内存	<code>*p</code> ; 访问指针 <code>p</code> 指向内存

```
package main
import "fmt"
int main(){
    var num int = 666
    var p *int = &num
    fmt.Println(num)
    fmt.Println(*p)
    num = 777
    fmt.Println(num)
    *p = 999
    fmt.Println(num)
}
```

- 注意点
 - 指针类型只支持相等运算, 不能做加减运算

```
#include <stdio.h>
int main()
{
    int ages[3] = {19, 23, 22};
    int *arrayP = &ages[0];
    printf("ages[0] = %i\n", *(arrayP + 0)); // *(arrayP + 0) == *arrayP
    printf("ages[1] = %i\n", *(arrayP + 1));
    printf("ages[2] = %i\n", *(arrayP + 2));
    return 0;
}
```

```
package main
import "fmt"
int main(){
    var ages [3]int = [3]int{19, 23, 22}
    var p *int = &ages[0]
    //fmt.Println(&ages[0])
    //fmt.Println(*p) // 19
    fmt.Println(*(p + 0)) // 编译报错
}
```

运算符优先级

- 和C语言一样, 只需记住()优先级最高即可

Go语言流程控制基本概念

- Go语言流程控制和C语言一样, 也有三大流程控制结构
 - 顺序结构(默认结构)
 - 选择结构(if / switch)
 - 循环结构(for)

选择结构if

- 和C语言不同的是
 - 条件表达式的值必须是布尔类型(Go语言中没有非零即真的概念)
 - 条件表达式前面可以添加初始化语句
 - 不需要编写圆括号
 - 左大括号必须和条件语句在同一行
- 第一种格式:
 - 条件表达式结果为true,那么执行if后面{}中代码

```
if 初始化语句; 条件表达式{
    语句块;
}
```

```

package main
import "fmt"
func main() {
    // 如果后续需要用到age变量，可以将变量放到if外面
    age := 18
    if age >= 18{
        fmt.Println("成年人")
    }
}

```

```

package main
import "fmt"
func main() {
    // 如果后续不需要用到age变量，可以将变量放到条件表达式前面
    if age := 18; age >= 18{
        fmt.Println("成年人")
    }
}

```

- 第二种格式:
 - 条件表达式结果为true,那么执行if后面{}中代码
 - 否则执行else后面{}中代码

```

if 初始化语句; 条件表达式{
    语句块;
}else{
    语句块;
}

```

```

package main
import "fmt"
func main() {
    if age := 18; age >= 18 {
        fmt.Println("成年人")
    }else{
        fmt.Println("未成年人")
    }
}

```

- 第三种格式:
 - if后面条件表达式结果为true,那么执行if后面{}中代码
 - 否则判断else if条件表达式是否为true,为true执行else if后面{}中代码
 - 否则依次判断后续else if条件表达式是否为true,哪个为true就执行哪个else if后面{}中代码
 - 都不满足则执行else后面{}中代码

```

if 初始化语句; 条件表达式{
    语句块;
}else if 条件表达式{
    语句块;
}
... ..
else{
    语句块;
}

```

```

package main
import "fmt"
func main() {
    if age := 18; age > 55{
        fmt.Println("老年人")
    }else if age >= 40{
        fmt.Println("中年人")
    }else if age >= 18{
        fmt.Println("成年人")
    }else{
        fmt.Println("未成年人")
    }
}

```

- 值得一提的是Go语言中没有C语言中的三目运算符, 所以C语言中三目能干的在Go语言中都只能通过if else的形式来完成

选择结构switch

- 和C语言不同的是
 - 和if一样,表达式前面可以添加初始化语句
 - 和if一样,不需要编写圆括号
 - 和if一样,左大括号必须和表达式在同一行
 - case表达式的值不一定要是常量, 甚至可以不用传递
 - 一个case后面可以有多个表达式, 满足其中一个就算匹配
 - case后面不需要添加break
 - 可以在case语句块最后添加fallthrough,实现case穿透
 - case后面定义变量不需要添加{}明确范围
- 格式

```

switch 初始化语句; 表达式{
    case 表达式1, 表达式2:
        语句块;
    case 表达式1, 表达式2:
        语句块;
    default:
        语句块;
}

```

```

package main
import "fmt"
func main() {

```

```

switch num := 3;num {
    case 1:
        fmt.Println("星期一")
    case 2:
        fmt.Println("星期二")
    case 3:
        fmt.Println("星期三")
    case 4:
        fmt.Println("星期四")
    case 5:
        fmt.Println("星期五")
    case 6:
        fmt.Println("星期六")
    case 7:
        fmt.Println("星期日")
    default:
        fmt.Println("Other...")
}
}

```

```

package main
import "fmt"
func main() {
    switch num := 3;num {
    case 1,2,3,4,5:
        fmt.Println("工作日")
    case 6,7:
        fmt.Println("非工作日")
    default:
        fmt.Println("Other...")
    }
}

```

- 注意点:
 - case后面不用编写break, 不会出现case穿透问题
 - 如果想让case穿透, 必须在case语句块最后添加fallthrough关键

```

package main
import "fmt"
func main() {
    switch num := 3;num {
    case 1:
        fallthrough
    case 2:
        fallthrough
    case 3:
        fallthrough
    case 4:
        fallthrough
    case 5:
        fmt.Println("工作日")
    case 6:
        fallthrough
    case 7:
        fmt.Println("非工作日")
    default:

```

```

        fmt.Println("other...")
    }
}

```

- case后面不仅仅可以放常量,还可以放变量和表达式

```

package main
import "fmt"
func main() {
    value := 2
    switch num:=1; num {
    case value: // 变量
        fmt.Println("num等于value")
    default:
        fmt.Println("num不等于value")
    }
}

```

```

package main
import "fmt"
func main() {
    value := 2
    switch num:=1; num {
    case getValue(): // 函数
        fmt.Println("num等于value")
    default:
        fmt.Println("num不等于value")
    }
}
func getValue() int {
    return 1
}

```

```

package main
import "fmt"
func main() {
    switch num:=18; {
    case num >=0 && num <=10: // 表达式
        fmt.Println("num是一个0~10之间的数")
    case num >10 && num <=20:
        fmt.Println("num是一个11~20之间的数")
    default:
        fmt.Println("num是一个大于20的数")
    }
}

```

- case后面定义变量不用添加{}明确作用于范围

```

package main
import "fmt"
func main() {
    switch num := 1; num {
    case 1:
        value := 10 // 不会报错
        fmt.Println(value)
    default:
        fmt.Println("Other...")
    }
}

```

- 其它特性和C语言一样

循环结构for

- 和C语言不同的是
 - 和if一样,条件表达式的值必须是布尔类型
 - 和if一样,不需要编写圆括号
 - 和if一样,左大括号必须和表达式在同一行
- 格式:

```

for 初始化表达式; 循环条件表达式; 循环后的操作表达式 {
    循环体语句;
}

```

```

package main
import "fmt"
func main() {
    for i:=0; i<10; i++){
        fmt.Println(i)
    }
}

```

- Go语言中没有while/dowhile循环, 所以可以通过如下格式实现C语言中while循环用法

```

package main
import "fmt"
func main() {
    i:=0
    for i<10 {
        fmt.Println(i)
        i++
    }
}

```

- 最简单死循环

```
package main
import "fmt"
func main() {
    for{
        fmt.Println("根本停不下来")
    }
}
```

- 除了实现基本的循环结构以外,Go语言还实现了一种高级for循环 `for...range` 循环
 - `for...range` 循环可以快速完成对字符串、数组、slice、map、channel遍历
- 格式

```
for 索引, 值 := range 被遍历数据{
}
```

```
package main
import "fmt"
func main() {
    // 1. 定义一个数组
    arr := []int{1, 3, 5}
    // 2. 快速遍历数组
    // i用于保存当前遍历到数组的索引
    // v用于保存当前遍历到数组的值
    for i, v := range arr{
        fmt.Println(i, v)
    }
}
```

四大跳转

- 和C语言一样,Go语言中也有四大跳转语句, 分别是return、break、continue、goto
- break语句
 - Go语言中的break语句可以用于,立即跳出switch、for和select
 - 但不同的是Go语言中的break语句可以指定标签

```
package main
import "fmt"
func main() {
    for i:=0; i<10; i++){
        if(i == 5){
            break // 跳出所在循环
        }
        fmt.Println(i)
    }
}
```

- 利用break跳转到指定标签
 - 标签必须在使用之前定义
 - 标签后面只能跟switch和循环语句, 不能插入其它语句
 - 跳转到标签之后switch和循环不会再次被执行

```

package main
import "fmt"
func main() {
outer:
    switch num:=2; num {
    case 1:
        fmt.Println(1)
    case 2:
        fmt.Println(2)
        break outer
    default:
        fmt.Println("other")
    }
    fmt.Println("come here")
}

```

```

package main
import "fmt"
func main() {
outer:
    for i:=0; i<5; i++){
        for j:=0; j<10; j++ {
            if (j == 5) {
                break outer// 跳出到指定标签
            }
            fmt.Println(j)
        }
    }
    fmt.Println("come here")
}

```

- continue语句
 - Go语言中的continue语句可以用于,立即进入下一次循环
 - 但不同的是Go语言中的continue语句可以指定标签

```

package main
import "fmt"
func main() {
    for i:=0; i<5; i++){
        if (i == 2) {
            continue
        }
        fmt.Println(i)
    }
}

```

- 利用continue 跳转到指定标签
 - 标签必须在使用之前定义
 - 标签后面只能跟循环语句, 不能插入其它语句
 - 对于单层循环和直接编写continue一样
 - 对于多层循环,相当于跳到最外层循环继续判断条件执行

```

package main
import "fmt"

```

```
func main() {
    outer:
        for i:=0; i<5; i++){
            fmt.Println("i =", i) // 0 1 2 3 4
            for j:=0; j<10; j++ {
                if (j == 5) {
                    continue outer // 跳出到指定标签
                }
            }
        }
        fmt.Println("come here")
    }
}
```

- goto语句
 - Go语言中的goto和C语言中用法一样, 用于在同一个函数中瞎跳

```
package main
import "fmt"
func main() {
    num := 1
    outer:
        if(num <= 10){
            fmt.Println(num)
            num++
            goto outer // 死循环
        }
        fmt.Println("come here")
    }
}
```

```
package main
import "fmt"
func main() {
    num := 1
    if(num <= 10){
        fmt.Println(num)
        num++
        goto outer // 死循环
    }
    outer:
        fmt.Println("come here")
    }
}
```

- Go语言中的return语句和C语言一模一样,都是用于结束函数,将结果返回给调用者

函数

- Go语言和C语言一样也有函数的概念, Go语言中函数除了定义格式和不用声明以外,其它方面几乎和C语言一模一样
- 格式:

```
func 函数名称(形参列表)(返回值列表){
    函数体;
}
```

- 无参数无返回值函数

```
func say() {  
    fmt.Println("Hello world!!!")  
}
```

- 有参数无返回值函数

```
func say(name string) {  
    fmt.Println("Hello ", name)  
}
```

- 无参数有返回值函数

```
func sum() int { // 只有一个返回值时,返回值列表的()可以省略  
    return 1 + 1  
}
```

- 有参数有返回值函数

```
func sum(a int, b int) int {  
    return a + b  
}
```

和C语言函数差异

- 和C语言不同的是,Go语言中可以给函数的返回值指定名称

```
// 给返回值指定了一个名称叫做res, return时会自动将函数体内部res作为返回值  
// 其实本质就是提前定义了一个局部变量res, 在函数体中使用的res就是这个局部变量,返回的也是这个局部变量  
func sum() (res int) {  
    res = 1 + 1  
    return  
}
```

- 和C语言不同的是,Go语言中的函数允许有多个返回值函数

```
func calculate(a int, b int) (sum int, sub int) {  
    sum = a + b  
    sub = a - b  
    return  
}
```

- 相邻同类型形参OR返回值类型可以合并, 可以将数据类型写到最后一个同类型形参OR返回值后面

```
// a, b都是int类型, 所以只需要在b后面添加int即可
func calculate(a, b int) (sum, sub int) {
    sum = a + b
    sub = a - b
    return
}
```

- 和C语言不同的是Go语言中的函数不需要先声明在使用

```
package main
import "fmt"
func main() {
    say();
}
func say() { // 在后面定义也可以在前面使用
    fmt.Println("Hello world!!!")
}
```

值传递和引用传递

- Go语言中 值类型 有: int系列、float系列、bool、string、数组、结构体
 - 值类型通常在栈中分配存储空间
 - 值类型作为函数参数传递, 是拷贝传递
 - 在函数体内修改值类型参数, 不会影响到函数外的值

```
package main
import "fmt"
func main() {
    num := 10
    change(num)
    fmt.Println(num) // 10
}
func change(num int) {
    num = 998
}
```

```
package main
import "fmt"
func main() {
    arr := [3]int{1, 3, 5}
    change(arr)
    fmt.Println(arr) // 1, 3, 5
}
func change(arr [3]int) {
    arr[1] = 8
}
```

```
package main
import "fmt"
type Person struct {
    name string
    age int
}
```

```

}
func main() {
    p := Person{"lnj", 33}
    change(p)
    fmt.Println(p.name) // lnj
}
func change(p Person) {
    p.name = "zs"
}

```

- Go语言中引用类型有: 指针、slice、map、channel
 - 引用类型通常在堆中分配存储空间
 - 引用类型作为函数参数传递,是引用传递
 - 在函数体内修改引用类型参数,会影响到函数外的值

```

package main
import "fmt"
func main() {
    num := 10
    change(&num)
    fmt.Println(num) // 998
}
func change(num *int) {
    *num = 998
}

```

```

package main
import "fmt"
func main() {
    arr := []int{1, 3, 5}
    change(arr)
    fmt.Println(arr) // 1, 8, 5
}
func change(arr []int) {
    arr[1] = 8
}

```

```

package main
import "fmt"
func main() {
    mp := map[string]string{"name":"lnj", "age":"33"}
    change(mp)
    fmt.Println(mp["name"]) // zs
}
func change(mp map[string]string) {
    mp["name"] = "zs"
}

```

匿名函数

- 匿名函数也是函数的一种, 它的格式和普通函数一模一样,只不过没有名字而已
 - 普通函数的函数名称是固定的, 匿名函数的函数名称是系统随机的

- 匿名函数可以定义在函数外(全局匿名函数),也可以定义在函数内(局部匿名函数), Go语言中的普通函数不能嵌套定义,但是可以通过匿名函数来实现函数的嵌套定义

- 全局匿名函数

```
package main
import "fmt"
// 方式一
var a = func() {
    fmt.Println("hello world1")
}
// 方式二
var (
    b = func() {
        fmt.Println("hello world2")
    }
)
func main() {
    a()
    b()
}
```

- 一般情况下我们很少使用全局匿名函数,大多数情况都是使用局部匿名函数,匿名函数可以直接调用、保存到变量、作为参数或者返回值
- 直接调用

```
package main
import "fmt"
func main() {
    func(s string){
        fmt.Println(s)
    }("hello lnj")
}
```

- 保存到变量

```
package main
import "fmt"
func main() {
    a := func(s string) {
        fmt.Println(s)
    }
    a("hello lnj")
}
```

- 作为参数

```

package main
import "fmt"
func main() {
    test(func(s string) {
        fmt.Println(s)
    })
}
func test(f func(s string)) {
    f("hello lnj")
}

```

- 作为返回值

```

package main
import "fmt"
func main() {
    res := test()
    res(10, 20)
}
func test() func(int, int) {
    return func(a int, b int) {
        fmt.Println(a + b)
    }
}

```

- 匿名函数应用场景
 - 当某个函数只需要被调用一次时, 可以使用匿名函数
 - 需要执行一些不确定的操作时, 可以使用匿名函数

```

package main
import "fmt"
func main() {
    // 项目经理的一天
    work(func() {
        fmt.Println("组织部门开会")
        fmt.Println("给部门员工分配今天工作任务")
        fmt.Println("检查部门员工昨天提交的代码")
        fmt.Println("... ..")
    })
    // 程序员的一天
    work(func() {
        fmt.Println("参加部门会议")
        fmt.Println("修改测试提交的BUG")
        fmt.Println("完成老大今天安排的任务")
        fmt.Println("... ..")
    })
}
// 假设我们需要编写一个函数, 用于描述一个人每天上班都需要干嘛
// 职场中的人每天上班前, 上班后要做事几乎都是相同的, 但是每天上班过程中要做的事确实不确定的
// 所以此时我们可以使用匿名函数来解决, 让上班的人自己觉得自己每天上班需要干什么
func work(custom func()) {
    // 上班前
    fmt.Println("起床")
    fmt.Println("刷牙")
    fmt.Println("洗脸")
}

```

```

fmt.Println("出门")
fmt.Println("上班打卡")
fmt.Println("开电脑")

// 上班中
custom()

// 上班后
fmt.Println("关电脑")
fmt.Println("下班打卡")
fmt.Println("出门")
fmt.Println("到家")
fmt.Println("吃饭")
fmt.Println("睡觉")
}

```

- 为了提升代码的可读性,我们还可以将这个大函数拆解为独立的匿名函数

```

func work(custom func()) {
    // 这种写法的好处是代码层次清晰,并且如果有一些变量
    // 只需要在上班前或上班后使用,还可以将这些变量隔离,不对外界造成污染
    // 上班前
    func(){
        fmt.Println("起床")
        fmt.Println("刷牙")
        fmt.Println("洗脸")
        fmt.Println("出门")
        fmt.Println("上班打卡")
        fmt.Println("开电脑")
    }()

    // 上班中
    custom()

    // 上班后
    func(){
        fmt.Println("关电脑")
        fmt.Println("下班打卡")
        fmt.Println("出门")
        fmt.Println("到家")
        fmt.Println("吃饭")
        fmt.Println("睡觉")
    }()
}

```

```

func work(custom func()) {
    // 前提条件是这个函数只在work函数中使用,两者有较强的关联性,否则建议定义为普通函数
    pre := func(){
        fmt.Println("起床")
        fmt.Println("刷牙")
        fmt.Println("洗脸")
        fmt.Println("出门")
        fmt.Println("上班打卡")
        fmt.Println("开电脑")
    }
}

```

```

}
latter := func(){
    fmt.Println("关电脑")
    fmt.Println("下班打卡")
    fmt.Println("出门")
    fmt.Println("到家")
    fmt.Println("吃饭")
    fmt.Println("睡觉")
}

// 上班前
pre()
// 上班中
custom()
// 上班后
latter()
}

```

闭包

- 闭包是一个特殊的匿名函数, 它是匿名函数和相关引用环境组成的一个整体
 - 也就是说只要匿名函数中用到了外界变量, 那么这个匿名函数就是一个闭包

```

package main
import "fmt"
func main() {
    num := 10
    a := func() {
        num++ // 在闭包中用到了main函数中的num, 所以这个匿名函数就是一个闭包
        fmt.Println(num) // 11
    }
    a()
}

```

- 闭包中使用的变量和外界变量是同一个变量, 所以可以在闭包中修改外界变量

```

package main
import "fmt"
func main() {
    num := 10
    a := func() {
        num = 6 // 在闭包中用到了main函数中的num, 所以这个匿名函数就是一个闭包
        fmt.Println(num) // 6
    }
    fmt.Println("执行闭包前", num) // 10
    a()
    fmt.Println("执行闭包后", num) // 6
}

```

- 只要闭包还在使用外界变量, 那么外界变量就会一直存在

```

package main
import "fmt"
func main() {

```

```

res := addUpper() // 执行addUpper函数,得到一个闭包
fmt.Println(res()) // 2
fmt.Println(res()) // 3
fmt.Println(res()) // 4
fmt.Println(res()) // 5
}
func addUpper() func() int {
    x := 1
    return func() int {
        x++ // 匿名函数中用到了addUpper中的x,所以这是一个闭包
        return x
    }
}

```

延迟调用

- Go语言中没有提供其它面向对象语言的析构函数,但是Go语言提供了defer语句用于实现其它面向对象语言析构函数的功能
- defer语句常用于 释放资源、解除锁定 以及 错误处理 等
 - 例如C语言中我们申请了一块内存空间,那么不使用时我们就必须释放这块存储空间
 - 例如C语言中我们打开了一个文件,那么我们不使用时就要关闭这个文件
 - 例如C语言中我们打开了一个数据库,那么我们不使用时就要关闭这个数据库
 - 这一类的操作在Go语言中都可以通过defer语句来完成
- 无论你在什么地方注册defer语句,它都会在所属函数执行完毕之后才会执行,并且如果注册了多个defer语句,那么它们会按照 后进先出 的原则执行
 - 正是因为defer语句的这种特性,所以在Go语言中关闭资源不用像C语言那样用完了再关闭,我们完全可以打开的同时就关闭,因为无论如何defer语句都会在所属函数执行完毕之后才会执行

```

package main
import "fmt"
func main() {
    defer fmt.Println("我是第一个被注册的") // 3
    fmt.Println("main函数中调用的Println") // 1
    defer fmt.Println("我是第二个被注册的") // 2
}

```

init函数

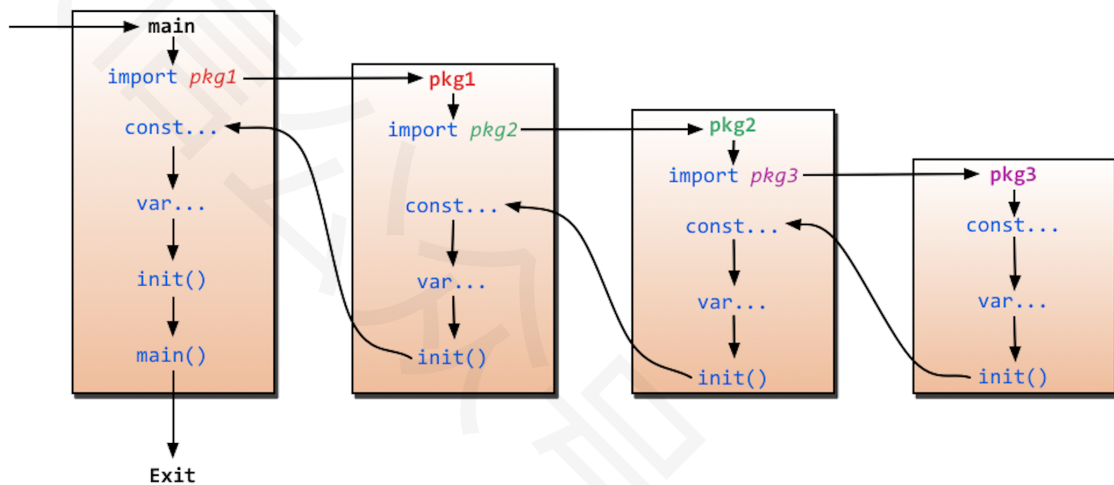
- golang里面有两个保留的函数:
 - init函数 (能够应用于所有的package)
 - main函数 (只能应用于package main)
 - 这两个函数在定义时不能有任何的参数和返回值
- go程序会自动调用init()和main(),所以你 不能 在任何地方调用这两个函数
- package main必须包含一个main函数,但是每个package中的init函数都是可选的
- 一个package里面可以写任意多个init函数,但这无论是对于可读性还是以后的可维护性来说,我们都强烈建议用户在一个package中每个文件只写一个init函数
- 单个包中代码执行顺序如下
 - `main包-->常量-->全局变量-->init函数-->main函数-->Exit`

```

package main
import "fmt"
const constValue = 998 // 1
var globalVarValue int = abc() // 2
func init() { // 3
    fmt.Println("执行main包中main.go中init函数")
}
func main() { // 4
    fmt.Println("执行main包中main.go中main函数")
}
func abc() int {
    fmt.Println("执行main包中全局变量初始化")
    return 998
}

```

- 多个包之间代码执行顺序如下



- init函数的作用
 - init函数用于处理当前文件的初始化操作, 在使用某个文件时的一些准备工作应该放到这里

数组

- 和C语言一样,Go语言中也有数组的概念, Go语言中的数组也是用于保存一组 相同类型 的数据
- 和C语言一样,Go语言中的数组也分为 一维数组 和 多维数组

一维数组

- 格式: `var arr [元素个数]数据类型`
 - 和C语言中数组不同, Go语言中数组定义之后就 有默认的初始值
 - 默认初始值就是保存数据类型的默认值(零值)

```

package main
import "fmt"
func main() {
    // 1.定义一个数组
    var arr [3]int
    // 2.打印数组
    fmt.Println(arr) //[0 0 0]

    // 1.定义一个数组
    var arr [3]bool
    // 2.打印数组
    fmt.Println(arr) //[false false false]
}

```

- 和C语言一样,Go语言中的数组也提供了好几种初始化方式

```

package main
import "fmt"
func main() {
    // 1.定义的同时完全初始化
    var arr1 [3]int = [3]int{1, 3, 5}
    // 2.打印数组
    fmt.Println(arr1) // [1 3 5]

    // 1.定义的同时部分初始化
    var arr4 [3]int = [3]int{8, 9}
    // 2.打印数组
    fmt.Println(arr4) // [8 9 0]

    // 1.定义的同时指定元素初始化
    var arr5 [3]int = [3]int{0:8, 2:9}
    // 2.打印数组
    fmt.Println(arr5) // [8 0 9]

    // 1.先定义再逐个初始化
    var arr3 [3]int
    arr3[0] = 1
    arr3[1] = 2
    arr3[2] = 3
    // 2.打印数组
    fmt.Println(arr3) // [1 2 3]
}

```

- 和C语言中的数组不同,Go语言中数组除了可以定义的同时初始化以外,还可以先定义再一次性初始化

```

package main
import "fmt"
func main() {
    // 1.先定义再一次性初始化
    var arr2 [3]int
    arr2 = [3]int{2, 4, 6}
    // 2.打印数组
    fmt.Println(arr2) // [2 4 6]
}

```

- 和C语言一样,Go语言中如果定义数组的同时初始化,那么元素个数可以省略,但是必须使用 ... 来替代
 - ...会根据初始化元素个数自动确定数组长度

```
package main
import "fmt"
func main() {
    // 1. 定义的同时完全初始化
    var arr1 = [...]int{1, 3, 5}
    // 2. 打印数组
    fmt.Println(arr1) // [1 3 5]

    // 1. 定义的同时指定元素初始化
    var arr2 = [...]int{6:5}
    // 2. 打印数组
    fmt.Println(arr2) // [0 0 0 0 0 5]
}
```

- Go语言中数组的访问和使用方式和C语言一样都是通过 数组名称[索引] 的方式

```
package main
import "fmt"
func main() {
    arr := [...]int{1, 3, 5}
    // 使用数组, 往数组中存放数据
    arr[1] = 666
    // 访问数组, 从数组中获取数据
    fmt.Println(arr[0])
    fmt.Println(arr[1])
    fmt.Println(arr[2])
}
```

- 遍历数组
 - Go语言中提供了两种遍历数组的方式, 一种是通过传统for循环遍历, 一种是通过for...range循环遍历

```
package main
import "fmt"
func main() {
    arr := [...]int{1, 3, 5}
    // 传统for循环遍历
    for i:=0; i<len(arr); i++){
        fmt.Println(i, arr[i])
    }
    // for...range循环遍历
    for i, v := range arr{
        fmt.Println(i, v)
    }
}
```

- 数组注意点
 - Go语言中 数组长度 也是 数据类型的一部分

```
package main
import "fmt"
func main() {
    var arr1 [3]int
    var arr2 [3]int
    //var arr3 [2]int
    fmt.Println(arr1 == arr2) // true
    //fmt.Println(arr1 == arr3) // 编译报错，不是相同类型不能比较
}
```

- 如果元素类型支持==、!=操作时,那么数组也支持此操作

```
package main
import "fmt"
func main() {
    var arr1 [3]int = [3]int{1, 3, 5}
    var arr2 [3]int = [3]int{1, 3, 5}
    var arr3 [3]int = [3]int{2, 4, 6}
    // 首先会判断`数据类型`是否相同,如果相同会依次取出数组中`对应索引的元素`进行比较,
    // 如果所有元素都相同返回true,否则返回false
    fmt.Println(arr1 == arr2) // true
    fmt.Println(arr1 == arr3) // false
}
```

- Go语言中的数组是值类型,赋值和传参时会复制整个数组

```
package main
import "fmt"
func main() {
    var arr1 [3]int = [3]int{1, 3, 5}
    var arr2 [3]int = arr1
    arr2[0] = 666
    fmt.Println(arr1) // [1 3 5]
    fmt.Println(arr2) // [666 3 5]
}
```

二维数组

- 用法和C语言数组一样,只是创建的格式不同
- 格式: `[行数][列数]类型`

```
package main
import "fmt"
func main() {
    // 创建一个两行三列数组
    arr := [2][3]int{
        {1, 2, 3},
        {4, 5, 6}, //注意: 数组换行需要以逗号结尾
    }
    fmt.Println(arr) // [[1 2 3] [4 5 6]]
}
```

- 创建多维数组时只允许第一维度使用...
- 格式: `[...][列数]类型`

```

package main
import "fmt"
func main() {
    // 创建一个两行三列数组
    arr := [...]int{
        {1, 2, 3},
        {4, 5, 6},
    }
    fmt.Println(arr) // [[1 2 3] [4 5 6]]
}

```

切片

- 无论是C语言中的数组还是Go语言中的数组,数组的长度一旦确定就不能改变,但在实际开发中我们可能事先不能确定数组的长度,为了解决这类问题Go语言中推出了一种新的数据类型 `切片`
- 切片可以简单的理解为长度可以变化的数组,但是Go语言中的切片本质上是一个结构体
 - 切片源码

```

type slice struct{
    array unsafe.Pointer // 指向底层数组指针
    len int // 切片长度(保存了多少个元素)
    cap int // 切片容量(可以保存多少个元素)
}

```

- **切片创建的三种方式**
- 方式一: 通过数组创建切片 `array[startIndex:endIndex]`

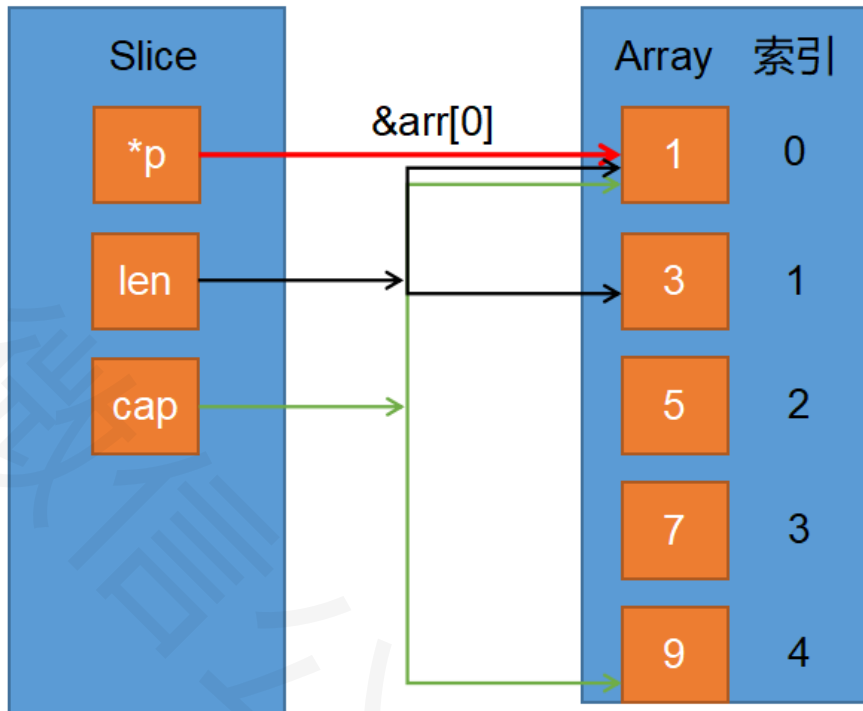
```

package main
import "fmt"
func main() {
    var arr = [5]int{1, 3, 5, 7, 9}
    // 从数组0下标开始取,一直取到2下标前面一个索引
    var sce = arr[0:2]
    fmt.Println(sce) // [1 3]
    // 切片len = 结束位置 - 开始位置
    fmt.Println(len(sce)) // 2 - 0 = 2
    fmt.Println(cap(sce)) // 5 - 0 = 5
    // 数组地址就是数组首元素的地址
    fmt.Printf("%p\n", &arr) // 0xc04200a330
    fmt.Printf("%p\n", &arr[0]) // 0xc04200a330
    // 切片地址就是数组中指定的开始元素的地址
    // arr[0:2]开始地址为0, 所以就是arr[0]的地址
    fmt.Printf("%p\n", sce) // 0xc04200a330
}

```

var sce = arr[0:2]

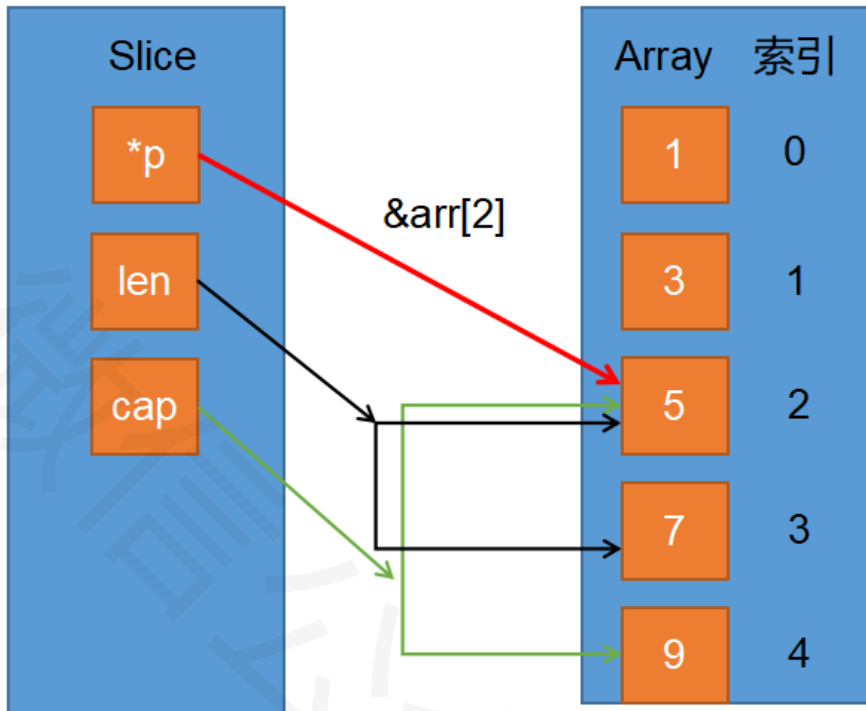
var arr = [5]int{1, 3, 5, 7, 9}



```
package main
import "fmt"
func main() {
var arr = [5]int{1, 3, 5, 7, 9}
// 根据数组的索引片段创建切片
var sce = arr[2:4]
fmt.Println(sce) // [5 7]
fmt.Println(len(sce)) // 4 - 2 = 2
fmt.Println(cap(sce)) // 5 - 2 = 3
fmt.Printf("%p\n", &arr[2]) // 0xc042076070
fmt.Printf("%p\n", sce) // 0xc042076070
}
```

var sce = arr[2:4]

var arr = [5]int{1, 3, 5, 7, 9}



- 指定起始位置时有三种方式可以指定
 - 开始位置和结束位置都指定
 - 只指定开始位置或结束位置
 - 开始位置和结束位置都不指定

```
package main
import "fmt"
func main() {
    var arr = [5]int{1, 3, 5, 7, 9}
    // 同时指定开始位置和结束位置
    var sce1 = arr[0:2]
    fmt.Println(sce1) // [1 3]

    // 只指定结束位置
    var sce3 = arr[:2]
    fmt.Println(sce3) // [1 3]

    // 只指定开始位置
    var sce2 = arr[0:]
    fmt.Println(sce2) // [1 3 5 7 9]

    // 都不指定
    var sce4 = arr[:]
    fmt.Println(sce4) // [1 3 5 7 9]
}
```

- 方式二: 通过make函数创建 `make(类型, 长度, 容量)`
 - 内部会先创建一个数组, 然后让切片指向数组
 - 如果没有指定容量, 那么容量和长度一样

```

package main
import "fmt"
func main() {
    // 第一个参数: 指定切片数据类型
    // 第二个参数: 指定切片的长度
    // 第三个参数: 指定切片的容量
    var sce = make([]int, 3, 5)
    fmt.Println(sce) // [0 0 0]
    fmt.Println(len(sce)) // 3
    fmt.Println(cap(sce)) // 5
    /*
    内部实现原理
    var arr = [5]int{0, 0, 0}
    var sce = arr[0:3]
    */
}

```

- 方式三:通过Go提供的语法糖快速创建
 - 和创建数组一模一样,但是不能指定长度
 - 通过该方式创建时,切片的长度和容量相等

```

package main
import "fmt"
func main() {
    var sce = []int{1, 3, 5}
    fmt.Println(sce) // [1 3 5]
    fmt.Println(len(sce)) // 3
    fmt.Println(cap(sce)) // 3
}

```

• 切片的使用

- 切片的基本使用方式和数组一样,可以通过 切片名称[索引] 方式操作切片

```

package main
import "fmt"
func main() {
    var sce = []int{1, 3, 5}
    // 使用切片, 往切片中存放数据
    sce[1] = 666
    // 访问切片, 从切片中获取数据
    fmt.Println(sce) // [1 666 5]
}

```

- 和数组一样,如果通过 切片名称[索引] 方式操作切片,不能越界

```

package main
import "fmt"
func main() {
    var sce = []int{1, 3, 5}
    // 编译报错, 越界
    sce[3] = 666
}

```

- 如果希望切片自动扩容,那么添加数据时必须使用append方法
 - append函数会在切片末尾添加一个元素,并返回一个追加数据之后的切片
 - 利用append函数追加数据时,如果追加之后没有超出切片的容量,那么返回原来的切片,如果追加之后超出了切片的容量,那么返回一个新的切片
 - append函数每次给切片扩容都会按照原有切片容量*2的方式扩容

```
package main
import "fmt"
func main() {
var sce = []int{1, 3, 5}
fmt.Println("追加数据前:", sce) // [1 3 5]
fmt.Println("追加数据前:", len(sce)) // 3
fmt.Println("追加数据前:", cap(sce)) // 3
fmt.Printf("追加数据前: %p\n", sce) // 0xc0420600a0
// 第一个参数: 需要把数据追加到哪个切片中
// 第二个参数: 需要追加的数据, 可以是一个或多个
sce = append(sce, 666)
fmt.Println("追加数据后:", sce) // [1 3 5 666]
fmt.Println("追加数据后:", len(sce)) // 4
fmt.Println("追加数据后:", cap(sce)) // 6
fmt.Printf("追加数据后: %p\n", sce) // 0xc042076b60
}
```

- 除了append函数外,Go语言还提供了一个copy函数,用于两个切片之间数据的快速拷贝
 - 格式: copy(目标切片, 源切片), 会将源切片中数据拷贝到目标切片中

```
package main
import "fmt"
func main() {
var sce1 = []int{1, 3, 5}
var sce2 = make([]int, 5)
fmt.Printf("赋值前: %p\n", sce1) // 0xc0420600a0
fmt.Printf("赋值前: %p\n", sce2) // 0xc042076060
// 将sce2的指向修改为sce1, 此时sce1和sce2底层指向同一个数组
sce2 = sce1
fmt.Printf("赋值后: %p\n", sce1) // 0xc0420600a0
fmt.Printf("赋值后: %p\n", sce2) // 0xc0420600a0
// copy(sce2, sce1)
fmt.Println(sce1) // [1 3 5]
fmt.Println(sce2) // [1 3 5]
sce2[1] = 666
fmt.Println(sce1) // [1 666 5]
fmt.Println(sce2) // [1 666 5]
}
```

```
package main
import "fmt"
func main() {
var sce1 = []int{1, 3, 5}
var sce2 = make([]int, 5)
fmt.Printf("赋值前: %p\n", sce1) // 0xc0420600a0
fmt.Printf("赋值前: %p\n", sce2) // 0xc042076060
// 将sce1中的数据拷贝到sce2中, 此时sce1和sce2底层指向不同数组
copy(sce2, sce1)
```

```

fmt.Printf("赋值后:%p\n", sce1) // 0xc0420600a0
fmt.Printf("赋值后:%p\n", sce2) // 0xc042076060
//copy(sce2, sce1)
fmt.Println(sce1) // [1 3 5]
fmt.Println(sce2) // [1 3 5 0 0]
sce2[1] = 666
fmt.Println(sce1) // [1 3 5]
fmt.Println(sce2) // [1 666 5 0 0]
}

```

- copy函数在拷贝数据时永远以小容量为准

```

package main
import "fmt"
func main() {
// 容量为3
var sce1 = []int{1, 3, 5}
// 容量为5
var sce2 = make([]int, 5)
fmt.Println("拷贝前:", sce2) // [0 0 0 0 0]
// sce2容量足够, 会将sce1所有内容拷贝到sce2
copy(sce2, sce1)
fmt.Println("拷贝后:", sce2) // [1 3 5 0 0]
}

```

```

package main
import "fmt"
func main() {
// 容量为3
var sce1 = []int{1, 3, 5}
// 容量为2
var sce2 = make([]int, 2)
fmt.Println("拷贝前:", sce2) // [0 0]
// sce2容量不够, 会将sce1前2个元素拷贝到sce2中
copy(sce2, sce1)
fmt.Println("拷贝后:", sce2) // [1 3]
}

```

• 切片的注意点

- 可以通过切片再次生成新的切片, 两个切片底层指向同一数组

```

package main
import "fmt"
func main() {
arr := [5]int{1, 3, 5, 7, 9}
sce1 := arr[0:4]
sce2 := sce1[0:3]
fmt.Println(sce1) // [1 3 5 7]
fmt.Println(sce2) // [1 3 5]
// 由于底层指向同一数组, 所以修改sce2会影响sce1
sce2[1] = 666
fmt.Println(sce1) // [1 666 5 7]
fmt.Println(sce2) // [1 666 5]
}

```

- 和数组不同, 切片只支持判断是否为nil, 不支持==、!=判断

```
package main
import "fmt"
func main() {
    var arr1 [3]int = [3]int{1, 3, 5}
    var arr2 [3]int = [3]int{1, 3, 5}
    var arr3 [3]int = [3]int{2, 4, 6}
    // 首先会判断`数据类型`是否相同, 如果相同会依次取出数组中`对应索引的元素`进行比较,
    // 如果所有元素都相同返回true, 否则返回false
    fmt.Println(arr1 == arr2) // true
    fmt.Println(arr1 == arr3) // false

    sce1 := []int{1, 3, 5}
    sce2 := []int{1, 3, 5}
    //fmt.Println(sce1 == sce2) // 编译报错
    fmt.Println(sce1 != nil) // true
    fmt.Println(sce2 == nil) // false
}
```

- 只声明当没有被创建的切片是不能使用的

```
package main
import "fmt"
func main() {
    // 数组声明后就可以直接使用, 声明时就会开辟存储空间
    var arr [3]int
    arr[0] = 2
    arr[1] = 4
    arr[2] = 6
    fmt.Println(arr) // [2 4 6]

    // 切片声明后不能直接使用, 只有通过make或语法糖创建之后才会开辟空间, 才能使用
    var sce []int
    sce[0] = 2 // 编译报错
    sce[1] = 4
    sce[2] = 6
    fmt.Println(sce)
}
```

- 字符串的底层是[]byte数组, 所以字符也支持切片相关操作

```
package main
import "fmt"
func main() {
    str := "abcdefg"
    // 通过字符串生成切片
    sce1 := str[3:]
    fmt.Println(sce1) // defg

    sce2 := make([]byte, 10)
    // 第二个参数只能是slice或者是数组
    // 将字符串拷贝到切片中
    copy(sce2, str)
    fmt.Println(sce2) // [97 98 99 100 101 102 103 0 0 0]
}
```

map(字典、映射)

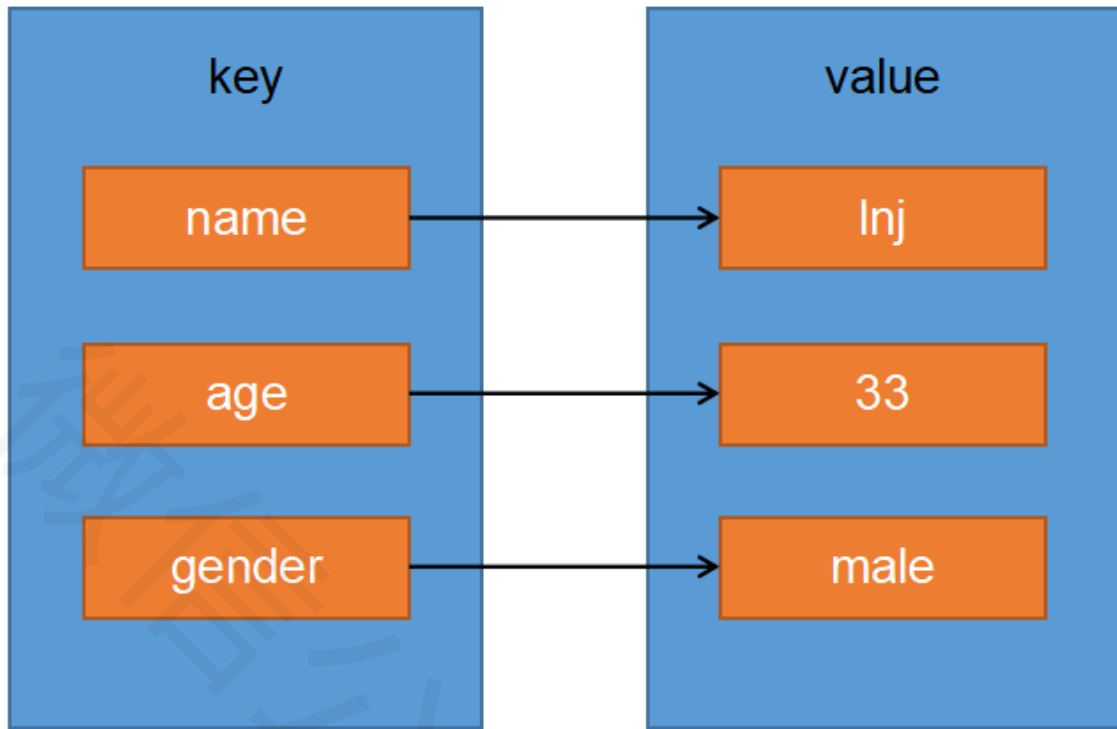
- map翻译过来就是字典或者映射, 可以把map看做是切片的升级版
 - 切片是用来存储一组相同类型的数据的, map也是用来存储一组相同类型的数据的
 - 在切片中我们可以通过索引获取对应的元素, 在map中我们可以通过key获取对应的元素
 - 切片的索引是系统自动生成的,从0开始递增. map中的key需要我们自己指定
 - 只要是可以做==、!=判断的数据类型都可以作为key(数值类型、字符串、数组、指针、结构体、接口)
 - map的key的数据类型不能是:slice、map、function
 - map和切片一样容量都不是固定的, 当容量不足时底层会自动扩容
- map格式: `var dic map[key数据类型]value数据类型`

```
package main
import "fmt"
func main() {
    var dic map[int]int = map[int]int{0:1, 1:3, 2:5}
    fmt.Println(dic) // map[0:1 1:3 2:5]

    // 获取map中某个key对应的值
    fmt.Println(dic[0]) // 1

    // 修改map中某个key对应的值
    dic[1] = 666
    fmt.Println(dic) // map[0:1 1:666 2:5]
}
```

```
package main
import "fmt"
func main() {
    var dict map[string]string = map[string]string{"name":"lnj", "age":"33",
    "gender":"male"}
    fmt.Println(dict) // map[name:lnj age:33 gender:male]
}
```



- 创建map的三种方式

- 方式一: 通过Go提供的语法糖快速创建

```
package main
import "fmt"
func main() {
    dict := map[string]string{"name":"lnj", "age":"33", "gender":"male"}
    fmt.Println(dict)// map[name:lnj age:33 gender:male]
}
```

- 方式二:通过make函数创建 make(类型, 容量)

```
package main
import "fmt"
func main() {
    var dict = make(map[string]string, 3)
    dict["name"] = "lnj"
    dict["age"] = "33"
    dict["gender"] = "male"
    fmt.Println(dict)// map[age:33 gender:male name:lnj]
}
```

- 方式二:通过make函数创建 make(类型)

```
package main
import "fmt"
func main() {
    var dict = make(map[string]string)
    dict["name"] = "lnj"
    dict["age"] = "33"
    dict["gender"] = "male"
    fmt.Println(dict) // map[age:33 gender:male name:lnj]
}
```

- 和切片一样,没有被创建的map是不能使用的

```
package main
import "fmt"
func main() {
    // map声明后不能直接使用，只有通过make或语法糖创建之后才会开辟空间,才能使用
    var dict map[string]string
    dict["name"] = "lnj" // 编译报错
    dict["age"] = "33"
    dict["gender"] = "male"
    fmt.Println(dict)
}
```

• map的增删改查

- 增加: 当map中没有指定的key时就会自动增加

```
package main
import "fmt"
func main() {
    var dict = make(map[string]string)
    fmt.Println("增加前:", dict) // map[]
    dict["name"] = "lnj"
    fmt.Println("增加后:", dict) // map[name:lnj]
}
```

- 修改: 当map中有指定的key时就会自动修改

```
package main
import "fmt"
func main() {
    var dict = map[string]string{"name":"lnj", "age":"33", "gender":"male"}
    fmt.Println("修改前:", dict) // map[name:lnj age:33 gender:male]
    dict["name"] = "zs"
    fmt.Println("修改后:", dict) // map[age:33 gender:male name:zs]
}
```

- 删除: 可以通过Go语言内置delete函数删除指定元素

```

package main
import "fmt"
func main() {
    var dict = map[string]string{"name":"lnj", "age":"33", "gender":"male"}
    fmt.Println("删除前:", dict) // map[name:lnj age:33 gender:male]
    // 第一个参数: 被操作的字典
    // 第二个参数: 需要删除元素对应的key
    delete(dict, "name")
    fmt.Println("删除后:", dict) // map[age:33 gender:male]
}

```

- 查询: 通过ok-idiom模式判断指定键值对是否存储

```

package main
import "fmt"
func main() {
    var dict = map[string]string{"name":"lnj", "age":"33", "gender":"male"}
    //value, ok := dict["age"]
    //if(ok){
    //    fmt.Println("有age这个key,值为", value)
    //}else{
    //    fmt.Println("没有age这个key,值为", value)
    //}
    if value, ok := dict["age"]; ok{
        fmt.Println("有age这个key,值为", value)
    }
}
```go
- ***map遍历***
+ 注意: map和数组以及切片不同,map中存储的数据是无序的, 所以多次打印输出的顺序可能不同
```go
var dict = map[string]string{"name":"lnj", "age":"33", "gender":"male"}
for key, value := range dict{
    fmt.Println(key, value)
}

```

结构体

- Go语言中的结构体和C语言中结构体一样, 都是用来保存一组 不同类型的数据
- Go语言中的结构体和C语言中结构体一样, 都需要先定义结构体类型再利用结构体类型定义结构体变量
- 定义结构体类型

```

type 类型名称 struct{
    属性名称 属性类型
    属性名称 属性类型
    ... ..
}

```

```

type Studentstruct {
    name string
    age int
}

```

- 创建结构体变量的两种方式

- 方式一: 先定义结构体类型, 再定义结构体变量

- 和C语言中的结构体一样, 如果结构体类型需要多次使用, 那么建议先定义类型再定义变量

```
package main
import "fmt"
func main() {
    type Student struct {
        name string
        age int
    }

    // 完全初始化
    var stu1= Student{"lnj", 33}
    fmt.Println(stu1)
    // 部分初始化
    // 部分初始化必须通过 属性名称: 方式指定要初始化的属性
    var stu2 = Student{name:"lnj"}
    fmt.Println(stu2)
}
```

- 方式二: 定义结构体类型同时定义结构体变量(匿名结构体)

- 和C语言中的结构体一样, 如果结构体类型只需要使用一次, 那么建议定义类型同时定义变量

```
package main
import "fmt"
func main() {
    // 注意: 这里不用写type和结构体类型名称
    var stu2 = struct {
        name string
        age int
    }{
        name: "lnj",
        age: 33,
    }
    fmt.Println(stu2)
}
```

- 结构体类型操作

```
package main
import "fmt"
type Student struct {
    name string
    age int
}

func main() {
    var stu= Student{"lnj", 33}
    // 获取结构体中某个属性对应的值
    fmt.Println(stu.name)

    // 修改结构体中某个属性对应的值
}
```

```

stu.name = "zs"
fmt.Println(stu)
}

```

- 和slice、map不同的是,只要定义了结构体变量就可以使用结构体变量
 - 默认情况下结构体中的所有属性都是属性对应类型的默认值

```

package main
import "fmt"
type Student struct {
    name string
    age int
}
func main() {
    var stu Student // 相当于 var stu = Student{}
    fmt.Println(stu) // { 0}
    stu.name = "lnj" // 不会报错
    stu.age = 33
    fmt.Println(stu) // {lnj 33}
}

```

- 复杂结构体成员
 - 创建时可以按照属性单独存在时初始化方式初始化

```

package main
import "fmt"
type Student struct {
    name string
    age int
}
func main() {
    type Demo struct {
        age int // 基本类型作为属性
        arr [3]int // 数组类型作为属性
        sce []int // 切片类型作为属性
        mp map[string]string // 字典类型作为属性
        stu Student // 结构体类型作为属性
    }

    var d Demo = Demo{
        33,
        [3]int{1, 3, 5},
        []int{2, 4, 6},
        map[string]string{"class": "one"},
        Student{
            "lnj",
            33,
        },
    }

    fmt.Println(d) // {33 [1 3 5] [2 4 6] map[class:one] {lnj 33}}
}

```

- slice、map类型属性默认值是nil,不能直接使用

```

package main
import "fmt"
type Student struct {
    name string
    age int
}
func main() {
type Demo struct {
    age int // 基本类型作为属性
    arr [3]int // 数组类型作为属性
    sce []int // 切片类型作为属性
    mp map[string]string // 字典类型作为属性
    stu Student // 结构体类型作为属性
}

// 定义结构体变量
var d Demo
// 可以直接使用基本类型属性
d.age = 33
// 可以直接使用数组类型属性
d.arr[0] = 666
// 不可以直接使用切片类型属性
//d.sce[0] = 888 // 编译报错
d.sce = make([]int, 5) // 先创建
d.sce[0] = 888 // 后使用
// 不可以直接使用字典类型属性
//d.mp["class"] = "one" // 编译报错
d.mp = make(map[string]string) // 先创建
d.mp["class"] = "one" // 后使用
// 可以直接使用结构体类型属性
d.stu.name = "lnj"
fmt.Println(d) // {33 [666 0 0] [888 0 0 0 0] map[class:one] {lnj 0}}
}

```

- 结构体类型转换

- 属性名、属性类型、属性个数、排列顺序都是类型组成部分
- 只有属性名、属性类型、属性个数、排列顺序都相同的结构体类型才能转换

```

package main
import "fmt"
func main() {
    type Person1 struct {
        name string
        age int
    }
    type Person2 struct {
        name string
        age int
    }
    type Person3 struct {
        age int
        name string
    }
    type Person4 struct {
        nm string
    }
}

```

```

    age int
}
type Person5 struct {
    name string
    age string
}
type Person6 struct {
    age int
    name string
    gender string
}

var p1 Person1 = Person1{"Inj", 33}
var p2 Person2
// 类型名称不一样不能直接赋值(Person1、Person2)
//p2 = p1

// 虽然类型名称不一样,但是两个类型中的`属性名称`、`属性类型`、`属性个数`、`排列顺序`都一样,所以可以强制转换
p2 = Person2(p1)
fmt.Println(p2)
// 两个结构体类型中的`属性名称`、`属性类型`、`属性个数`都一样,但是`排列顺序`不一样,所以不能强制转换
//var p3 Person3
//p3 = Person3(p1)
//fmt.Println(p3)

// 两个结构体类型中的`属性类型`、`属性个数`、`排列顺序`都一样,但是`属性名称`不一样,所以不能强制转换
//var p4 Person4
//p4 = Person4(p1)
//fmt.Println(p4)

// 两个结构体类型中的`属性名称`、`属性个数`、`排列顺序`都一样,但是`属性类型`不一样,所以不能强制转换
//var p5 Person5
//p5 = Person5(p1)
//fmt.Println(p5)

// 两个结构体类型中的`属性名称`、`属性类型`、`排列顺序`都一样,但是`属性个数`不一样,所以不能强制转换
//var p6 Person6
//p6 = Person6(p1)
//fmt.Println(p6)
}

```

- 匿名属性

- 没有指定属性名称,只有属性的类型,我们称之为匿名属性
- 任何有命名的数据类型都可以作为匿名属性(int、float、bool、string、struct等)

```

package main
import "fmt"
func main() {
type Person struct {
    int

```

```

float32
bool
string
}
// 不指定名称初始化
per1 := Person{3, 3.14, false, "lnj"}
fmt.Println(per1)

// 可以把数据类型作为名字显示初始化
per2 := Person{
    int: 3,
    float32: 3.14,
    bool: true,
    string: "lnj",
}
fmt.Println(per2)

// 可以把数据类型当做属性名称操作结构体
per2.int = 666
fmt.Println(per2.int) // 666
}

```

- Go语言中最常见的匿名属性是用 结构体类型作为匿名属性

```

package main
import "fmt"
func main() {
type Person struct {
    name string
    age int
}
type Student struct {
    Person // 匿名属性
    class string
}

stu := Student{
    Person{"lnj", 33},
    "学前一班",
}
fmt.Println(stu) // {{lnj 33} 学前一班}
}

```

- 如果结构体作为匿名属性, 想访问匿名属性的属性有两种方式

```

package main
import "fmt"
func main() {
type Person struct {
    name string
    age int
}
type Student struct {
    Person // 匿名属性
    class string
}
}

```

```

stu := Student{
    Person{"lnj", 33},
    "学前一班",
}
fmt.Println(stu) // {{lnj 33} 学前一班}

// 方式一：先找到匿名属性,再访问匿名属性中的属性
stu.Person.name = "zs"
fmt.Println(stu) // {{zs 33} 学前一班}
// 方式二：直接访问匿名属性中的属性
// 系统会先查找当前结构体有没有名称叫做name的属性
// 如果没有会继续查找匿名属性中有没有名称叫做name的属性
stu.name = "ww"
fmt.Println(stu) // {{ww 33} 学前一班}
}

```

- 注意点: 如果多个匿名属性的属性名称相同,那么不能通过方式二操作,只能通过方式一

```

package main
import "fmt"
func main() {
    type Person struct {
        name string
        age int
    }
    type Class struct {
        name string
        time string
    }
    type Student struct {
        Person // 匿名属性
        Class  // 匿名属性
    }
    stu := Student{
        Person{"lnj", 33},
        Class{"学前一班", "2020-12-12"},
    }
    fmt.Println(stu) // {{lnj 33} {学前一班 2020-12-12}}
    // 编译报错, 系统搞不清楚要找哪个name
    //stu.name = "zs"

    stu.Person.name = "zs"
    stu.Class.name = "小学一年级"
    fmt.Println(stu) // {{zs 33} {小学一年级 2020-12-12}}
}

```

- 注意点: 只有匿名结构体才支持向上查找

```

package main
import "fmt"
func main() {
    type Person struct {
        name string
    }
    type Student struct {

```

```

    per Person
    age int
}

var stu Student = Student{Person{"Inj"}, 18}
//fmt.Println(stu.name) // 报错
fmt.Println(stu.per.name) // 必须通过属性进一步查找
fmt.Println(stu.age)
}

```

- 注意点: 如果匿名属性是一个结构体类型, 那么这个结构体类型不能是自己

```

package main
import "fmt"
func main() {
    type Person struct {
        Person // 错误
        name string
    }
    type Student struct {
        *Student // 正确, 链表
        age int
    }
}

```

普通指针

- 和C语言一样, 允许用一个变量来存放其它变量的地址, 这种专门用于存储其它变量地址的变量, 我们称之为指针变量
- 和C语言一样, Go语言中的指针无论是什么类型占用内存都一样(32位4个字节, 64位8个字节)

```

package main

import (
    "fmt"
    "unsafe"
)

func main() {

    var p1 *int;
    var p2 *float64;
    var p3 *bool;
    fmt.Println(unsafe.Sizeof(p1)) // 8
    fmt.Println(unsafe.Sizeof(p2)) // 8
    fmt.Println(unsafe.Sizeof(p3)) // 8
}

```

- 和C语言一样, 只要一个指针变量保存了另一个变量对应的内存地址, 那么就可以通过*来访问指针变量指向的存储空间

```

package main

import (
    "fmt"

```

```

)

func main() {

    // 1. 定义一个普通变量
    var num int = 666
    // 2. 定义一个指针变量
    var p *int = &num
    fmt.Printf("%p\n", &num) // 0xc042064080
    fmt.Printf("%p\n", p) // 0xc042064080
    fmt.Printf("%T\n", p) // *int
    // 3. 通过指针变量操作指向的存储空间
    *p = 888
    // 4. 指针变量操作的就是指向变量的存储空间
    fmt.Println(num) // 888
    fmt.Println(*p) // 888
}

```

指向数组指针

- 在C语言中, 数组名, &数组名, &数组首元素保存的都是同一个地址

```

#include <stdio.h>

int main(){
    int arr[3] = {1, 3, 5};
    printf("%p\n", arr); // 0060FEA4
    printf("%p\n", &arr); // 0060FEA4
    printf("%p\n", &arr[0]); // 0060FEA4
}

```

- 在Go语言中通过数组名无法直接获取数组的内存地址

```

package main
import "fmt"

func main() {
    var arr [3]int = [3]int{1, 3, 5}
    fmt.Printf("%p\n", arr) // 乱七八糟东西
    fmt.Printf("%p\n", &arr) // 0xc0420620a0
    fmt.Printf("%p\n", &arr[0]) // 0xc0420620a0
}

```

- 在C语言中, 无论我们将数组名, &数组名, &数组首元素赋值给指针变量, 都代表指针变量指向了这个数组

```

#include <stdio.h>

int main(){
    int arr[3] = {1, 3, 5};
    int *p1 = arr;
    p1[1] = 6;
    printf("%d\n", arr[1]);
}

```

```

int *p2 = &arr;
p2[1] = 7;
printf("%d\n", arr[1]);

int *p3 = &arr[0];
p3[1] = 8;
printf("%d\n", arr[1]);
}

```

- 在Go语言中, 因为只有数据类型一模一样才能赋值, 所以只能通过&数组名赋值给指针变量, 才代表指针变量指向了这个数组

```

package main

import "fmt"

func main() {
    // 1. 错误, 在Go语言中必须类型一模一样才能赋值
    // arr类型是[3]int, p1的类型是*[3]int
    var p1 *[3]int
    fmt.Printf("%T\n", arr)
    fmt.Printf("%T\n", p1)
    p1 = arr // 报错
    p1[1] = 6
    fmt.Println(arr[1])

    // 2. 正确, &arr的类型是*[3]int, p2的类型也是*[3]int
    var p2 *[3]int
    fmt.Printf("%T\n", &arr)
    fmt.Printf("%T\n", p2)
    p2 = &arr
    p2[1] = 6
    fmt.Println(arr[1])

    // 3. 错误, &arr[0]的类型是*int, p3的类型也是*[3]int
    var p3 *[3]int
    fmt.Printf("%T\n", &arr[0])
    fmt.Printf("%T\n", p3)
    p3 = &arr[0] // 报错
    p3[1] = 6
    fmt.Println(arr[1])
}

```

- 注意点:
 - Go语言中的指针, 不支持C语言中的+1 -1和++ -- 操作

```

package main

import "fmt"

func main() {

    var arr [3]int = [3]int{1, 3, 5}
    var p *[3]int
    p = &arr
}

```

```

fmt.Printf("%p\n", &arr) // 0xc0420620a0
fmt.Printf("%p\n", p) // 0xc0420620a0
fmt.Println(&arr) // &[1 3 5]
fmt.Println(p) // &[1 3 5]
// 指针指向数组之后操作数组的几种方式
// 1. 直接通过数组名操作
arr[1] = 6
fmt.Println(arr[1])
// 2. 通过指针间接操作
(*p)[1] = 7
fmt.Println((*p)[1])
fmt.Println(arr[1])
// 3. 通过指针间接操作
p[1] = 8
fmt.Println(p[1])
fmt.Println(arr[1])

// 注意点: Go语言中的指针, 不支持+1 -1和++ --操作
*(p + 1) = 9 // 报错
fmt.Println(*p++) // 报错
fmt.Println(arr[1])
}

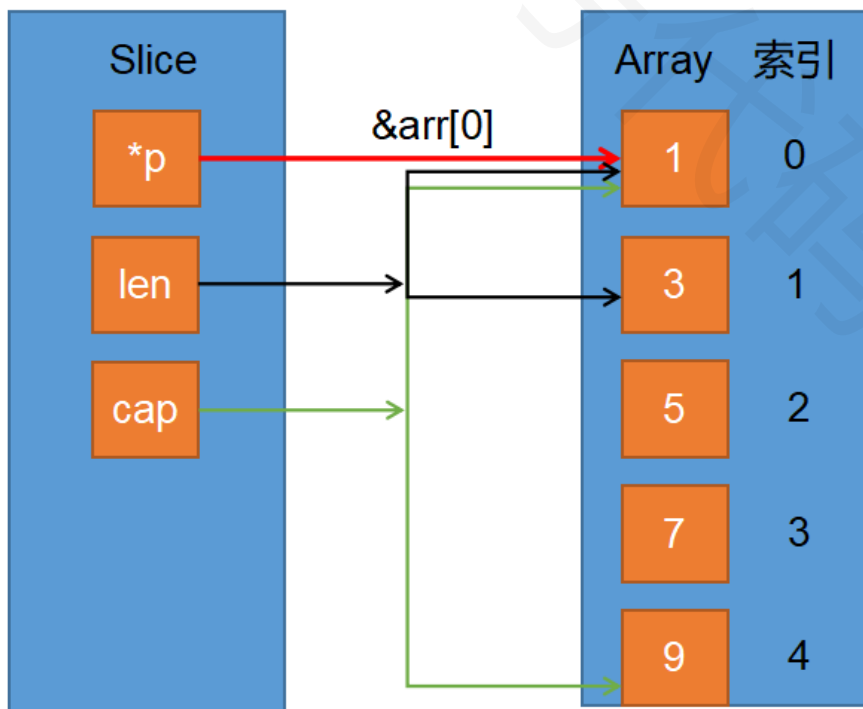
```

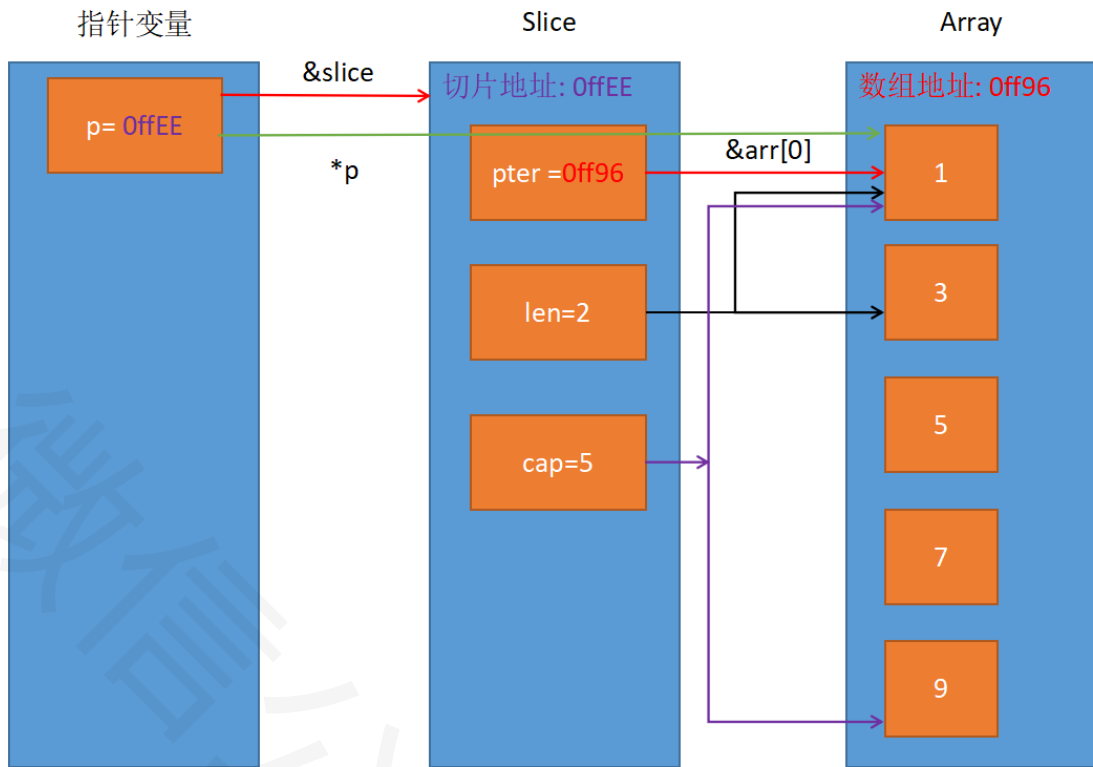
指向切片的指针

- 值得注意的是切片的本质就是一个指针指向数组, 所以指向切片的指针是一个二级指针

var scc = arr[0:2]

var arr = [5]int{1, 3, 5, 7, 9}





```
package main

import "fmt"

func main() {
    // 1. 定义一个切片
    var sce []int = []int{1, 3, 5}
    // 2. 打印切片的地址
    // 切片变量中保存的地址，也就是指向的那个数组的地址 sce = 0xc0420620a0
    fmt.Printf("sce = %p\n", sce)
    fmt.Println(sce) // [1 3 5]
    // 切片变量自己的地址，&sce = 0xc04205e3e0
    fmt.Printf("&sce = %p\n", &sce)
    fmt.Println(&sce) // &[1 3 5]
    // 3. 定义一个指向切片的指针
    var p *[]int
    // 因为必须类型一致才能赋值，所以将切片变量自己的地址给了指针
    p = &sce
    // 4. 打印指针保存的地址
    // 直接打印p打印出来的是保存的切片变量的地址 p = 0xc04205e3e0
    fmt.Printf("p = %p\n", p)
    fmt.Println(p) // &[1 3 5]
    // 打印*p打印出来的是切片变量保存的地址，也就是数组的地址 *p = 0xc0420620a0
    fmt.Printf("*p = %p\n", *p)
    fmt.Println(*p) // [1 3 5]

    // 5. 修改切片的值
    // 通过*p找到切片变量指向的存储空间(数组)，然后修改数组中保存的数据
    (*p)[1] = 666
    fmt.Println(sce[1])
}
```

指向字典指针

- 与普通指针并无差异

```
package main
import "fmt"
func main() {

    var dict map[string]string = map[string]string{"name":"lnj", "age":"33"}
    var p *map[string]string = &dict
    (*p)["name"] = "zs"
    fmt.Println(dict)
}
```

指向结构体指针

- Go语言中指向结构体的指针和C语言一样
- 结构体和指针
 - 创建结构体指针变量有两种方式

```
package main
import "fmt"
type Student struct {
    name string
    age int
}
func main() {
    // 创建时利用取地址符号获取结构体变量地址
    var p1 = &Student{"lnj", 33}
    fmt.Println(p1) // &{lnj 33}

    // 通过new内置函数传入数据类型创建
    // 内部会创建一个空的结构体变量，然后返回这个结构体变量的地址
    var p2 = new(Student)
    fmt.Println(p2) // &{ 0}
}
```

- 利用结构体指针操作结构体属性

```
package main
import "fmt"
type Student struct {
    name string
    age int
}
func main() {
    var p = &Student{}
    // 方式一：传统方式操作
    // 修改结构体中某个属性对应的值
    // 注意：由于.运算符优先级比*高，所以一定要加上()
    (*p).name = "lnj"
    // 获取结构体中某个属性对应的值
    fmt.Println((*p).name) // lnj
}
```

```
// 方式二：通过Go语法糖操作
// Go语言作者为了程序员使用起来更加方便，在操作指向结构体的指针时可以像操作接头体变量一样通过.来操作
// 编译时底层会自动转发为(*p).age方式
p.age = 33
fmt.Println(p.age) // 33
}
```

指针作为函数参数和返回值

- 如果指针类型作为函数参数, 修改形参会影响实参
- 不能将函数内的指向局部变量的指针作为返回值, 函数结束指向空间会被释放
- 可以将函数内的局部变量作为返回值, 本质是拷贝一份

方法

- Go语言中的方法其实就是一个特殊函数, 只不过这个函数是和某种属性类型绑定在一起的而已
- Go语言中的方法一般用于将函数和结构体绑定在一起, 让结构体除了能够保存数据外还能具备某些行为
- 将函数和数据类型绑定的格式
 - 只需要在函数名称前面加上(接收者 数据类型), 即可将函数和某种数据类型绑定在一起

```
func (接收者 数据类型)方法名称(形参列表)(返回值列表){
    方法体
}
```

- 示例: 给结构体添加一个方法,此时结构体除了可以保存数据,还具备说出自己名字和年龄的行为

```
package main
import "fmt"
// 1. 定义一个结构体
type Person struct {
    name string
    age int
}
// 2. 定义一个函数，并将这个函数和Person结构体绑定在一起
func (p Person)say() {
    fmt.Println("my name is", p.name, "my age is", p.age)
}
func main() {
    // 3. 创建一个结构体变量
    per := Person{"lnj", 33}
    // 4. 利用结构体变量调用和结构体绑定的方法
    // 调用时会自动将调用者(per)传递给方法的接收者(p)
    // 所以可以在方法内部通过p方法结构体变量的属性
    per.say()
}
```

- 方法和函数异同
 - 方法的数据类型也是函数类型, 所以也可以定义变量保存(作为参数返回值等)

```

package main
import "fmt"
type Person struct {
    name string
    age int
}
// 定义一个方法
func (p Person) say() {
    fmt.Println("say方法")
}
// 定义一个函数
func say() {
    fmt.Println("say函数")
}
func main() {
    p := Person{"lnj", 33}
    fmt.Printf("%T\n", p.say) // func()
    fmt.Printf("%T\n", say) // func()

    // 定义一个保存没有形参没有返回值的函数类型变量
    var fn func()
    // 利用函数类型变量保存一个方法
    fn = p.say
    // 利用函数类型变量调用一个方法
    fn()

    // 利用函数类型变量保存一个函数
    fn = say
    // 利用函数类型变量调用一个函数
    fn()
}

```

- 方法只能通过绑定类型的变量调用, 函数可以直接调用

```

package main
import "fmt"
type Person struct {
    name string
    age int
}
// 定义一个方法
func (p Person) say() {
    fmt.Println("my name is", p.name, "my age is", p.age)
}
// 定义一个函数
func test() {
    fmt.Println("test")
}
func main() {
    per := Person{"lnj", 33}
    per.say() // 方法只能通过绑定类型的变量调用
    //say() // 编译报错, 不能直接调用
    test() // 编译通过, 可以直接调用
}

```

- 方法的接收者可以看做就是函数的一个形参

```

package main
import "fmt"
type Person struct {
    name string
    age int
}
// 定义一个方法
func (p Person) say() {
    fmt.Println("my name is", p.name, "my age is", p.age)
}
// 定义一个函数
func test(p Person) {
    fmt.Println("my name is", p.name, "my age is", p.age)
}
func main() {
    per := Person{"lnj", 33}
    per.say() // my name is lnj my age is 33
    test(per) // my name is lnj my age is 33
}

```

- 既然可以看做形参, 那么自然也具备形参的特点(值传递和地址传递)

```

package main
import "fmt"
type Person struct {
    name string
    age int
}
// 接收者是一个变量
func (p Person) setName(name string) {
    p.name = name
}
// 接收者是一个指针
func (p *Person) setAge(age int) {
    p.age = age
}
func main() {
    per := Person{"lnj", 33}
    fmt.Println(per) // {lnj 33}
    // 值传递, 方法内部修改不会影响方法外部
    per.setName("zs")
    fmt.Println(per) // {lnj 33}
    p := &per
    // 地址传递, 方法内部修改会影响方法外部
    (*p).setAge(18)
    fmt.Println(per) // {lnj 18}
}

```

- 地址传递的几种调用方式

```

package main
import "fmt"
type Person struct {
    name string
    age int
}

```

```
// 接收者是一个变量
func (p Person)setName(name string) {
    p.name = name
}
// 接收者是一个指针
func (p *Person)setAge(age int) {
    p.age = age
}
func main() {
    per := Person{"lnj", 33}
    // 方式一：先拿到指针,然后再通过指针调用
    p := &per
    (*p).setAge(18)
    fmt.Println(per) // {lnj 18}
    // 方式二：直接利用变量调用,底层会自动获取变量地址传递给接收者
    per.setAge(66)
    fmt.Println(per) // {lnj 66}
}
```

接口

- Go语言中的接口和现实生活中的USB插槽很像,它定义某种标准,但不关心具体实现
 - 无论你到哪个商店里面去购买USB线,只要你告诉商家你需要一根USB线,买回家之后就一定能插到电脑上使用.之所以能用,原因就是电脑厂商在指定了USB插槽的标准(尺寸、排线等等),生产厂家只需要按照标准生产即可
 - 同样在Go语言中我们可以通过接口来定义某种标准(函数声明),但不用不关心具体实现(函数实现),只要将来有人按照标准实现了接口,我们就可以使用
- 定义接口格式

```
type 接口名称 interface{
    函数声明
}
```

- 示例: 定义一个通用的USB接口

```
package main
import "fmt"
// 1. 定义一个接口
type usber interface {
    start()
    stop()
}
type Computer struct {
    name string
    model string
}
// 2. 实现接口中的所有方法
func (cm Computer)start() {
    fmt.Println("启动电脑")
}
func (cm Computer)stop() {
    fmt.Println("关闭电脑")
}
```

```

type Phone struct {
    name string
    model string
}
// 2.实现接口中的所有方法
func (p Phone)start() {
    fmt.Println("启动手机")
}
func (p Phone)stop() {
    fmt.Println("关闭手机")
}

// 3.使用接口定义的方法
func working(u usber) {
    u.start()
    u.stop()
}
func main() {
    cm := Computer{"戴尔", "F1234"}
    working(cm) // 启动电脑 关闭电脑

    p := Phone{"华为", "M10"}
    working(p)  // 启动手机 关闭手机
}

```

- 接口注意点
 - 接口中只能有方法的声明不能有方法的实现

```

type usber interface {
    func start(){ // 错误
        fmt.Println("启动")
    }
    func stop() { // 错误
        fmt.Println("停止")
    }
}

```

- 接口中只能有方法不能有字段

```

type usber interface {
    name string // 错误
    start()
    stop()
}

```

- 只有实现了接口中所有的方法, 才算实现了接口, 才能用 该接口类型 接收

```

package main
import "fmt"
// 1.定义一个接口
type usber interface {
    start()
    stop()
}
type Computer struct {

```

```

    name string
    model string
}
// 2.实现接口中的所有方法
func (cm Computer)start() {
    fmt.Println("启动电脑")
}
func (cm Computer)stop() {
    fmt.Println("关闭电脑")
}
// 2.只实现了接口中部分方法
type Phone struct {
    name string
    model string
}
func (p Phone)start() {
    fmt.Println("启动手机")
}
func main() {
    // 1.定义一个usber接口类型变量
    var i usber
    // 2.用usber接口类型变量接收Computer类型结构体
    i = Computer{"戴尔", "F1234"} // 实现了所有方法, 不会报错
    // 3.用usber接口类型变量接收Phone类型结构体
    //i = Phone{"华为", "M10"} // 只实现了部分方法, 会报错
    fmt.Println(i)
}

```

- 和结构体一样,接口中也可以嵌入接口

```

package main
import "fmt"
type A interface {
    fna()
}
type B interface {
    fnb()
}
type C interface {
    A // 嵌入A接口
    B // 嵌入B接口
    fnc()
}
type Person struct {}
func (p Person)fna() {
    fmt.Println("实现A接口中的方法")
}
func (p Person)fnb() {
    fmt.Println("实现B接口中的方法")
}
func (p Person)fnc() {
    fmt.Println("实现C接口中的方法")
}
func main() {
    p := Person{}
    p.fna() // 实现A接口中的方法
    p.fnb() // 实现B接口中的方法
}

```

```
p.fnc() // 实现C接口中的方法
}
```

- 和结构体一样,接口中嵌入接口时不能嵌入自己

```
type A interface {
    A // 报错, 不能自己搞自己
}
```

- 接口中嵌入接口时不能出现相同的方法名称

```
type A interface {
    fn()
}
type B interface {
    fn()
}
type C interface {
    A
    B // 报错, A接口和B接口都有名称叫做fn的方法
    fnc()
}
```

- 超集接口变量可以自动转换成子集接口变量, 子集接口变量不能转换为超集接口变量(本质就是没有实现所有方法)

```
package main
import "fmt"
type aer interface {
    fna()
}
type ber interface {
    aer
    fnb()
}
// Person实现了超集接口所有方法
type Person struct {}
func (p Person)fna() {
    fmt.Println("实现A接口中的方法")
}
func (p Person)fnb() {
    fmt.Println("实现B接口中的方法")
}
// Student实现了子集接口所有方法
type Student struct {}
func (p Student)fna() {
    fmt.Println("实现A接口中的方法")
}
func main() {
    var i ber
    // 子集接口变量不能转换为超集接口变量
    //i = Student{}
    fmt.Println(i)
    var j aer
    // 超集接口变量可以自动转换成子集接口变量,
    j = Person{}
```

```
fmt.Println(j)
}
```

- 空接口类型可以接收任意类型数据

```
package main
import "fmt"
func main() {
    // 1. 定义一个空接口类型变量
    var i interface{}
    // 2. 用接口类型保存任意类型数据
    i = 123
    fmt.Println(i) // 123
    i = 3.14
    fmt.Println(i) // 3.14
    i = "lnj"
    fmt.Println(i) // lnj
    i = [3]int{1, 3, 5}
    fmt.Println(i) // [1 3 5]
    i = []int{2, 4, 6}
    fmt.Println(i) // [2 4 6]
    i = map[string]string{"name": "lnj"}
    fmt.Println(i) // map[name:lnj]
    i = Computer{"戴尔", "F1234"}
    fmt.Println(i) // {戴尔 F1234}
}
```

- 只要是自定义类型就可以实现接口

```
package main
import "fmt"
// 1. 定义一个接口
type usber interface {
    start()
    stop()
}
// 2. 自定义int类型
type integer int
// 2. 实现接口中的所有方法
func (i integer)start() {
    fmt.Println("int类型实现接口")
}
func (i integer)stop() {
    fmt.Println("int类型实现接口")
}
func main() {
    var i integer = 666
    i.start() // int类型实现接口
    i.stop() // int类型实现接口
}
```

- 接口类型转换

- 接口类型变量可以接收实现了该接口类型的变量, 但是只能调用该变量中的方法, 不能访问该变量的属性

```

package main
import "fmt"
type studier interface {
    read()
}
type Person struct {
    name string
    age int
}
func (p Person)read() {
    fmt.Println(p.name, "正在学习")
}
func main() {
    // 1. 定义一个接口类型变量
    var s studier
    // 2. 用接口类型变量接收实现了接口的结构体
    s = Person{"lnj", 33}
    s.name = "zs" // 报错, 由于s是接口类型, 所以不能访问属性
    fmt.Println(s)
}

```

- 想要访问变量中的属性, 必须将接口类型还原为原始类型

```

package main
import "fmt"
type studier interface {
    read()
}
type Person struct {
    name string
    age int
}
func (p Person)read() {
    fmt.Println(p.name, "正在学习")
}
func main() {
    var s studier
    s = Person{"lnj", 33}
    s.name = "zs" // 报错, 由于s是接口类型, 所以不能访问属性
    // 2. 定义一个结构体类型变量
    //var p Person
    // 不能用强制类型转换方式将接口类型转换为原始类型
    //p = Person(s) // 报错

    // 2. 利用ok-idiom模式将接口类型还原为原始类型
    // s.(Person)这种格式我们称之为: 类型断言
    if p, ok := s.(Person); ok {
        p.name = "zs"
        fmt.Println(p)
    }

    // 2. 通过 type switch将接口类型还原为原始类型
    // 注意: type switch不支持fallthrough
    switch p := s.(type) {
        case Person:
            p.name = "zs"
            fmt.Println(p) // {zs 33}
    }
}

```

```

        default:
            fmt.Println("不是Person类型")
    }
}

```

- 除了可以将接口类型转换为原始类型以外, 企业开发中有时候可能我们还需要将抽象接口类型转换为具体接口类型

```

package main
import "fmt"
type studier interface {
    read()
}
type Person struct {
    name string
    age int
}
func (p Person)read() {
    fmt.Println(p.name, "正在学习")
}
func main() {
    // 1. 定义一个抽象接口类型
    var i interface{}
    i = Person{"lnj", 33}
    // 不能调用read方法, 因为抽象接口中没有这个方法
    //i.read()
    // 2. 利用ok-idiom模式将抽象接口转换为具体接口
    if s, ok := i.(studier); ok{
        // 可以调用read方法, 因为studier中声明了这个方法, 并且结构体中实现了这个方法
        s.read() // lnj 正在学习
    }
}

```

面向对象基本概念

面向对象思想

- 面向对象(Object Oriented, OO)是软件开发方法
- 面向对象是一种对现实世界抽象的理解, 是计算机编程技术发展一定阶段后的产物

- Object Oriented Programming-OOP ——面向对象编程



面向对象和面向过程区别

- 面向对象是相对面向过程而言
 - 面向对象和面向过程都是一种思想
 - 面向过程
 - 强调的是功能行为
 - 关注的是解决问题需要哪些步骤
 - 回想下前面我们完成一个需求的步骤：
 - 首先搞清楚我们要做什么
 - 然后分析怎么做
 - 最后我用代码体现
 - 一步一步去实现，而具体的每一步都需要我们去实现和操作
 - 在上面每一个具体步骤中我们都是参与者，并且需要面对具体的每一个步骤和过程，这就是面向过程最直接的体现
-
- 面向对象
 - 将功能封装进对象，强调具备了功能的对象
 - 关注的是解决问题需要哪些对象
 - 当需求单一，或者简单时，我们一步一步去操作没问题，并且效率也挺高。可随着需求的更改，功能的增加，发现需要面对每一个步骤非常麻烦，这时就开始思索，能不能把这些步骤和功能再进行封装，封装时根据不同的功能，进行不同的封装，功能类似的封装在一起。这样结构就清晰多了，用的时候，找到对应的类就可以了，这就是面向对象思想
-
- 示例
 - 买电脑
 - 面向过程

- 了解电脑
 - 了解自己的需求
 - 对比参数
 - 去电脑城
 - 砍价，付钱
 - 买回电脑
 - 被坑
 - 面向对象
 - 找班长
 - 描述需求
 - 班长把电脑买回来
- *

-
- 吃饭
 - 面向过程
 - 去超市卖菜
 - 摘菜
 - 洗菜
 - 切菜
 - 炒菜
 - 盛菜
 - 吃
 - 面向对象
 - 去饭店
 - 点菜
 - 吃

-
- 洗衣服
 - 面向过程
 - 脱衣服
 - 放进盆里
 - 放洗衣液
 - 加水
 - 放衣服
 - 搓一搓
 - 清一清
 - 拧一拧
 - 晒起来
 - 面向对象
 - 脱衣服
 - 打开洗衣机
 - 丢进去
 - 一键洗衣烘干
 - 终极面向对象
 - 买电脑/吃饭/洗衣服
 - 找个对象

- 现实生活中我们是如何应用面相对象思想的

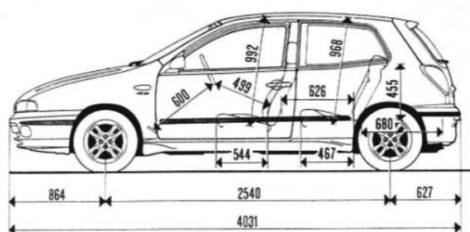
- 包工头
- 汽车坏了
- 面试

面向对象的特点

- 是一种符合人们思考习惯的思想
- 可以将复杂的事情简单化
- 将程序员从执行者转换成了指挥者
- 完成需求时：
 - 先要去找具有所需的功能的对象来用
 - 如果该对象不存在，那么创建一个具有所需功能的对象
 - 这样简化开发并提高复用

类与对象的关系

- 面向对象的核心就是对象,那怎么创建对象?
 - 现实生活中可以根据模板创建对象,编程语言也一样,也必须先有一个模板,在这个模板中说清楚将来创建出来的对象有哪些 属性 和 行为



创建



创建



- Go语言中的类相当于图纸，用来描述一类事物。也就是说要想创建对象必须先有类
- Go语言利用类来创建对象，对象是类的具体存在, 因此面向对象解决问题应该是先考虑需要设计哪些类，再利用类创建多少个对象

如何设计一个类

- 生活中描述事物无非就是描述事物的 属性 和 行为。
 - 如：人有身高，体重等属性，有说话，打架等行为。

事物名称(类名):人(Person)
属性:身高(height)、年龄(age)
行为(功能):跑(run)、打架(fight)

- Go语言中用类来描述事物也是如此
 - 属性: 对应类中的成员变量。
 - 行为: 对应类中的成员方法。
- 定义类其实在定义类中的成员(成员变量和成员方法)
- 拥有相同或者类似 属性 (状态特征) 和 行为 (能干什么事) 的对象都可以抽象成为一个类



如何分析一个类

- 一般名词都是类(名词提炼法)
 - 飞机发射两颗炮弹摧毁了8辆装甲车

飞机
炮弹
装甲车

- 隔壁老王在公车上牵着一只叼着热狗的草泥马

老王
热狗
草泥马

如何定义一个类

- 类是用于描述事物的属性和行为的, 而Go语言中的结构体正好可以用于描述事物的属性和行为
- 所以在Go语言中我们使用结构体来定义一个类型

```

type Person struct {
    name string // 人的属性
    age  int  // 人的属性
}
// 人的行为
func (p Person) Say() {
    fmt.Println("my name is", p.name, "my age is", p.age)
}

```

如何通过类创建一个对象

- 不过就是创建结构体的时候, 根据每个对象的特征赋值不同的属性罢了

```

// 3. 创建一个结构体变量
p1 := Person{"lnj", 33}
per.say()

p2 := Person{"zs", 18}
per.Say()

```

不同包中变量、函数、方法、类型公私有问题

- 在Go语言中通过首字母大小写来控制变量、函数、方法、类型的公私有
- 如果首字母小写, 那么代表私有, 仅在当前包中可以使用
- 如果首字母大写, 那么代表共有, 其它包中也可以使用

```

package demo

import "fmt"

var num1 int = 123 // 当前包可用
var Num1 int = 123 // 其它包也可用

type person struct { // 当前包可用
    name string // 当前包可用
    age  int  // 当前包可用
}

type Student struct { // 其它包也可用
    Name string // 其它包也可用
    Age  int  // 其它包也可用
}

func test1() { // 当前包可用
    fmt.Println("test1")
}

func Test2() { // 其它包也可用
    fmt.Println("Test2")
}

func (p person) say() { // 当前包可用
    fmt.Println(p.name, p.age)
}

func (s Student) Say() { // 其它包也可用

```

```
fmt.Println(s.Name, s.Age)
}
```

面向对象三大特性

- 封装性
 - 封装性就是隐藏实现细节，仅对外公开接口



- 类是数据与功能的封装，数据就是成员变量，功能就是方法
- 为什么要封装？
 - 不封装的缺点：当一个类把自己的成员变量暴露给外部的时候,那么该类就失去对该成员变量的管理权，别人可以任意的修改你的成员变量
 - 封装就是将数据隐藏起来,只能用此类的方法才可以读取或者设置数据,不可被外部任意修改是面向对象设计本质(将变化隔离)。这样降低了数据被误用的可能 (提高安全性和 灵活性)

```
package model
import "fmt"
type Person struct { // 其它包也可用
    name string // 当前包可用
    age int // 当前包可用
}
func (p *Person) SetAge(age int) {
    // 安全校验
    if age < 0 {
        fmt.Println("年龄不能为负数")
    }
    p.age = age
}
```

```
package main
import (
    "fmt"
    "main/model"
)
func main() {
    // 报错，因为name和age不是公开的
    //p := model.Person{"lnj", 18}

    // 方式一
    //p := model.Person{}
    //p.SetAge(18)
```

```
//fmt.Println(p)

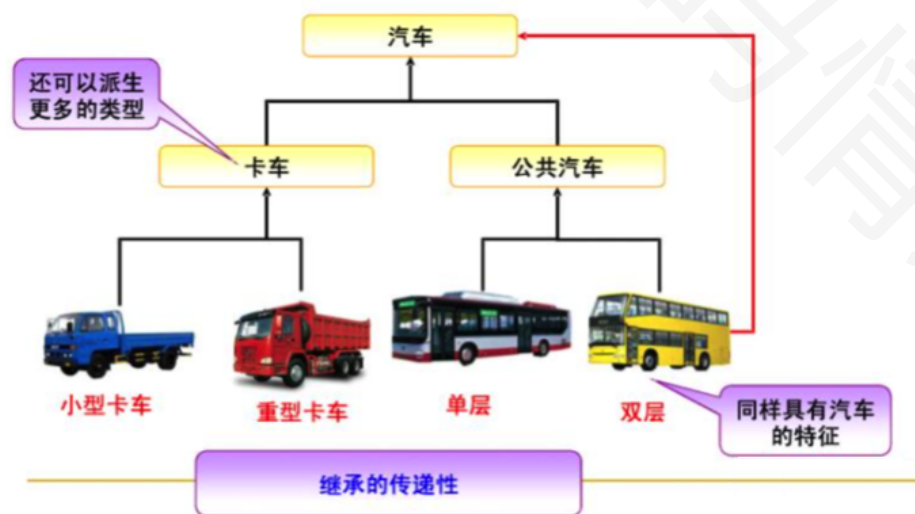
// 方式二
//p := new(model.Person)
//p.SetAge(18)
//fmt.Println(p)
}
```

- 封装原则

- 将不需要对外提供的内容都隐藏起来,把属性都隐藏,提供公共的方法对其访问

- 继承性

- Go语言认为虽然继承能够提升代码的复用性,但是会让代码腐烂,并增加代码的复杂度.
 - 所以go语言坚持了“组合优于继承”的原则, Go语言中所谓的继承其实是利用组合实现的(匿名结构体属性)



- 普通继承(组合)

```
package main

import "fmt"
```

```

type Person struct {
    name string
    age int
}
type Student struct {
    Person // 学生继承了人的特性
    score int
}
type Teacher struct {
    Person // 老师继承了人的特性
    Title string
}
func main() {
    s := Student{Person{"lnj", 18}, 99}
    //fmt.Println(s.Person.name)
    fmt.Println(s.name) // 两种方式都能访问
    //fmt.Println(s.Person.age)
    fmt.Println(s.age) // 两种方式都能访问
    fmt.Println(s.score)
}

```

- 继承结构中出现重名情况, 采用就近原则

```

package main

import "fmt"

type Person struct {
    name string // 属性重名
    age int
}
type Student struct {
    Person
    name string // 属性重名
    score int
}
func main() {
    s := Student{Person{"zs", 18}, "ls", 99}

    fmt.Println(s.Person.name) // zs
    fmt.Println(s.name) // ls
    //fmt.Println(s.Person.age)
    fmt.Println(s.age) // 两种方式都能访问
    fmt.Println(s.score)
}

```

- 多重继承

```

package main

import "fmt"

type Object struct {
    life int
}

```

```

type Person struct {
    Object
    name string
    age int
}
type Student struct {
    Person
    score int
}
func main() {
    s := Student{Person{Object{77}, "zs", 33}, 99}
    //fmt.Println(s.Person.Object.life)
    //fmt.Println(s.Person.life)
    fmt.Println(s.life) // 三种方式都可以
    //fmt.Println(s.Person.name)
    fmt.Println(s.name) // 两种方式都能访问
    //fmt.Println(s.Person.age)
    fmt.Println(s.age) // 两种方式都能访问
    fmt.Println(s.score)
}

```

```

package main

import "fmt"

type Object struct {
    life int
}
type Person struct {
    name string
    age int
}
type Student struct {
    Object
    Person
    score int
}
func main() {
    s := Student{Object{77}, Person{"zs", 33}, 99}
    //fmt.Println(s.Person.life)
    fmt.Println(s.life) // 两种方式都可以
    //fmt.Println(s.Person.name)
    fmt.Println(s.name) // 两种方式都能访问
    //fmt.Println(s.Person.age)
    fmt.Println(s.age) // 两种方式都能访问
    fmt.Println(s.score)
}

```

- 方法继承
 - 在Go语言中子类不仅仅能够继承父类的属性, 还能够继承父类的方法

```

package main

import "fmt"
type Person struct {
    name string
    age int
}

```

```

}
// 父类方法
func (p Person) say() {
    fmt.Println("name is ", p.name, "age is ", p.age)
}

type Student struct {
    Person
    score float32
}

func main() {
    stu := Student{Person{"zs", 18}, 59.9}
    stu.say()
}

```

- 继承中的方法重写
 - 如果子类有和父类同名的方法, 那么我们称之为方法重写

```

package main

import "fmt"
type Person struct {
    name string
    age int
}
// 父类方法
func (p Person) say() {
    fmt.Println("name is ", p.name, "age is ", p.age)
}

type Student struct {
    Person
    score float32
}
// 子类方法
func (s Student) say() {
    fmt.Println("name is ", s.name, "age is ", s.age, "score is ", s.score)
}
func main() {
    stu := Student{Person{"zs", 18}, 59.9}
    // 和属性一样, 访问时采用就近原则
    stu.say()
    // 和属性一样, 方法同名时可以通过指定父类名称的方式, 访问父类方法
    stu.Person.say()
}

```

- 注意点:
 - 无论是属性继承还是方法继承, 都只能子类访问父类, 不能父类访问子类

-
- 多态性
 - 多态就是某一类事物的多种形态

猫：猫-->动物
狗：狗-->动物
男人：男人 -->人 -->高级动物
女人：女人 -->人 -->高级动物

- Go语言中的多态是采用接口来实现的

```
package main

import "fmt"
// 1.定义接口
type Animal interface {
    Eat()
}
type Dog struct {
    name string
    age int
}
// 2.实现接口方法
func (d Dog)Eat() {
    fmt.Println(d.name, "正在吃东西")
}

type Cat struct {
    name string
    age int
}
// 2.实现接口方法
func (c Cat)Eat() {
    fmt.Println(c.name, "正在吃东西")
}
// 3.对象特有方法
func (c Cat)Special() {
    fmt.Println(c.name, "特有方法")
}

func main() {
    // 1.利用接口类型保存实现了所有接口方法的对象
    var a Animal
    a = Dog{"旺财", 18}
    // 2.利用接口类型调用对象中实现的方法
    a.Eat()
    a = Cat{"喵喵", 18}
    a.Eat()
    // 3.利用接口类型调用对象特有的方法
    //a.Special() // 接口类型只能调用接口中声明的方法，不能调用对象特有方法
    if cat, ok := a.(Cat); ok{
        cat.Special() // 只有对象本身才能调用对象的特有方法
    }
}
```

- 多态优点
 - 多态的主要好处就是简化了编程接口。它允许在类和类之间重用一些习惯性的命名,而不用为每一个新的方法命名一个新名字。这样,编程接口就是一些抽象的行为的集合,从而和实现接口的类的区分开来

- 多态也使得代码可以分散在不同的对象中而不用试图在一个方法中考虑到所有可能的对象。这样使得您的代码扩展性和复用性更好一些。当一个新的情景出现时,您无须对现有的代码进行改动,而只需要增加一个新的类和新的同名方法

异常处理

- 程序运行时,发生的不被期望的事件,它阻止了程序按照程序员的预期正常执行,这就是异常
- go语言中提供了两种处理异常的方式
 - 一种是程序发生异常时,将异常信息反馈给使用者
 - 一种是程序发生异常时,立刻退出终止程序继续运行

打印异常信息

- Go语言中提供了两种创建异常信息的方式
- 方式一:通过fmt包中的Errorf函数创建错误信息,然后打印

```
package main
import "fmt"
func main() {
    // 1.创建错误信息
    var err error = fmt.Errorf("这里是错误信息")
    // 2.打印错误信息
    fmt.Println(err) // 这里是错误信息
}
```

- 方式二:通过errors包中的New函数创建错误信息,然后打印

```
package main
import "fmt"
func main() {
    // 1.创建错误信息
    var err error = errors.New("这里是错误信息")
    // 2.打印错误信息
    fmt.Println(err) // 这里是错误信息
}
```

- 两种创建异常信息实现原理解析
 - Go语言中创建异常信息其实都是通过一个error接口实现的
 - Go语言再 builtin 包中定义了一个名称叫做error的接口. 源码如下

```
package builtin
// 定义了一个名称叫做error的接口
// 接口中声明了一个叫做Error() 的方法
type error interface {
    Error() string
}
```

- 在errors包中定义了一个名称叫做errorString的结构体,利用这个结构体实现了error接口中指定的方法
- 并且在errors 包中还提供了一个New方法,用于创建实现了error接口的结构体对象,并且在创建时就会把指定的字符串传递给这个结构体

```
// 指定包名为errors
package errors
// 定义了一个名称叫做errorString的结构体，里面有一个字符串类型属性s
type errorString struct {
    s string
}
// 实现了error接口中的Error方法
// 内部直接将结构体中保存的字符串返回
func (e *errorString) Error() string {
    return e.s
}
// 定义了一个New函数，用于创建异常信息
// 注意：New函数的返回值是一个接口类型
func New(text string) error {
    // 返回一个创建好的errorString结构体地址
    return &errorString{text}
}
```

- fmt包中Errorf底层的实现原理其实就是在内部自动调用了errors包中的New函数

```
func Errorf(format string, a ...interface{}) error {
    return errors.New(Sprintf(format, a...))
}
```

- 应用场景

```
package main
import "fmt"
func div(a, b int) (res int, err error) {
    if(b == 0){
        // 一旦传入的除数为0，就会返回error信息
        err = errors.New("除数不能为0")
    }else{
        res = a / b
    }
    return
}
func main() {
    //res, err := div(10, 5)
    res, err := div(10, 0)
    if(err != nil){
        fmt.Println(err) // 除数不能为0
    }else{
        fmt.Println(res) // 2
    }
}
```

中断程序

- Go语言中提供了一个叫做panic函数, 用于发生异常时终止程序继续运行

```
package main
import "fmt"
func div(a, b int) (res int) {
```

```

if(b == 0){
    //一旦传入的除数为0，程序就会终止
    panic("除数不能为0")
}else{
    res = a / b
}
return
}
func main() {
    res := div(10, 0)
    fmt.Println(res)
}

```

- Go语言中有两种方式可以触发panic终止程序
 - 我们自己手动调用panic函数
 - 程序内部出现问题自动触发panic函数

```

package main
import "fmt"
func main() {
    // 例如:数组角标越界，就会自动触发panic
    var arr = [3]int{1, 3, 5}
    arr[5] = 666 // 报错
    fmt.Println(arr)

    // 例如:除数为0，就会自动触发panic
    var res = 10 / 0
    fmt.Println(res)
}

```

- 除非是不可恢复性、导致系统无法正常工作的错误, 否则不建议使用panic

恢复程序

- 程序和人一样都需要具备一定的容错能力, 学会知错就改. 所以如果不是不可恢复性、导致系统无法正常工作的错误, 如果发生了panic我们需要恢复程序, 让程序继续执行, 并且需要记录到底犯了什么错误
- 在Go语言中我们可以通过defer和recover来实现panic异常的捕获, 让程序继续执行

```

package main
import "fmt"
func div(a, b int) (res int) {
    // 定义一个延迟调用的函数，用于捕获panic异常
    // 注意：一定要在panic之前定义
    defer func() {
        if err := recover(); err != nil{
            res = -1
            fmt.Println(err) // 除数不能为0
        }
    }()
    if(b == 0){
        //err = errors.New("除数不能为0")
        panic("除数不能为0")
    }else{

```

```

        res = a / b
    }
    return
}

func setValue(arr []int, index int, value int) {
    arr[index] = value
}

func main() {
    res := div(10, 0)
    fmt.Println(res) // -1
}

```

- panic注意点
 - panic异常会沿着调用堆栈向外传递, 所以也可以在外层捕获

```

package main
import "fmt"
func div(a, b int) (res int) {
    if(b == 0){
        //err = errors.New("除数不能为0")
        panic("除数不能为0")
    }else{
        res = a / b
    }
    return
}

func main() {
    // panic异常会沿着调用堆栈向外传递, 所以也可以在外层捕获
    defer func() {
        if err := recover(); err != nil{
            fmt.Println(err) // 除数不能为0
        }
    }()
    div(10, 0)
}

```

- 多个异常,只有第一个会被捕获

```

package main
import "fmt"
func test1() {
    // 多个异常,只有第一个会被捕获
    defer func() {
        if err := recover(); err != nil{
            fmt.Println(err) // 异常A
        }
    }()
    panic("异常A") // 相当于return, 后面代码不会继续执行
    panic("异常B")
}

func main() {
    test1(10, 0)
}

```

- 如果有异常写在defer中, 那么只有defer中的异常会被捕获

```
package main
import "fmt"
func test2() {
    // 如果有异常写在defer中, 并且其它异常写在defer后面, 那么只有defer中的异常会被捕获
    defer func() {
        if err := recover(); err != nil{
            fmt.Println(err) // 异常A
        }
    }()

    defer func() {
        panic("异常B")
    }()
    panic("异常A")
}
func main() {
    test1(10, 0)
}
```

字符串相关方法

- 获取字符串长度
 - 注意: Go语言编码方式是UTF-8,在UTF-8中一个汉字占3个字节

```
package main
import "fmt"
func main() {
    str1 := "lnj"
    fmt.Println(len(str1)) // 3
    str2 := "公号: 代码情缘"
    fmt.Println(len(str2)) // 12
}
```

- 如果字符串中包含中文, 又想精确的计算字符串中字符的个数而不是占用的字节, 那么必须先将字符串转换为rune类型数组
 - Go语言中byte用于保存字符, rune用于保存汉字

```
package main
import "fmt"
func main() {
    str := "公号: 代码情缘"
    // 注意byte占1个字节, 只能保存字符不能保存汉字, 因为一个汉字占用3个字节
    arr1 := []byte(str) // 12
    fmt.Println(len(arr1))
    for _, v := range arr1{
        fmt.Printf("%c", v) // lnjæ••å••æ±•
    }

    // Go语言中rune类型就是专门用于保存汉字的
    arr2 := []rune(str)
    fmt.Println(len(arr2)) // 6
    for _, v := range arr2{
        fmt.Printf("%c", v) // lnj李南江
    }
}
```

```
}  
}
```

- 查找子串在字符串中出现的位置

- func Index(s, sep string) int
- func IndexByte(s string, c byte) int
- func IndexRune(s string, r rune) int
- func IndexAny(s, chars string) int
- func IndexFunc(s string, f func(rune) bool) int
- func LastIndex(s, sep string) int
- func LastIndexByte(s string, c byte) int
- func LastIndexAny(s, chars string) int
- func LastIndexFunc(s string, f func(rune) bool) int

```
package main  
import (  
    "strings"  
    "fmt"  
)  
func main() {  
    // 查找`字符`在字符串中第一次出现的位置，找不到返回-1  
    res := strings.IndexByte("hello 李南江", 'l')  
    fmt.Println(res) // 2  
  
    // 查找`汉字`OR`字符`在字符串中第一次出现的位置，找不到返回-1  
    res = strings.IndexRune("hello 李南江", '李')  
    fmt.Println(res) // 6  
    res = strings.IndexRune("hello 李南江", 'l')  
    fmt.Println(res) // 2  
  
    // 查找`汉字`OR`字符`中任意一个在字符串中第一次出现的位置，找不到返回-1  
    res = strings.IndexAny("hello 李南江", "wmh")  
    fmt.Println(res) // 2  
    // 会把wmh拆开逐个查找，w、m、h、l只要任意一个被找到，立刻停止查找  
    res = strings.IndexAny("hello 李南江", "wmhl")  
    fmt.Println(res) // 0  
    // 查找`子串`在字符串第一次出现的位置，找不到返回-1  
    res = strings.Index("hello 李南江", "lle")  
    fmt.Println(res) // 2  
    // 会把lle当做一个整体去查找，而不是拆开  
    res = strings.Index("hello 李南江", "lle")  
    fmt.Println(res) // -1  
    // 可以查找字符也可以查找汉字  
    res = strings.Index("hello 李南江", "李")  
    fmt.Println(res) // 6  
  
    // 会将字符串先转换为[]rune，然后遍历rune切片逐个取出传给自定义函数  
    // 只要函数返回true,代表符合我们的需求，既立即停止查找  
    res = strings.IndexFunc("hello 李南江", custom)  
    fmt.Println(res) // 6  
  
    // 倒序查找`子串`在字符串第一次出现的位置，找不到返回-1  
    res := strings.LastIndex("hello 李南江", "l")  
    fmt.Println(res) // 3  
}  
func custom(r rune) bool {
```

```

fmt.Printf("被调用了，当前传入的是%c\n", r)
if r == 'o' {
    return true
}
return false
}

```

- 判断字符串中是否包含子串

- func Contains(s, substr string) bool
- func ContainsRune(s string, r rune) bool
- func ContainsAny(s, chars string) bool
- func HasPrefix(s, prefix string) bool
- func HasSuffix(s, suffix string) bool

```

package main
import (
    "strings"
    "fmt"
)
func main() {
    // 查找`子串`在字符串中是否存在，存在返回true，不存在返回false
    // 底层实现就是调用strings.Index函数
    res := strings.Contains( "hello 代码情缘", "llo")
    fmt.Println(res) // true

    // 查找`汉字`OR`字符`在字符串中是否存在，存在返回true，不存在返回false
    // 底层实现就是调用strings.IndexRune函数
    res = strings.ContainsRune( "hello 代码情缘", 'l')
    fmt.Println(res) // true
    res = strings.ContainsRune( "hello 代码情缘", '李')
    fmt.Println(res) // true

    // 查找`汉字`OR`字符`中任意一个在字符串中是否存在，存在返回true，不存在返回false
    // 底层实现就是调用strings.IndexAny函数
    res = strings.ContainsAny( "hello 代码情缘", "wmhl")
    fmt.Println(res) // true

    // 判断字符串是否已某个字符串开头
    res = strings.HasPrefix("lnj-book.avi", "lnj")
    fmt.Println(res) // true

    // 判断字符串是否已某个字符串结尾
    res = strings.HasSuffix("lnj-book.avi", ".avi")
    fmt.Println(res) // true
}

```

- 字符串比较

- func Compare(a, b string) int
- func EqualFold(s, t string) bool

```

package main
import (
    "strings"
    "fmt"
)

```

```

func main() {
    // 比较两个字符串大小，会逐个字符地进行比较ASCII值
    // 第一个参数 > 第二个参数 返回 1
    // 第一个参数 < 第二个参数 返回 -1
    // 第一个参数 == 第二个参数 返回 0
    res := strings.Compare("bcd", "abc")
    fmt.Println(res) // 1
    res = strings.Compare("bcd", "bdc")
    fmt.Println(res) // -1
    res = strings.Compare("bcd", "bcd")
    fmt.Println(res) // 0

    // 判断两个字符串是否相等，可以判断字符和中文
    // 判断时会忽略大小写进行判断
    res2 := strings.EqualFold("abc", "def")
    fmt.Println(res2) // false
    res2 = strings.EqualFold("abc", "abc")
    fmt.Println(res2) // true
    res2 = strings.EqualFold("abc", "ABC")
    fmt.Println(res2) // true
    res2 = strings.EqualFold("代码情缘", "代码情缘")
    fmt.Println(res2) // true
}

```

- 字符串转换

- func ToUpper(s string) string
- func ToLower(s string) string
- func ToTitle(s string) string
- func ToUpperSpecial(_case unicode.SpecialCase, s string) string
- func ToLowerSpecial(_case unicode.SpecialCase, s string) string
- func ToTitleSpecial(_case unicode.SpecialCase, s string) string
- func Title(s string) string

```

package main
import (
    "strings"
    "fmt"
)
func main() {
    // 将字符串转换为小写
    res := strings.ToLower("ABC")
    fmt.Println(res) // abc

    // 将字符串转换为大写
    res = strings.ToUpper("abc")
    fmt.Println(res) // ABC

    // 将字符串转换为标题格式，大部分`字符`标题格式就是大写
    res = strings.ToTitle("hello world")
    fmt.Println(res) // HELLO WORLD
    res = strings.ToTitle("HELLO WORLD")
    fmt.Println(res) // HELLO WORLD

    // 将单词首字母变为大写，其它字符不变
    // 单词之间用空格OR特殊字符隔开
    res = strings.Title("hello world")
}

```

```
fmt.Println(res) // Hello world
}
```

- 字符串拆合

- func Split(s, sep string) []string
- func SplitN(s, sep string, n int) []string
- func SplitAfter(s, sep string) []string
- func SplitAfterN(s, sep string, n int) []string
- func Fields(s string) []string
- func FieldsFunc(s string, f func(rune) bool) []string
- func Join(a []string, sep string) string
- func Repeat(s string, count int) string
- func Replace(s, old, new string, n int) string

```
package main
import (
    "strings"
    "fmt"
)
func main() {
    // 按照指定字符串切割原字符串
    // 用,切割字符串
    arr1 := strings.Split("a,b,c", ",")
    fmt.Println(arr1) // [a b c]
    arr2 := strings.Split("ambmc", "m")
    fmt.Println(arr2) // [a b c]

    // 按照指定字符串切割原字符串，并且指定切割为几份
    // 如果最后一个参数为0，那么会返回一个空数组
    arr3 := strings.SplitN("a,b,c", ",", 2)
    fmt.Println(arr3) // [a b,c]
    arr4 := strings.SplitN("a,b,c", ",", 0)
    fmt.Println(arr4) // []

    // 按照指定字符串切割原字符串，切割时包含指定字符串
    arr5 := strings.SplitAfter("a,b,c", ",")
    fmt.Println(arr5) // [a, b, c]

    // 按照指定字符串切割原字符串，切割时包含指定字符串，并且指定切割为几份
    arr6 := strings.SplitAfterN("a,b,c", ",", 2)
    fmt.Println(arr6) // [a, b,c]

    // 按照空格切割字符串，多个空格会合并为一个空格处理
    arr7 := strings.Fields("a b c d")
    fmt.Println(arr7) // [a b c d]

    // 将字符串转换成切片传递给函数之后由函数决定如何切割
    // 类似于IndexFunc
    arr8 := strings.FieldsFunc("a,b,c", custom)
    fmt.Println(arr8) // [a b c]

    // 将字符串切片按照指定连接符号转换为字符串
    sce := []string{"aa", "bb", "cc"}
    str1 := strings.Join(sce, "-")
    fmt.Println(str1) // aa-bb-cc
}
```

```

// 返回count个s串联的指定字符串
str2 := strings.Repeat("abc", 2)
fmt.Println(str2) // abcabc

// 第一个参数：需要替换的字符串
// 第二个参数：旧字符串
// 第三个参数：新字符串
// 第四个参数：用新字符串 替换 多少个旧字符串
// 注意点：传入-1代表只要有旧字符串就替换
// 注意点：替换之后会生成新字符串，原字符串不会受到影响
str3 := "abcdefabcdefabc"
str4 := strings.Replace(str3, "abc", "mmm", -1)
fmt.Println(str3) // abcdefabcdefabc
fmt.Println(str4) // mmmdefmmmdefmmm
}
func custom(r rune) bool {
    fmt.Printf("被调用了，当前传入的是%c\n", r)
    if r == ',' {
        return true
    }
    return false
}
}

```

- 字符串清理

- func Trim(s string, cutset string) string
- func TrimLeft(s string, cutset string) string
- func TrimRight(s string, cutset string) string
- func TrimFunc(s string, f func(rune) bool) string
- func TrimLeftFunc(s string, f func(rune) bool) string
- func TrimRightFunc(s string, f func(rune) bool) string
- func TrimSpace(s string) string
- func TrimPrefix(s, prefix string) string
- func TrimSuffix(s, suffix string) string

```

package main
import (
    "strings"
    "fmt"
)
func main() {
    // 去除字符串两端指定字符
    str1 := strings.Trim("!!!abc!!!def!!!", "!")
    fmt.Println(str1) // abc!!!def
    // 去除字符串左端指定字符
    str2 := strings.TrimLeft("!!!abc!!!def!!!", "!")
    fmt.Println(str2) // abc!!!def!!!
    // 去除字符串右端指定字符
    str3 := strings.TrimRight("!!!abc!!!def!!!", "!")
    fmt.Println(str3) // !!!abc!!!def
    // // 去除字符串两端空格
    str4 := strings.TrimSpace("  abc!!!def ")
    fmt.Println(str4) // abc!!!def

    // 按照方法定义规则，去除字符串两端符合规则内容
    str5 := strings.TrimFunc("!!!abc!!!def!!!", custom)
}

```

```

fmt.Println(str5) // abc!!!def
// 按照方法定义规则,去除字符串左端符合规则内容
str6 := strings.TrimLeftFunc("!!!abc!!!def!!!", custom)
fmt.Println(str6) // abc!!!def!!!
// 按照方法定义规则,去除字符串右端符合规则内容
str7 := strings.TrimRightFunc("!!!abc!!!def!!!", custom)
fmt.Println(str7) // !!!abc!!!def

// 取出字符串开头的指定字符串
str8 := strings.TrimPrefix("lnj-book.avi", "lnj-")
fmt.Println(str8) // book.avi

// 取出字符串结尾的指定字符串
str9 := strings.TrimSuffix("lnj-book.avi", ".avi")
fmt.Println(str9) // lnj-book
}

```

正则表达式

- 正则表达式是对字符串操作的一种逻辑公式，就是用事先定义好的一些特定字符、及这些特定字符的组合，组成一个“规则字符串”，这个“规则字符串”用来表达对字符串的一种过滤逻辑。
- 相关规则标准详见
 - [百度百科](#)
 - [Go语言官方文档](#)regexp包
- Go语言中正则表达式使用步骤
 - 1.创建一个正则表达式匹配规则对象
 - 2.利用正则表达式匹配规则对象匹配指定字符串

```

package main
import (
    "strings"
    "fmt"
)
func main() {
    // 创建一个正则表达式匹配规则对象
    // reg := regexp.MustCompile(规则字符串)
    // 利用正则表达式匹配规则对象匹配指定字符串
    // 会将所有匹配到的数据放到一个字符串切片中返回
    // 如果没有匹配到数据会返回nil
    // res := reg.FindAllString(需要匹配的字符串, 匹配多少个)

    str := "Hello 李南江 1232"
    reg := regexp.MustCompile("2")
    res := reg.FindAllString(str, -1)
    fmt.Println(res) // [2 2]
    res = reg.FindAllString(str, 1)
    fmt.Println(res) // [2]
}

```

- 匹配电话号码

```

package main
import (

```

```

"strings"
"fmt"
)
func main() {
    res2 := findPhoneNumber("13554499311")
    fmt.Println(res2) // true

    res2 = findPhoneNumber("03554499311")
    fmt.Println(res2) // false

    res2 = findPhoneNumber("1355449931")
    fmt.Println(res2) // false
}
func findPhoneNumber(str string) bool {
    // 创建一个正则表达式匹配规则对象
    reg := regexp.MustCompile("^1[1-9]{10}")
    // 利用正则表达式匹配规则对象匹配指定字符串
    res := reg.FindAllString(str, -1)
    if(res == nil){
        return false
    }
    return true
}

```

- 匹配Email

```

package main
import (
    "strings"
    "fmt"
)
func main() {
    res2 = findEmail("123@qq.com")
    fmt.Println(res2) // true

    res2 = findEmail("ab?de@qq.com")
    fmt.Println(res2) // false

    res2 = findEmail("123@qqcom")
    fmt.Println(res2) // false
}
func findEmail(str string) bool {
    reg := regexp.MustCompile("^[a-zA-Z0-9_]+@[a-zA-Z0-9]+\.[a-zA-Z0-9]+")
    res := reg.FindAllString(str, -1)
    if(res == nil){
        return false
    }
    return true
}

```

时间和日期函数

- 获取当前时间

```
package main

import (
    "fmt"
    "time"
)

func main() {
    var t time.Time = time.Now()
    // 2018-09-27 17:25:11.653198 +0800 CST m=+0.009759201
    fmt.Println(t)
}
```

- 获取年月日时分秒

```
package main

import (
    "fmt"
    "time"
)

func main() {
    var t time.Time = time.Now()
    fmt.Println(t.Year())
    fmt.Println(t.Month())
    fmt.Println(t.Day())
    fmt.Println(t.Hour())
    fmt.Println(t.Minute())
    fmt.Println(t.Second())
}
```

- 格式化时间

```
package main

import (
    "fmt"
    "time"
)

func main() {
    var t time.Time = time.Now()
    fmt.Printf("当前的时间是: %d-%d-%d %d:%d:%d\n", t.Year(),
        t.Month(), t.Day(), t.Hour(), t.Minute(), t.Second())

    var dateStr = fmt.Sprintf("%d-%d-%d %d:%d:%d", t.Year(),
        t.Month(), t.Day(), t.Hour(), t.Minute(), t.Second())
    fmt.Println("当前的时间是:", dateStr)
}
```

```
package main

import (
    "fmt"
    "time"
)

func main() {
```

```

var t time.Time = time.Now()
// 2006/01/02 15:04:05这个字符串是Go语言规定的，各个数字都是固定的
// 字符串中的各个数字可以只有组合，这样就能按照需求返回格式化好的时间
str1 := t.Format("2006/01/02 15:04:05")
fmt.Println(str1)
str2 := t.Format("2006/01/02")
fmt.Println(str2)
str3 := t.Format("15:04:05")
fmt.Println(str3)
}

```

- 时间常量

- 一般用于指定时间单位, 和休眠函数配合使用
- 例如: 100毫秒, `100 * time.Millisecond`

```

const (
    Nanosecond Duration = 1    // 纳秒
    Microsecond      = 1000 * Nanosecond // 微秒
    Millisecond       = 1000 * Microsecond // 毫秒
    Second           = 1000 * Millisecond  // 秒
    Minute           = 60 * Second         // 分钟
    Hour             = 60 * Minute         // 小时
)

```

```

package main

import (
    "fmt"
    "time"
)

func main() {
    for{
        // 1秒钟打印一次
        time.Sleep(time.Second * 1)
        // 0.1秒打印一次
        //time.Sleep(time.Second * 0.1)
        time.Sleep(time.Millisecond * 100)
        fmt.Println("Hello LNJ")
    }
}

```

- 获取当前时间戳

- Unix秒
- UnixNano纳秒
- 一般用于配合随机函数使用, 作为随机函数随机种子

```

package main

import (
    "fmt"
    "time"
)

func main() {

```

```

t := time.Now()
// 获取1970年1月1日距离当前的时间(秒)
fmt.Println(t.Unix())
// 获取1970年1月1日距离当前的时间(纳秒)
fmt.Println(t.UnixNano())
}

```

```

package main

import (
    "fmt"
    "math/rand"
    "time"
)

func main() {
    // 创建随机数种子
    rand.Seed(time.Now().UnixNano())
    // 生成一个随机数
    fmt.Println(rand.Intn(10))
}

```

Go语言中调用C语言函数

- 在Go语言开篇中我们已经知道, Go语言与C语言之间有着千丝万缕的关系, 甚至被称之为21世纪的C语言
- 所以在Go与C语言互操作方面, Go更是提供了强大的支持。尤其是在Go中使用C, 你甚至可以直接在Go源文件中编写C代码, 这是其他语言所无法望其项背的
- 格式:
 - 在import "C"之前通过单行注释或者通过多行注释编写C语言代码
 - 在import "C"之后编写Go语言代码
 - 在Go语言代码中通过C.函数名称() 调用C语言代码即可
 - 注意: import "C"和前面的注释之间不能出现空行或其它内容, 必须紧紧相连

```

package main
//#include <stdio.h>
//void say(){
// printf("Hello world\n");
//}
import "C"

func main() {
    C.say()
}

```

```

package main
/*
#include <stdio.h>
void say(){
    printf("Hello world\n");
}
*/
import "C"

func main() {
    C.say()
}

```

- Go语言中没有包名是C的包, 但是这个导入会促使Go编译器利用cgo工具预处理文件
- 在预处理过程中,cgo会产生一个临时包, 这个包里包含了所有C函数和类型对应的Go语言声明
- 最终使得cgo工具可以通过一种特殊的方式来调用import "C"之前的C语言代码
- 常规问题:
 - 如果编译报错 `cc1.exe: sorry, unimplemented: 64-bit mode not compiled in`
 - 说明你使用的是64的golang, 而你使用的32位的MinGW, 所以需要下载64位的mingw并配置环境变量
 - 下载地址: <https://www.baidu.com/s?wd=sorry%2C%20unimplemented%3A%2064-bit%20mode%20not%20compiled%20in>

C语言中调用Go语言函数(很少使用)

- 在Go代码中通过 `//export Go函数名称` 导出Go的函数名称
- 在C代码中通过 `extern 返回值类型 Go函数名称(形参列表);` 声明Go中导出的函数名称
- 注意: `//export Go函数名称` 和 `extern 返回值类型 Go函数名称(形参列表);` 不能在同一个文件中

```

package main

import "C"
import "fmt"
// 导出Go函数声明, 给C使用
//export GoFunction
func GoFunction() {
    fmt.Println("GoFunction!!!")
}

```

```

package main
/*
#include <stdio.h>
// 声明Go中的函数
extern void GoFunction();

void CFunction() {
    printf("CFunction!\n");
    GoFunction();
}
*/
import "C"

```

```
func main() {
    C.CFunction()
}
```

- 由于不在同一个文件, 所以需要通过go build或者go install同时编译多个文件

- Go中使用C语言的类型
- 基本数据类型
 - 在Go中可以用如下方式访问C原生的数值类型:

```
C.char,
C.schar (signed char),
C.uchar (unsigned char),
C.short,
C.ushort (unsigned short),
C.int, C.uint (unsigned int),
C.long,
C.ulong (unsigned long),
C.longlong (long long),
C.ulonglong (unsigned long long),
C.float,
C.double
```

- Go的数值类型与C中的数值类型不是——对应的。因此在使用对方类型变量时必须显式转型操作

```
package main
/*
#include <stdio.h>
int num = 123;
float value = 3.14;
char ch = 'N';
*/
import "C"
import "fmt"

func main() {
    var num1 C.int = C.num
    fmt.Println(num1)
    var num2 int
    //num2 = num1 // 报错
    num2 = int(num1)
    fmt.Println(num2)

    var value1 C.float = C.value
    fmt.Println(value1)
    var value2 float32 = float32(C.value)
    fmt.Println(value2)

    var ch1 C.char = C.ch
    fmt.Println(ch1)
    var ch2 byte = byte(C.ch)
    fmt.Println(ch2)
```

```
}
```

- 字符串类型

- C语言中并没有字符串类型，在C中用带结尾'\0'的字符数组来表示字符串；而在Go中string类型是原生类型，因此在两种语言互操作是必须要做字符串类型的转换
- C字符串转换Go字符串: `C.GoString(str)`
- Go字符串转换C字符串: `C.CString(str)`

```
package main
/*
#include <stdio.h>
char *str = "www.it666.com";
void say(char *name){
    printf("my name is %s", name);
}
*/
import "C"
import (
    "fmt"
    "unsafe"
)

func main() {
    // 1.C语言字符串转换Go语言字符串
    str1 := C.str
    str2 := C.GoString(str1)
    fmt.Println(str2)

    // 2.Go语言字符串转换C语言字符串
    str := "lnj"
    cs := C.CString(str)
    C.say(cs)
    // 注意：转换后所得到的C字符串cs并不能由Go的gc所管理，我们必须手动释放cs所占用的内存
    C.free(unsafe.Pointer(cs))
}
```

- 指针类型

- 原生数值类型的指针类型可按Go语法在类型前面加上*，例如:var p *C.int。
- 而void*比较特殊，用Go中的unsafe.Pointer表示。
 - unsafe.Pointer：通用指针类型，用于转换不同类型的指针，不能进行指针运算
- uintptr：用于指针运算，GC 不把 uintptr 当指针，uintptr 无法持有对象。uintptr 类型的目标会被回收
- 也就是说 unsafe.Pointer 是桥梁，可以让任意类型的指针实现相互转换，也可以将任意类型的指针转换为uintptr 进行指针运算

```
package main
/*
#include <stdio.h>
int num = 123;
void *ptr = &num;
*/
```

```

import "C"
import (
    "fmt"
    "unsafe"
)

func main() {
    // 这是一个C语言变量
    var num C.int = C.num
    // 这是一个C语言指针
    var p1 *C.int = &num
    fmt.Println(*p1)

    //var p2 *C.void = C.ptr // 报错
    // 利用unsafe.Pointer接收void *
    var p2 unsafe.Pointer = C.ptr
    // 将unsafe.Pointer转换为具体类型
    var p3 *C.int = (*C.int)(p2)
    fmt.Println(*p3)
}

```

- 枚举类型

- C语言中的枚举类型在Go语言中的表现形式为C.enum_XXX
- 访问枚举和访问普通变量无异, 直接通过C.枚举值即可

```

package main
/*
#include <stdio.h>
enum Gender {
    GenderMale,
    GenderFemale,
    GenderYao
};
*/
import "C"
import "fmt"

func main() {
    var sex C.enum_Gender = C.GenderMale
    fmt.Println(sex)
    sex = C.GenderFemale
    fmt.Println(sex)
    sex = C.GenderYao
    fmt.Println(sex)
}

```

- 结构体类型

- C语言中的结构体类型在Go语言中的表现形式为C.struct_XXX
- 访问结构体 直接通过 结构体变量.属性名称 即可

```

package main
/*
#include <stdio.h>

```

```

struct Point {
    float x;
    float y;
};
*/
import "C"
import (
    "fmt"
)

func main() {
    // 1.利用C的结构体类型创建结构体
    var cp C.struct_Point = C.struct_Point{6.6, 8.8}
    fmt.Println(cp)
    fmt.Printf("%T\n", cp)

    // 2.将C语言结构体转换为Go语言结构体
    type GoPoint struct {
        x float32
        y float32
    }
    var gp GoPoint
    gp.x = float32(cp.x)
    gp.y = float32(cp.y)
    fmt.Println(gp)
}

```

- 数组类型

- C语言中的数组与Go语言中的数组差异较大，C中的数组是指针类型，Go中的数组是值类型
- 目前似乎无法直接显式的在两者之间进行转型，官方文档也没有说明。

```

package main
/*
#include <stdio.h>
int cArray[5] = {1, 2, 3, 4, 5};
*/
import "C"
import "fmt"

func main() {
    var cArr [5]C.int = C.cArray
    fmt.Println(cArr)
    fmt.Printf("%T\n", cArr)
}

```

- 利用Go语言调用C语言函数, 实现无缓冲区输入
 - 请在终端运行

```

package main
/*
#include <stdio.h>
char lowercase(char ch){
    // 1.判断当前是否是小写字母

```

```

    if(ch >= 'a' && ch <= 'z'){
        return ch;
    }
    // 注意点: 不能直接编写else, 因为执行到else不一定是一个大写字母
    else if(ch >= 'A' && ch <= 'Z'){
        return ch + ('a' - 'A');
    }
    return ' ';
}
char getCh(){
    // 1.接收用户输入的数据
    char ch;
    scanf("%c", &ch);
    setbuf(stdin, NULL);
    // 2.大小写转换
    ch = lowerCase(ch);
    // 3.返回转换好的字符
    return ch;
}
*/
import "C"
import "fmt"

func main() {
    for{
        fmt.Println("请输入w a s d其中一个字符, 控制小人走出迷宫")
        var ch byte = byte(C.getCh())
        fmt.Printf("%c", ch)
    }
}

```

文件的打开和关闭

- 和C语言一样, Go语言中操作文件也是通过一个FILE结构体

```

type file struct {
    pfd      poll.FD
    name     string
    dirinfo  *dirInfo
}
type File struct {
    *file // os specific
}

```

- Open函数
 - func Open(name string) (file *File, err error)
 - Open打开一个文件用于 `读取`
- Close函数
 - func (f *File) Close() error
 - Close关闭文件f

```

package main
import (
    "fmt"
    "os"

```

```

)

func main() {
    // 1.打开一个文件
    // 注意：文件不存在不会创建，会报错
    // 注意：通过Open打开只能读取，不能写入
    fp, err := os.Open("d:/lnj.txt")
    if err != nil{
        fmt.Println(err)
    }else{
        fmt.Println(fp)
    }

    // 2.关闭一个文件
    defer func() {
        err = fp.Close()
        if err != nil {
            fmt.Println(err)
        }
    }()
}

```

文件读取

- Read函数(不带缓冲区去读)
 - func (f *File) Read(b []byte) (n int, err error)
 - Read方法从f中读取最多len(b)字节数据并写入b,

```

package main

import (
    "fmt"
    "io"
    "os"
)

func main() {
    // 1.打开一个文件
    // 注意：文件不存在不会创建，会报错
    // 注意：通过Open打开只能读取，不能写入
    fp, err := os.Open("d:/lnj.txt")
    if err != nil{
        fmt.Println(err)
    }else{
        fmt.Println(fp)
    }

    // 2.关闭一个文件
    defer func() {
        err = fp.Close()
        if err != nil {
            fmt.Println(err)
        }
    }()
}

```

```

// 3.读取指定指定字节个数据
// 注意点: \n也会被读取进来
//buf := make([]byte, 50)
//count, err := fp.Read(buf)
//if err != nil {
//    fmt.Println(err)
//}else{
//    fmt.Println(count)
//    fmt.Println(string(buf))
//}

// 4.读取文件中所有内容, 直到文件末尾为止
buf := make([]byte, 10)
for{
    count, err := fp.Read(buf)
    // 注意: 这行代码要放到判断EOF之前, 否则会出现少读一行的情况
    fmt.Print(string(buf[:count]))
    if err == io.EOF {
        break
    }
}
}

```

- ReadBytes和ReadString函数(带缓冲区去读)
 - func (b *Reader) ReadBytes(delim byte) (line []byte, err error)
 - ReadBytes读取直到第一次遇到delim字节
 - func (b *Reader) ReadString(delim byte) (line string, err error)
 - ReadString读取直到第一次遇到delim字节

```

package main

import (
    "bufio"
    "fmt"
    "io"
    "os"
)

func main() {
    // 1.打开一个文件
    // 注意: 文件不存在不会创建, 会报错
    // 注意: 通过open打开只能读取, 不能写入
    fp, err := os.Open("d:/lnj.txt")
    if err != nil{
        fmt.Println(err)
    }else{
        fmt.Println(fp)
    }

    // 2.关闭一个文件
    defer func() {
        err = fp.Close()
        if err != nil {
            fmt.Println(err)
        }
    }()
}

```

```

// 3. 读取一行数据
// 创建读取缓冲区，默认大小4096
//r :=bufio.NewReader(fp)
//buf, err := r.ReadBytes('\n')
//buf, err := r.ReadString('\n')
//if err != nil{
//    fmt.Println(err)
//}else{
//    fmt.Println(string(buf))
//}

// 4. 读取文件中所有内容，直到文件末尾为止
r :=bufio.NewReader(fp)
for{
    //buf, err := r.ReadBytes('\n')
    buf, err := r.ReadString('\n')
    fmt.Print(string(buf))
    if err == io.EOF{
        break
    }
}
}

```

- ReadFile函数

- func ReadFile(filename string) ([]byte, error)
- 从filename指定的文件中读取数据并返回文件的所有内容
- 不适合大文件读取

```

package main

import (
    "fmt"
    "io/ioutil"
)

func main() {

    filePath := "d:/lnj.txt"
    buf, err := ioutil.ReadFile(filePath)
    if err != nil {
        fmt.Println(err)
    }else{
        fmt.Println(string(buf))
    }
}

```

文件创建和写入

- Create函数

- func Create(name string) (file *File, err error)
- Create采用模式0666（任何人都可读写，不可执行）创建一个名为name的文件
- 如果文件存在会覆盖原有文件

- Write函数

- func (f *File) Write(b []byte) (n int, err error)
- 将指定字节数组写入到文件中
- WriteString函数
 - func (f *File) WriteString(s string) (ret int, err error)
 - 将指定字符串写入到文件中

```
package main

import (
    "fmt"
    "os"
)

func main() {
    // 1.创建一个文件
    fp, err := os.Create("d:/lnj.txt")
    if err != nil {
        fmt.Println(err)
    }
    // 2.关闭打开的文件
    defer func() {
        err := fp.Close()
        if err != nil {
            fmt.Println(err)
        }
    }()
    // 2.往文件中写入数据
    // 注意: windows换行是\r\n
    bytes := []byte{'l', 'n', 'j', '\r', '\n'}
    fp.Write(bytes)

    fp.WriteString("www.it666.com\r\n")
    fp.WriteString("www.itzb.com\r\n")
    // 注意: Go语言采用UTF-8编码, 一个中文占用3个字节
    fp.WriteString("李南江")
}
```

- OpenFile函数
 - func OpenFile(name string, flag int, perm FileMode) (file *File, err error)
 - 第一个参数: 打开的路径
 - 第二个参数: 打开的模式

```
const (
    O_RDONLY int = syscall.O_RDONLY // 只读模式打开文件
    O_WRONLY int = syscall.O_WRONLY // 只写模式打开文件
    O_RDWR  int = syscall.O_RDWR   // 读写模式打开文件
    O_APPEND int = syscall.O_APPEND // 写操作时将数据附加到文件尾部
    O_CREATE int = syscall.O_CREAT  // 如果不存在将创建一个新文件
    O_EXCL   int = syscall.O_EXCL   // 和O_CREATE配合使用, 文件必须不存在
    O_SYNC   int = syscall.O_SYNC   // 打开文件用于同步I/O
    O_TRUNC  int = syscall.O_TRUNC  // 如果可能, 打开时清空文件
)
```

- 第三个参数: 指定权限

- 0没有任何权限
- 1.执行权限(如果是可执行程序, 可以运行)
- 2.写权限
- 3.写权限和执行权限
- 4.读权限
- 5.读权限和执行权限
- 6.读权限和写权限
- 7.读权限和写权限以及执行权限

```
const (
    // 单字符是被String方法用于格式化的属性缩写。
    ModeDir          FileMode = 1 << (32 - 1 - iota) // d: 目录
    ModeAppend                // a: 只能写入, 且只能写入到末尾
    ModeExclusive             // l: 用于执行
    ModeTemporary            // T: 临时文件 (非备份文件)
    ModeSymlink              // L: 符号链接 (不是快捷方式文件)
    ModeDevice               // D: 设备
    ModeNamedPipe            // p: 命名管道 (FIFO)
    ModeSocket               // S: Unix域socket
    ModeSetuid               // u: 表示文件具有其创建者用户id权限
    ModeSetgid               // g: 表示文件具有其创建者组id的权限
    ModeCharDevice           // c: 字符设备, 需已设置
    ModeDevice
    ModeSticky               // t: 只有root/创建者能删除/移动文件
    // 覆盖所有类型位 (用于通过&获取类型位), 对普通文件, 所有这些位都不应被设置
    ModeType = ModeDir | ModeSymlink | ModeNamedPipe | ModeSocket | ModeDevice
    ModePerm FileMode = 0777 // 覆盖所有Unix权限位 (用于通过&获取类型位)
)
```

- 不带缓冲区写入

```
package main

import (
    "fmt"
    "os"
)

func main() {

    // 注意点: 第三个参数在windows没有效果
    // -rw-rw-rw- (666) 所有用户都有文件读、写权限。
    // -rwxrwxrwx (777) 所有用户都有读、写、执行权限。
    // 1. 打开文件
    // fp, err := os.OpenFile("d:/lnj.txt", os.O_CREATE|os.O_RDWR, 0666)
    fp, err := os.OpenFile("d:/lnj.txt", os.O_CREATE|os.O_APPEND, 0666)
    if err != nil {
        fmt.Println(err)
    }
    // 2. 关闭打开的文件
    defer func() {
        err := fp.Close()
    }
}
```

```

        if err != nil {
            fmt.Println(err)
        }
    }()

    // 注意点:
    // 如果O_RDWR模式打开, 被打开文件已经有内容, 会从最前面开始覆盖
    // 如果O_APPEND模式打开, 被打开文件已经有内容, 会在最后追加
    // 3. 往文件中写入数据
    bytes := []byte{'l', 'n', 'j', '\r', '\n'}
    fp.Write(bytes)
    fp.WriteString("www.it666.com\r\n")
}

```

- 带缓冲区写入

```

package main

import (
    "bufio"
    "fmt"
    "os"
)

func main() {

    // 1. 打开文件
    fp, err := os.OpenFile("d:/lnj.txt", os.O_CREATE|os.O_APPEND, 0666)
    if err != nil {
        fmt.Println(err)
    }
    // 2. 关闭打开的文件
    defer func() {
        err := fp.Close()
        if err != nil {
            fmt.Println(err)
        }
    }()

    // 3. 创建缓冲区
    w := bufio.NewWriter(fp)

    // 4. 写入数据到缓冲区
    bytes := []byte{'l', 'n', 'j', '\r', '\n'}
    w.Write(bytes)
    w.WriteString("www.it666.com\r\n")

    // 5. 将缓冲区中的数据刷新到文件
    w.Flush()
}

```

- WriteFile函数

```

package main

import (

```

```

    "fmt"
    "io/ioutil"
)

func main() {

    // 1. 写入数据到指定文件
    data := []byte{'l', 'n', 'j', '\r', '\n'}
    err := ioutil.WriteFile("d:/abc.txt", data, 0666)
    if err != nil {
        fmt.Println(err)
    } else {
        fmt.Println("写入成功")
    }
}

```

判断文件是否存在

- Stat函数
 - func Stat(name string) (fi FileInfo, err error)
- 返回值: FileInfo

```

type FileInfo interface {
    Name() string           // 文件的名称（不含扩展名）
    Size() int64            // 普通文件返回值表示其大小；其他文件的返回值含义各系统不同
    Mode() FileMode        // 文件的模式位
    ModTime() time.Time    // 文件的修改时间
    IsDir() bool           // 等价于 Mode().IsDir()
    Sys() interface{}      // 底层数据来源（可以返回nil）
}

```

- 返回值: error
 - 返回值error等于nil, 代表文件存在
 - 返回值error不等于nil, 可以进一步通过IsNotExist判断, 如果返回true代表文件不存在
 - 返回值error如果返回其它错误, 则不确定文件是否存在

```

package main

import (
    "fmt"
    "os"
)

func main() {

    info, err := os.Stat("d:/lnj.txt")
    if err == nil {
        fmt.Println("文件存在")
        fmt.Println(info.Name())
    } else if os.IsNotExist(err) {
        fmt.Println("文件不存在")
    } else {
        fmt.Println("不确定")
    }
}

```

练习

- 将一个文本文件拷贝到另外一个文件中
 - 尝试用上面学习的其它方法实现下

```
package main

import (
    "fmt"
    "io/ioutil"
)

func main() {

    // 1. 读取一个文件
    buf, err := ioutil.ReadFile("d:/lnj.txt")
    if err != nil {
        fmt.Println(err)
        return
    }
    // 2. 写入读取的数据到另一个文件
    err = ioutil.WriteFile("d:/abc.txt", buf, 0666)
    if err != nil {
        fmt.Println(err)
        return
    }
    fmt.Println("拷贝完成")
}
```

- 将一个图片/视频文件拷贝到另一个文件

```
package main

import (
    "bufio"
    "fmt"
    "io"
    "os"
)

func main() {

    // 1. 定义拷贝文件的路径
    scrPath := "D:/a.png"
    destPath := "E:/b.png"
    // 2. 打开被拷贝文件
    fr, err := os.Open(scrPath)
    if err != nil {
        fmt.Println(err)
        return
    }
    // 3. 关闭打开文件
    defer func() {
```

```

    err := fr.Close()
    if err != nil{
        fmt.Println(err)
    }
}()
// 4.创建读取缓冲区
r := bufio.NewReader(fr)

// 1.创建写入文件
fw, err := os.Create(destPath)
if err != nil {
    fmt.Println(err)
    return
}
// 2.关闭打开文件
defer func() {
    err := fw.Close()
    if err != nil{
        fmt.Println(err)
    }
}()
// 3.创建写入缓冲区
w := bufio.NewWriter(fw)
// 4.利用系统copy函数完成拷贝
count, err := io.Copy(w, r)
if err != nil {
    fmt.Println(err)
    return
}
fmt.Println(count)
fmt.Println("拷贝完成")
}

```

- 自己查文档实现遍历文件夹
 - 例如: 给一个文件夹路径, 获取该文件夹下所有文件, 并将所有文件路径保存到切片中

并发编程基本概念

- 学习并发编程之前我们需要脑补几个基础知识和思考一个问题
 - 什么是串行?
 - 什么是并行?
 - 什么是并发?
 - 什么是程序?
 - 什么是进程?
 - 什么是线程?
 - 什么是协程?

什么是串行?

- 串行就是按顺序执行, 就好比银行只有1个窗口, 有3个人要办事, 那么必须排队, 只有前面的人办完走人, 才能轮到你
- 在计算机中, **同一时刻, 只能有一条指令, 在一个CPU上执行, 后面的指令必须等到前面指令执行完才能执行**, 就是串行

+



什么是并行？

- 并行就是同时执行, 就好比银行有3个窗口, 有3个人要办事, 只需要到空窗口即可立即办事.
- 在计算机中, **同一时刻, 有多条指令, 在多个CPU上执行**, 就是并行
- 从以上分析不难看出, 并行的速度优于串行

+



什么是并发？

- 并发是伪并行, 就好比银行只有1个窗口, 有3个人要办事, 那么没轮到后面的人时, 后面的人可以用拖鞋先排队, 去吃个早餐, 买个东西啥的, 感觉差不多要到自己时再回来办事
- 在计算机中, **同一时刻, 只能有一条指令, 在一个CPU上执行, 但是CPU会快速的在多条指令之间轮询执行**就是并发
- 并行和并发的区别就好比古代的三妻四妾(名正言顺, 光明正大)和现代三妻四妾(抽空幽会, 小三小四)



- 总结:
 - 多线程程序在单核上运行, 就是并发
 - 多线程程序在多核上运行, 就是并行

什么是程序?

- 程序 是指将 编译型语言 编写好的代码通过编译工具编译之后 存储在硬盘 上的一个 二进制文件, 会占用磁盘空间, 但不会占用系统资源
 - 例如我们通过C++编写了一个聊天程序, 然后通过C++编译器将编写好的代码编译成一个二进制的文件, 那么这个二进制的文件就是一个程序

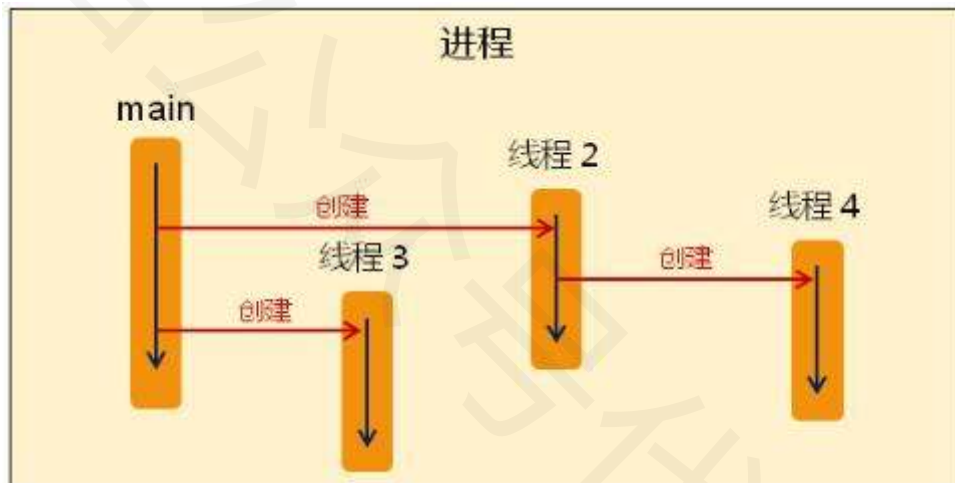
什么是进程?

- 进程 是指 程序 在操作系统中的一次执行过程, 是系统进行资源分配和调度的基本单位
- 例如:
 - 启动记事本这个程序, 在系统中就会创建一个记事本进程

- 再次启动记事本这个程序, 又会在系统中创建一个记事本进程
- 程序和进程的关系就好比剧本和演出的关系
 - 剧本对应程序, 演出对应进程. 同一个剧本可以在多个舞台同时演出互不影响, 同一个程序可以在系统中开启多个进程互不影响
- 所以程序和进程的关系是1:N, 所以多个进程的空间是独立的

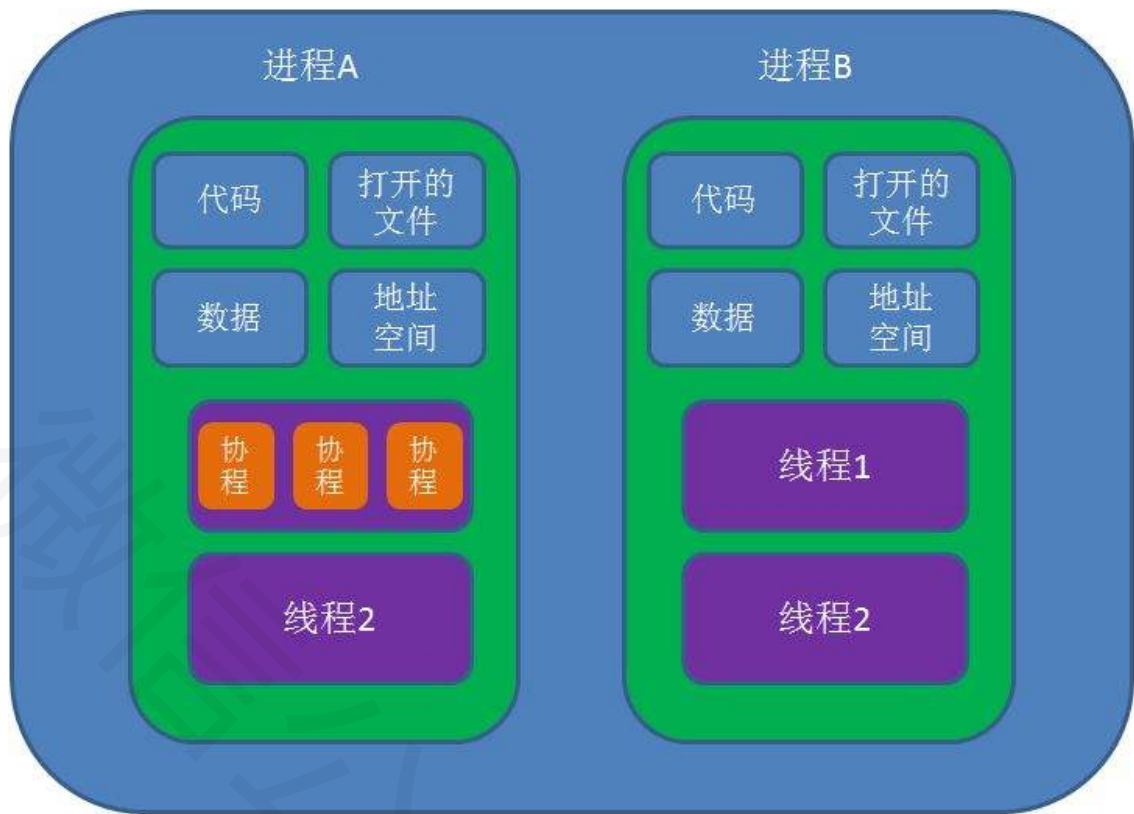
什么是线程?

- 线程是指进程中的一个执行实例, 是程序执行的最小单元, 它是比进程更小的能独立运行的基本单位
- 一个进程中至少有一个线程, 这个线程我们称之为 **主线程**
- 一个进程中除了 **主线程** 以外, 我们还可以创建和销毁多个线程
- 例如:
 - 启动迅雷这个程序, 系统会创建一个 **迅雷进程**, 并且默认会有一个 **主线程**, 用于执行迅雷默认的业务逻辑
 - 当我们利用迅雷下载 多个任务 的时候, 会发现多个任务都在 **同时下载**, 此时为了能够 **同时执行** 下载操作, 迅雷就会创建多个线程, 将不同的下载任务放到不同的线程中执行



什么是协程?

- 协程是一种用户态的轻量级线程, 又称微线程, 英文名Coroutine
- 与传统的系统级别进程和线程相比, 协程最大的优势在于"轻量级". 可以轻松创建上万个不会导致系统资源衰竭. 而线程和进程通常很难超过1万个. 这也是协程称之为"轻量级线程"的原因
- 一个线程中可以有任意多个协程, 但 **某一时刻只能有一个协程在运行**, 多个协程分享所在线程分配到的计算机资源



操作系统

- 在协程中, 调用一个任务就像调用一个函数一样, 消耗系统资源极少, 但能达到进程、线程相同的并发效果

Go并发

- Go在语言级别支持协程(多数语言在语法层面并不直接支持协程), 叫做goroutine.
- 人们把Go语言称之为21世纪的C语言. 第一是因为Go语言设计简单, 第二是因为21世纪最重要的就是并行程序设计. 而Go从语言层面就支持并发和并行
- Go并发小案例

```
package main

import (
    "fmt"
    "time"
)

func sing() {
    for i:=0; i< 10; i++){
        fmt.Println("我在唱歌")
        time.Sleep(time.Millisecond)
    }
}

func dance() {
    for i:=0; i< 10; i++){
        fmt.Println("我在跳舞---")
        time.Sleep(time.Millisecond)
    }
}
```

```
func main() {
    // 串行：必须先唱完歌才能跳舞
    //sing()
    //dance()

    // 并行：可以边唱歌，边跳舞
    // 注意点：主线程不能死，否则程序就退出了
    go sing() // 开启一个协程
    go dance() // 开启一个协程
    for{
        ;
    }
}
```

- runtime包中常用的函数
 - Gosched:使当前go程放弃处理器，以让其它go程运行

```
package main
import (
    "fmt"
    "runtime"
)

func sing() {
    for i:=0; i< 10; i++){
        fmt.Println("我在唱歌")
        // Gosched使当前go程放弃处理器，以让其它go程运行。
        // 它不会挂起当前go程，因此当前go程未来会恢复执行
        runtime.Gosched()
    }
}

func dance() {
    for i:=0; i< 10; i++){
        fmt.Println("我在跳舞---")
        runtime.Gosched()
    }
}

func main() {

    go sing()
    go dance()
    for{
        ;
    }
}
```

- Goexit: 终止调用它的go程, 其它go程不会受影响

```
package main
import (
    "fmt"
    "runtime"
)

func main() {
```

```

go func() {
    fmt.Println("123")
    // 退出当前协程
    //runtime.Goexit()
    // 退出当前函数
    //return
    test()
    fmt.Println("456")
}()
for{
    ;
}

func test() {
    fmt.Println("abc")
    // 只会结束当前函数，协程中的其它代码会继续执行
    //return
    // 会结束整个协程，Goexit之后整个协程中的其它代码不会执行
    runtime.Goexit()
    fmt.Println("def")
}

```

- NumCPU: 返回本地机器的逻辑CPU个数

```

package main
import (
    "fmt"
    "runtime"
)
func main() {
    num := runtime.NumCPU()
    fmt.Println(num)
}

```

- GOMAXPROCS: 设置可同时执行的最大CPU数，并返回先前的设置
 - Go语言1.8之前, 需要我们手动设置
 - Go语言1.8之后, 不需要我们手动设置

```

func main() {

```

```

// 获取带来了CPU个数
num := runtime.NumCPU()
// 设置同时使用CPU个数
runtime.GOMAXPROCS(num)
}

```

多线程同步问题

- 互斥锁
 - + 互斥锁的本质是当一个goroutine访问的时候，其它goroutine都不能访问
 - + 这样就能实现资源同步，但是在避免资源竞争的同时也降低了程序的并发性能。程序由原来的并发执行变成了串行
- 案例：

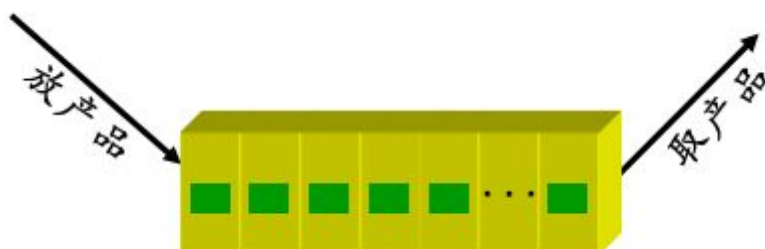
- + 有一个打印函数，用于逐个打印字符串中的字符，有两个人都开启了goroutine去打印
- + 如果没有添加互斥锁，那么两个人都有机会输出自己的内容
- + 如果添加了互斥锁，那么会先输出某一个的，输出完毕之后再输出另外一个人的

```
``go
package main
import (
    "fmt"
    "sync"
    "time"
)
// 创建一把互斥锁
var lock sync.Mutex

func printer(str string) {
    // 让先来的人拿到锁，把当前函数锁住，其它人都无法执行
    // 上厕所关门
    lock.Lock()
    for _, v := range str{
        fmt.Printf("%c", v)
        time.Sleep(time.Millisecond * 500)
    }
    // 先来的人执行完毕之后，把锁释放掉，让其它人可以继续使用当前函数
    // 上厕所开门
    lock.Unlock()
}
func person1() {
    printer("hello")
}
func person2() {
    printer("world")
}
func main() {
    go person1()
    go person2()
    for{
        ;
    }
}
```

生产者消费者问题

- 所谓的生产者消费者模型就是
 - 某个模块(函数)负责生产数据, 这些数据由另一个模块来负责处理
 - 一般生产者消费者模型包含三个部分"生产者"、"缓冲区"、"消费者"



- 为什么生产者消费者模型要含三个部分？直接生产和消费不行么？
- 一个案例说明一切

- 生产者好比现实生活中的某个人
- 缓冲区好比现实生活中的邮箱
- 消费者好比现实生活中的邮递员
- 如果只有生产者和消费者, 那么相当于只有写信的人和邮递员, 那么如果将来过去的邮递员离职了, 你想邮寄信件必须想办法结识新的邮递员(消费者发生变化, 会直接影响生产者, 耦合性太强)
- 如果在生产者和消费者之间添加一个缓冲区, 那么就好比有了邮箱, 以后邮寄信件不是找邮递员, 只需把信件投递到邮箱中即可, 写信的人不需要关心邮递员是谁(解耦)
- 如果只有生产者和消费者, 那么每个人邮寄信件都需要直接找邮递员(1对1关系), 如果有10个人要邮寄信件, 那么邮递员只能依次找到每个人, 然后才能取件(效率低下)
- 如果在生产者和消费者之间添加一个缓冲区, 那么所有的人只需要将信件投递到邮箱即可, 邮递员不用关心有多少人要邮寄信件, 也不用依次取件, 只需要找到邮箱从邮箱中统一取件即可(效率提高)
- 如果只有生产者和消费者, 那么如果邮寄信件太多邮递员无法一次拿走, 这个时候非常难办
- 如果在生产者和消费者之间添加一个缓冲区, 那么如果信件太多可以先拿走一部分, 剩下的继续放到邮箱中下次再拿
-

生产者和消费者资源竞争问题

- 例如生产比较慢, 而消费比较快, 就会导致消费者消费到错误数据

```
package main

import (
    "fmt"
    "math/rand"
    "sync"
    "time"
)

// 创建一把互斥锁
var lock = sync.Mutex{}

// 定义缓冲区
var scc []int = make([]int, 10)

// 定义生产者
func producer(){
    // 加锁, 注意是lock就是我们的锁, 全局公用一把锁
    lock.Lock()
    rand.Seed(time.Now().UnixNano())
    for i:=0;i<10;i++){
        num := rand.Intn(100)
        scc[i] = num
        fmt.Println("生产者生产了: ", num)
        time.Sleep(time.Millisecond * 500)
    }
    // 解锁
    lock.Unlock()
}

// 定义消费者
func consumer() {
    // 加锁, 注意和生产者中用的是同一把锁
    // 如果生产者中已加过了, 则阻塞直到解锁后再重新加锁
}
```

```

lock.Lock()
for i:=0;i<10;i++){
    num := sce[i]
    fmt.Println("---消费者消费了", num)
}
lock.Unlock()
}

func main() {
    go producer()
    go consumer()
    for{
        ;
    }
}

```

- 思考: 那如果是一对多, 或者多对多的关系, 上述代码有问题么?

管道(Channel)

- 上述实现并发的代码中为了保持主线程不挂掉, 我们都会在最后写上一个死循环或者写上一个定时器来实现等待goroutine执行完毕
- 上述实现并发的代码中为了解决生产者消费者资源同步问题, 我们利用加锁来解决, 但是这仅仅是一对一的情况, 如果是一对多或者多对多, 上述代码还是会出现问题
- 综上所述, 企业开发中需要一种更牛X的技术来解决上述问题, 那就是 **管道(Channel)**

- Channel的本质是一个队列



- Channel是线程安全的, 也就是自带锁定功能
- Channel声明和初始化
 - 声明: `var 变量名chan 数据类型`
 - 初始化: `mych := make(chan 数据类型, 容量)`
 - Channel和切片还有字典一样, 必须make之后才能使用
 - Channel和切片还有字典一样, 是引用类型

```

package main
import "fmt"
func main() {
    // 1. 声明一个管道
    var mych chan int
    // 2. 初始化一个管道
    mych = make(chan int, 3)
    // 3. 查看管道的长度和容量
    fmt.Println("长度是", len(mych), "容量是", cap(mych))
    // 4. 像管道中写入数据
    mych<- 666
    fmt.Println("长度是", len(mych), "容量是", cap(mych))
}

```

```
// 5.取出管道中写入的数据
num := <-mych
fmt.Println("num = ", num)
fmt.Println("长度是", len(mych), "容量是", cap(mych))
}
```

- 注意点:

- 管道中只能存放声明的数据类型, 不能存放其它数据类型
- 管道中如果已经没有数据, 再取就会报错
- 如果管道中数据已满, 再写入就会报错

```
package main

import "fmt"

func main() {
    // 1.声明一个管道
    var mych chan int
    // 2.初始化一个管道
    mych = make(chan int, 3)

    // 注意点: 管道中只能存放声明的数据类型, 不能存放其它数据类型
    //mych<-3.14

    // 注意点: 管道中如果已经没有数据,
    // 并且检测不到有其它协程再往管道中写入数据, 那么再取就会报错
    //num = <-mych
    //fmt.Println("num = ", num)

    // 注意点: 如果管道中数据已满, 再写入就会报错
    mych<- 666
    mych<- 777
    mych<- 888
    mych<- 999
}
```

- 管道的关闭和遍历

```
package main

import "fmt"

func main() {
    // 1.创建一个管道
    mych := make(chan int, 3)
    // 2.往管道中存入数据
    mych<-666
    mych<-777
    mych<-888
    // 3.遍历管道
    // 第一次遍历i等于0, len = 3,
    // 第二次遍历i等于1, len = 2
    // 第三次遍历i等于2, len = 1
}
```

```

//for i:=0; i<len(myCh); i++){
//  fmt.Println(<-myCh) // 输出结果不正确
//}

// 3.写入完数据之后先关闭管道
// 注意点：管道关闭之后只能读不能写
close(myCh)
//myCh<- 999 // 报错

// 4.遍历管道
// 利用for range遍历，必须先关闭管道，否则会报错
//for value := range myCh{
//  fmt.Println(value)
//}

// close主要用途：
// 在企业开发中我们可能不确定管道有还没有有数据，所以我们可能一直获取
// 但是我们可以通过ok-idiom模式判断管道是否关闭，如果关闭会返回false给ok
for{
    if num, ok:= <-myCh; ok{
        fmt.Println(num)
    }else{
        break;
    }
}
fmt.Println("数据读取完毕")
}

```

- Channel阻塞现象

- 单独在主线程中操作管道, 写满了会报错, 没有数据去获取也会报错
- 只要在协程中操作管道过, 写满了就会阻塞, 没有就数据去获取也会阻塞

```

package main
import (
    "fmt"
    "time"
)
// 创建一个管道
var myCh = make(chan int, 5)
func demo() {
    var myCh = make(chan int, 5)
    //myCh<-111
    //myCh<-222
    //myCh<-333
    //myCh<-444
    //myCh<-555
    //fmt.Println("我是第六次添加之前代码")
    //myCh<-666
    //fmt.Println("我是第六次添加之后代码")

    fmt.Println("我是第六次直接获取之前代码")
    <-myCh
    fmt.Println("我是第六次直接获取之后代码")
}
func test() {

```

```

//myCh<-111
//myCh<-222
//myCh<-333
//myCh<-444
//myCh<-555
//fmt.Println("我是第六次添加之前代码")
//myCh<-666
//fmt.Println("我是第六次添加之后代码")

//fmt.Println("我是第六次直接获取之前代码")
//<-myCh
//fmt.Println("我是第六次直接获取之后代码")
}
func example() {
    time.Sleep(time.Second * 2)
    myCh<-666
}
func main() {
    // 1. 同一个go程中操作管道
    // 写满了会报错
    //myCh<-111
    //myCh<-222
    //myCh<-333
    //myCh<-444
    //myCh<-555
    //myCh<-666

    // 没有了去取也会报错
    //<-myCh

    // 2. 在协程中操作管道
    // 写满了不会报错，但是会阻塞
    //go test()

    // 没有了去取也不会报错，也会阻塞
    //go test()

    //go demo()
    //go demo()

    // 3. 只要在协程中操作了管道，就会发生阻塞现象
    go example()
    fmt.Println("myCh之前代码")
    <-myCh
    fmt.Println("myCh之后代码")

    //for{
    //    ;
    //}
}

```

- 利用Channel实现生产者消费者

```
package main
```

```
import (
```

```

    "fmt"
    "math/rand"
    "time"
)
// 定义缓冲区
var myCh = make(chan int, 5)
var exitCh = make(chan bool, 1)

// 定义生产者
func producer(){
    rand.Seed(time.Now().UnixNano())
    for i:=0;i<10;i++){
        num := rand.Intn(100)
        fmt.Println("生产者生产了: ", num)
        // 往管道中写入数据
        myCh<-num
        //time.Sleep(time.Millisecond * 500)
    }
    // 生产完毕之后关闭管道
    close(myCh)
    fmt.Println("生产者停止生产")
}

// 定义消费者
func consumer() {
    // 不断从管道中获取数据，直到管道关闭位置
    for{
        if num, ok := <-myCh; !ok{
            break
        }else{
            fmt.Println("---消费者消费了", num)
        }
    }
    fmt.Println("消费者停止消费")
    exitCh<-true
}

func main() {
    go producer()
    go consumer()
    fmt.Println("exitCh之前代码")
    <-exitCh
    fmt.Println("exitCh之后代码")
}

```

- 无缓冲Channel

```

package main
import "fmt"
var myCh1 = make(chan int, 5)
var myCh2 = make(chan int, 0)
func main() {
    // 有缓冲管道
    // 只写入，不读取不会报错
    //myCh1<-1
    //myCh1<-2
    //myCh1<-3
}

```

```

//myCh1<-4
//myCh1<-5
//fmt.Println("len =",len(myCh1), "cap =", cap(myCh1))

// 无缓冲管道
// 只有两端同时准备好才不会报错
go func() {
    fmt.Println(<-myCh2)
}()
// 只写入，不读取会报错
myCh2<-1
//fmt.Println("len =",len(myCh2), "cap =", cap(myCh2))
// 写入之后在同一个线程读取也会报错
//fmt.Println(<-myCh2)
// 在主程中先写入，在子程中后读取也会报错
//go func() {
//    fmt.Println(<-myCh2)
//}()
}

```

- 无缓冲Channel和有缓冲Channel
 - 有缓冲管道具备异步的能力(写几个读一个或读几个)
 - 无缓冲管道具备同步的能力(写一个读一个)

```

package main
import (
    "fmt"
    "math/rand"
    "time"
)
// 定义缓冲区
//var myCh = make(chan int, 0)
var myCh = make(chan int)
var exitCh = make(chan bool, 1)

// 定义生产者
func producer(){
    rand.Seed(time.Now().UnixNano())
    for i:=0;i<10;i++){
        num := rand.Intn(100)
        fmt.Println("生产者生产了: ", num)
        // 往管道中写入数据
        myCh<-num
        //time.Sleep(time.Millisecond * 500)
    }
    // 生产完毕之后关闭管道
    close(myCh)
    fmt.Println("生产者停止生产")
}

// 定义消费者
func consumer() {
    // 不断从管道中获取数据，直到管道关闭位置
    for{
        if num, ok := <-myCh; !ok{
            break

```

```

    }else{
        fmt.Println("---消费者消费了", num)
    }
}
fmt.Println("消费者停止消费")
exitCh<-true
}

func main() {
    go producer()
    go consumer()
    fmt.Println("exitCh之前代码")
    <-exitCh
    fmt.Println("exitCh之后代码")
}

```

IO的延迟说明:

看到的输出结果和我们想象的不太一样, 是因为IO输出非常消耗性能, 输出之后还没来得及赋值可能就跑去执行别的协程了

- 单向管道和双向管道
 - 默认情况下所有管道都是双向了(可读可写)
 - 但是在企业开发中, 我们经常需要用到将一个管道作为参数传递
 - 在传递的过程中希望对方只能单向使用, 要么只能写, 要么只能读
- 双向管道
 - `var myCh chan int = make(chan int, 0)`
- 单向管道
 - `var myCh chan<- int = make(chan<- int, 0)`
 - `var myCh <-chan int = make(<-chan int, 0)`
- 注意点:
 - 双向管道可以自动转换为任何一种单向管道
 - 单向管道不能转换为双向管道

```

package main

import "fmt"

func main() {
    // 1. 定义一个双向管道
    var myCh chan int = make(chan int, 5)

    // 2. 将双向管道转换单向管道
    var myCh2 chan<- int
    myCh2 = myCh
    fmt.Println(myCh2)
    var myCh3 <-chan int
    myCh3 = myCh
    fmt.Println(myCh3)

    // 3. 双向管道, 可读可写
    myCh<-1
    myCh<-2
    myCh<-3
}

```

```

fmt.Println(<-myCh)

// 3.只写管道,只能写,不能读
// myCh2<-666
// fmt.Println(<-myCh2)

// 4.指读管道,只能读,不能写
fmt.Println(<-myCh3)
//myCh3<-666

// 注意点: 管道之间赋值是地址传递, 以上三个管道底层指向相同容器
}

```

- 单向管道作为函数参数

```

package main
import (
    "fmt"
    "math/rand"
    "time"
)
// 定义生产者
func producer(myCh chan<- int){
    rand.Seed(time.Now().UnixNano())
    for i:=0;i<10;i++){
        num := rand.Intn(100)
        fmt.Println("生产者生产了: ", num)
        // 往管道中写入数据
        myCh<-num
        //time.Sleep(time.Millisecond * 500)
    }
    // 生产完毕之后关闭管道
    close(myCh)
    fmt.Println("生产者停止生产")
}
// 定义消费者
func consumer(myCh <-chan int) {
    // 不断从管道中获取数据, 直到管道关闭位置
    for{
        if num, ok := <-myCh; !ok{
            break
        }else{
            fmt.Println("---消费者消费了", num)
        }
    }
    fmt.Println("消费者停止消费")
}

func main() {
    // 定义缓冲区
    var myCh = make(chan int, 5)
    go producer(myCh)
    consumer(myCh)
}

```

select选择结构

- select是Go中的一个控制结构，类似于switch语句，用于处理异步IO操作
 - 如果有多个case都可以运行，select会随机选出一个执行，其他不会执行。
 - 如果没有可运行的case语句，且有default语句，那么就会执行default的动作。
 - 如果没有可运行的case语句，且没有default语句，select将阻塞，直到某个case通信可以运行

```
select {
case IO操作1:
    IO操作1读取或写入成功就执行
case IO操作2:
    IO操作2读取或写入成功就执行
default:
    如果上面case都没有成功，则进入default处理流程
}
```

- 注意点:
 - select的case后面必须是一个IO操作
 - 一般情况下使用select结构不用写default

```
package main

import (
    "fmt"
    "time"
)

func main() {
    // 创建管道
    var myCh = make(chan int)
    var exitCh = make(chan bool)

    // 生产数据
    go func() {
        for i:=0;i <10;i++){
            myCh<-i
            time.Sleep(time.Second)
        }
        //close(myCh)
        exitCh<-true
    }()

    // 读取数据
    for{
        fmt.Println("读取代码被执行了")
        select {
        case num:= <-myCh:
            fmt.Println("读到了", num)
        case <-exitCh:
            //break // 没用，跳出的是select
            return
        }
        fmt.Println("-----")
    }
}
```

- select应用场景
 - 实现多路监听
 - 实现超时处理

```
package main
import (
    "fmt"
    "runtime"
    "time"
)

func main() {
    // 1.创建管道
    myCh := make(chan int, 5)
    exitCh := make(chan bool)

    // 2.生成数据
    go func() {
        for i:=0; i<10; i++ {
            myCh<-i
            time.Sleep(time.Second * 3)
        }
    }()

    // 3.获取数据
    go func() {
        for{
            select {
            case num:= <-myCh:
                fmt.Println(num)
            case <-time.After(time.Second * 2):
                exitCh<-true
                runtime.Goexit()
            }
        }
    }()

    <-exitCh
    fmt.Println("程序结束")
}
```

定时器补充

- 一次性定时器
- NewTimer函数
 - func NewTimer(d Duration) *Timer
 - NewTimer创建一个Timer，它会在到期后向Timer自身的C字段发送当时的时间

```
type Timer struct {
    C <-chan Time // 对于我们来说，这个属性是只读的管道
    r runtimeTimer
}
```

```

package main
import (
    "fmt"
    "time"
)
func main() {
    start := time.Now()
    fmt.Println("开始时间", start)
    timer := time.NewTimer(time.Second * 3)
    fmt.Println("读取之前代码被执行")
    end := <-timer.C // 系统写入数据之前会阻塞
    fmt.Println("读取之后代码被执行")
    fmt.Println("结束时间", end)
}

```

- After函数
 - func After(d Duration) <-chan Time
 - 底层就是对NewTimer的封装, 只不过返回值不同而已

```

func After(d Duration) <-chan Time {
    return NewTimer(d).C
}

```

```

package main
import (
    "fmt"
    "time"
)
func main() {
    start := time.Now()
    fmt.Println("开始时间", start)
    timer := time.After(time.Second * 3)
    fmt.Println("读取之前代码被执行")
    end := <-timer // 系统写入数据之前会阻塞
    fmt.Println("读取之后代码被执行")
    fmt.Println("结束时间", end)
}

```

-
- 周期性定时器
 - NewTicker函数
 - func NewTicker(d Duration) *Ticker
 - 和NewTimer差不多, 只不过NewTimer只会往管道中写入一次数据, 而NewTicker每隔一段时间就会写一次

```

type Ticker struct {
    C <-chan Time // 周期性传递时间信息的通道
    // 内含隐藏或非导出字段
}

```

```

package main
import (
    "fmt"

```

```
"time"
)
func main() {
    // 1.创建一个周期定时器
    ticker := time.NewTicker(time.Second)
    // 2.不断从重启定时器中获取时间
    for{
        t := <-ticker.C // 系统写入数据之前会阻塞
        fmt.Println(t)
    }
}
```

声明：未经授权请勿转载，转载请在公众号 代码情缘 获取授权，侵权必究。

码字不易，你的转发就是对我最大的支持。

扫码关注获取资料



回复 C语言代码 获取配套代码

回复 书籍 获取瓜哥精华电子书

回复 微信 添加瓜哥微信交流